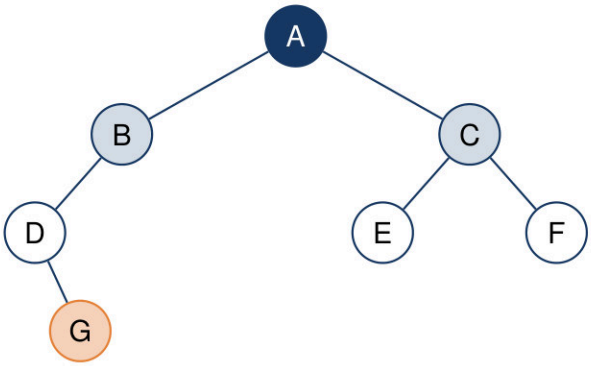
《数据结构与算法》　第 6 章

树与二叉树

可视化基础讲义 · 期中复习版

从“为什么需要树”到哈夫曼编码：图解、推导、C++ 实现与 38 道分层练习



先序 A B D G C E F　中序 D G B A E C F　后序 G D B E F C A　层次 A B C D E F G

本讲义覆盖（与课件编号对应）

6.1	树的定义、基本术语、森林、与线性结构比较
6.2	二叉树定义、五种形态、性质 1–5、满/完全二叉树、顺序与链式存储
6.3	先序/中序/后序/层次遍历、表达式树、由序列构造二叉树、遍历应用
6.5	双亲/孩子链表/孩子兄弟表示、树与森林和二叉树的转换、树与森林的遍历
6.6	路径长度、WPL、哈夫曼树构造与性质、前缀编码与哈夫曼编码

所有程序语言均为 C++；所有代码已用 g++ 实际编译运行。

目录

导学：这份讲义怎么用 1

本专题在课程体系中的位置 1

学科小史：这个知识是怎样出现的？ 1

从课本到现实：这个知识到底有什么用？ 2

原始资料的重新编码与纠错 3

1　树的基本概念[课件 6.1 · 笔记 p578–580] 4

1.1　为什么需要树：线性结构装不下“层次” . . . 4
1.2　树的定义：用“树”来定义“树” . . . 4
1.3　树的四种表示法 . . . 5
1.4　基本术语：一张图看懂 14 个名词 . . . 5
1.5　有序树、森林，以及树和森林的关系 . . . 6
1.6　与线性结构的比较 . . . 7

2　二叉树：定义、形态与性质[课件 6.2.1–6.2.2 · 笔记 p580–585] 7

2.1　为什么偏偏是“二”叉树 . . . 8
2.2　二叉树的抽象数据类型与基本操作[课件 p13–18 · 笔记 p582–583] . . . 9
2.3　二叉树的五条性质[课件 p19–24] . . . 10
2.4　两种特殊形态：满二叉树与完全二叉树[课件 p22 · 笔记 p584] . . . 12

3　二叉树的存储结构[课件 6.2.3 · 笔记 p585–587] 14

3.1　顺序存储：用下标表示父子关系 . . . 15
3.2　链式存储：用指针显式记录关系 . . . 16

4　遍历二叉树[课件 6.3.1–6.3.2 · 笔记 p587–593] 18

4.1　遍历的定义与六种次序 . . . 18
4.2　先序、中序、后序遍历 . . . 18
4.3　层次遍历：借助队列[课件 p68–70 · 笔记 p593] . . . 20
4.4　用二叉树表示表达式[课件 p61–68 · 笔记 p590] . . . 21

5　遍历的应用[课件 6.3.3 · 笔记 p590–594] 22

5.1　由遍历序列确定二叉树 . . . 23
5.2　统计、求高、复制：把“访问”换成别的事 . . . 25
5.3　二叉树的计数：卡特兰数[课件 p74–75] . . . 26

6　树和森林[课件 6.5 · 笔记 p594–601] 27

6.1　树的三种存储结构 . . . 27

6.2　树与二叉树的转换[笔记 p596–598 · 课件 p90–91] 29
6.3　森林与二叉树的转换[笔记 p598–599 · 课件 p94–96] 30
6.4　树和森林的遍历[课件 6.5.3–6.5.4 · 笔记 p600–601] 31

7　哈夫曼树与哈夫曼编码[课件 6.6 · 笔记 p601–605]　32
7.1　路径长度与带权路径长度 33
7.2　哈夫曼树及其构造算法 34
7.3　前缀编码与哈夫曼编码[课件 6.6.3 · 笔记 p604–605] 36

8　全章知识地图与总结　38
8.1　本章知识地图 38
8.2　高频易错点总表 38
8.3　基础解题步骤总结 38
8.4　相近概念总对比 39
8.5　跨学科 / 相似知识总对比表 39

9　正式练习（38 道）　40
9.1　A. 基础巩固训练（25 道） 40
9.2　B. 中等偏下提升训练（10 道） 43
9.3　C. 跨学科迁移训练（3 道） 44

10　答案速查　45

11　详细解析　45
11.1　基础单选题解析 45
11.2　基础多选题解析 47
11.3　基础填空题解析 48
11.4　基础解答题解析 49
11.5　提升单选题解析 51
11.6　提升多选题解析 51
11.7　提升填空题解析 52
11.8　提升综合大题解析 52
11.9　跨学科迁移题解析 54

12　后续课程连接　55

附录 A　考试代码合集（C++，已编译运行）　55

附录 B　拓展：非递归中序遍历（课件附录，标注“不讲”）　61

附录 C　参考资料　62

导学：这份讲义怎么用

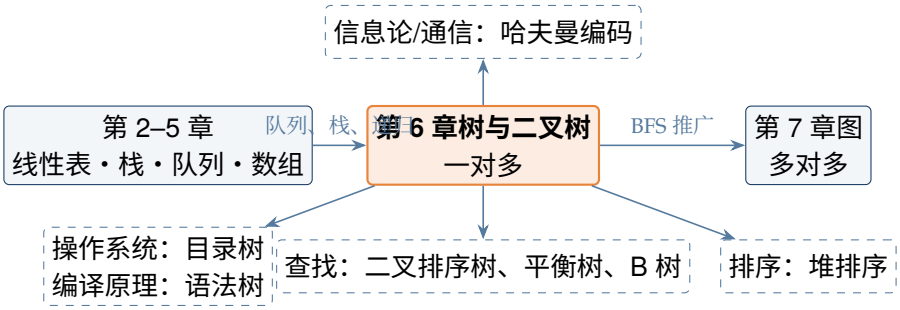
本讲义面向**第一次系统学习“树与二叉树”、准备期中考试**的同学。它不是课件的翻印，而是把课件、课堂笔记和课程源程序重新组织成一条连续的学习路线：每个知识点都按**“为什么需要 → 先看图 → 正式定义 → 公式/算法 → 推导或证明 → 最简单例子 → 易错点 → 相似知识对比 → 后续联系”**展开。



- **颜色有固定含义**：蓝 = 必须掌握的核心知识；红 = 易错与纠错；米色 = 例题、生活应用、历史与科普；绿 = 总结与自检；淡紫框 = **教学补全**（课件或笔记里省略、由本讲义依据标准教材补充的内容）。
- **正文例题不计入正式练习**。正式练习共 **38 道**：基础 25 + 提升 10 + 跨学科迁移 3，答案集中放在练习之后，请先独立作答。
- **约定**：本讲义采用课件与国内教材的口径——**根结点在第 1 层**，空树深度（高度）为 0；顺序存储**从下标 1 开始编号**（与课程源程序 `SqBinaryTree` 一致），从 0 编号的公式会单独对照给出。
- **考试范围提示**（来自课件第 2 页）：6.3 的非递归遍历、6.6 的哈夫曼树程序实现课上不讲。本讲义把它们放在“拓展”中，只要求理解思路。

本专题在课程体系中的位置

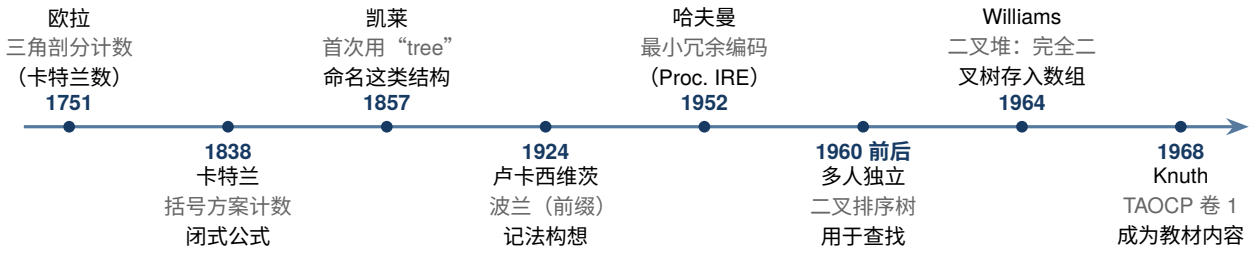
前面几章学过的线性表、栈、队列、数组，都属于**线性结构**：每个元素最多一个前驱、一个后继，像一串珠子。但现实中大量数据是**分层**的——文件夹里套文件夹、学院下设系、表达式里括号套括号。用一根“珠串”去表示这种“一对多”的关系，要么丢失层次信息，要么查找极慢。**树**就是为表达“一对多的层次关系”而引入的第一种非线性结构（笔记 p578 红笔批注：“线性结构 1 对 1，树形结构 1 对多，图形结构多对多”）。



前置知识检查：(1) 单链表结点与指针（`->`、`NULL`）；(2) 链队列的入队/出队（层次遍历要用）；(3) 递归函数的执行过程（遍历全部靠递归）；(4) 等比数列求和 $1+2+\cdots+2^{k-1}=2^k-1$；(5) 对数与取整 $\lfloor \log_2 n \rfloor$。如果其中某项不熟，先翻到第 2、3 章复习，再读本讲义。

学科小史：这个知识是怎样出现的？

“树”不是计算机学科凭空发明的。它最早出现在数学里，用来计数和组织；计算机出现后，人们发现它恰好能解决“如何存放分层数据”“如何快速查找”“如何压缩数据”这些新问题。下面只列与本章直接相关、且有可靠出处的节点（年份均为文献公认年份；“首创”存在争议的地方采用保守表述）。



几个值得知道的历史细节

- **计数先于数据结构。**欧拉在 1751 年致哥德巴赫的信中研究凸多边形三角剖分的方案数；卡特兰在 1838 年研究括号添加方案时得到闭式。这串数 $1, 1, 2, 5, 14, 42, \ldots$ 后来被称作“卡特兰数”——它恰好就是 n 个结点的不同二叉树的个数（§5.3）。“卡特兰数”这个名字并不意味着卡特兰最早发现它。
- **“树”这个名字。**1857 年凯莱在论文 *On the theory of the analytical forms called trees* 中首次用“tree”称呼这种分支结构，当时是为了整理多重求导的式子。
- **前缀/后缀表达式。**卢卡西维茨自述 1924 年构想了不用括号的前缀记法（“波兰记法”），1929 年见于出版物；1950 年代起，后缀（逆波兰）记法被多位研究者独立用于计算机中的栈式求值。表达式树的先序、后序遍历正好产生这两种记法（§4.4）。
- **哈夫曼的学期论文。**1951 年，MIT 研究生哈夫曼在 Fano 的信息论课上选择写学期论文而不是参加期末考试，提出了“自底向上合并最小权值”的方法，并证明它是最优的；此前 Fano 等人的自顶向下方法（香农–范诺编码）并不总是最优。论文 1952 年发表于 *Proceedings of the IRE*。

为什么今天仍然要学它？因为“层次 + 递归”这一思想贯穿后续几乎所有课程：查找（二叉排序树）、排序（堆排序）、编译原理（语法树）、操作系统（目录树）、数据库（索引树）、通信（哈夫曼编码）。本章是它们共同的地基。

从课本到现实：这个知识到底有什么用？

①压缩文件与图片
ZIP/gzip/PNG 使用的 DEFLATE 格式（RFC 1951）、基线 JPEG（ITU-T T.81）、MP3 的最后一步熵编码，都用到**哈夫曼编码**（§7）。

②网页结构
浏览器把 HTML 解析成 DOM 树；DOM 标准规定“文档顺序就是树的**先序**深度优先遍历”（§4）。注意：DOM 是多叉树。

③计算器与编译器
HP 科学计算器采用逆波兰（后缀）输入；编译器把表达式变成**表达式树**，再按**后序**生成计算步骤（§4.4）。

④优先队列
C++ 的 `std::priority_queue` 底层是“堆”：一棵**完全二叉树**按层存在数组里，父子下标靠 $i/2$、$2i$、$2i+1$ 换算（§3.1）。

这四个例子分别对应本章的四大块：编码（§7）、遍历（§4）、表达式树（§4.4）、顺序存储（§3）。课堂上学的不是“玩具”，而是这些真实系统里的标准零件——但也要注意边界：在压缩格式里，哈夫曼编码只是其中一步，并不等于整个压缩算法；文件系统、DOM、组织架构是多叉树，并不是二叉树。

原始资料的重新编码与纠错

资料清单与内容对应

资料	主要内容	本讲义位置
笔记 p578–582	线性/非线性结构回顾；树的定义、表示法、14 个术语；与线性结构比较；二叉树定义、特点、五种形态	§1, §2.1
笔记 p582–586	案例引入（数据压缩、表达式求值）；ADT；性质 1–5 及证明；满/完全二叉树；顺序存储、二叉链表、三叉链表、$n+1$ 个空指针	§2.2–§3
笔记 p587–594	遍历定义与六种次序；先/中/后序算法与例题；表达式前中后缀；由序列确定二叉树（两例）；递归执行图；“三次经过”；层次遍历；遍历应用	§4–§5
笔记 p594–600	树和森林：双亲表示、孩子链表、孩子兄弟；树/森林与二叉树互转；树与森林的遍历	§6
笔记 p601–605	路径长度、WPL、哈夫曼树与算法、两例构造、前缀编码与哈夫曼编码	§7
课件 1–124	与笔记同一主线，另含 ADT 各操作、顺序/链式二叉树类代码、课堂单选与主观题、卡特兰计数、复制与建树代码、哈夫曼编码报文例题、课后作业	全文
课件 125–150	附录：用栈实现非递归中序/先序遍历的动画（标注“不讲”）	附录 B
源程序 6 份	二叉链表/顺序存储二叉树类；先、中、后、层次遍历演示（教材图 6.9）	§3, §4, 附录 A

原图纠错汇总

下列问题均在正文对应位置再次标注。凡属“无法可靠辨认”的内容，本讲义不做猜测。

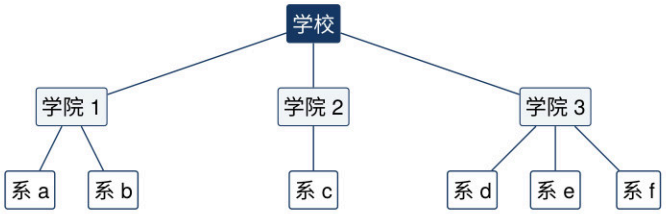
原图纠错：笔记与课件中的错误或不严谨之处

位置	原始写法	规范写法与理由
笔记 p583 性质 1	“在二叉树第 i 层**至少**有 2^{i-1} 个结点（至少有 1 个结点）”	应为“**至多**有 2^{i-1} 个结点”。括号里“至少 1 个”是对的，前半句把“至多”误写成“至少”。影响：性质 2 的推导依赖“至多”。
笔记 p579 叶子	叶子结点举例“(J, G, F)”	只是部分列举。该树全部叶子为 K、L、F、G、M、I、J。考试写“所有叶子”时不能漏。
课件 p7 叶子	叶子结点：O, P, F, G, L, M, N	**漏了 I**。图中 I 是 D 的孩子且没有孩子，度为 0，是叶子。正确：O, P, F, G, L, I, M, N。
课件 p116 WPL	“节点 11 ……即 $3\times11=15$”；“WPL= $15+25+28+42+58+24+15+46=253$”	应为 $3\times11=33$，WPL $=15+25+28+42+58+24+33+46=$ **271**。可用“WPL = 所有非叶结点权值之和”验算：$8+15+19+29+42+58+100=271$（§7.2）。
笔记 p602	“WPL 要求在′ 度相同′ 的树中比较”	应为：在**叶子个数相同、叶子权值相同**的二叉树之间比较 WPL。哈夫曼树的定义前提是“n 个叶子、给定权值”。
笔记 p603–604 第二例	权值为 2 的结点标成 a	题设 a,b,c,d,e 权为 7,5,5,2,4，权 2 的是 **d**。不影响 WPL $=52$（§7.3）。
笔记 p595/p596	同一棵树，p595 最右叶子写 I，p596 写 K	两页是同一示例树，字母不一致属笔误，不影响方法。
笔记 p583 证明	“$\because n=n_2+n_1+n_0$”旁的红色记号	**原始资料此处无法可靠辨认**红色记号的含义；该式本身正确。
课程源程序	`CreateBinaryTree` 返回 `new` 出来对象的引用	调用者若按值接收会复制一份，堆上原对象永不释放（内存泄漏）。附录 A 改为返回根指针，交给 `BinaryTree(r)` 管理。

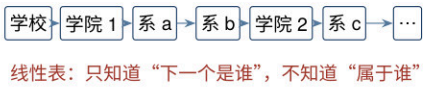
1　树的基本概念 [课件 6.1 • 笔记 p578–580]

1.1　为什么需要树：线性结构装不下“层次”

先看一个问题：某高校　→ 若干学院 → 每个学院若干系 → 每个系若干专业。如果把它们存进一个线性表，比如“某高校　, 某学院　, 自动化系, ……, 数学学院, ……”，表里只记录了“谁排在谁后面”，却没有记录“谁属于谁”。要回答“某学院　下面有哪些系”，只能从头扫描并另外记录归属——这就是线性结构的局限：**每个元素只能有一个后继**。



树：每个结点可以有多个“下一层”，层次关系一目了然



线性表：只知道“下一个是谁”，不知道“属于谁”

树把“一个元素对应多个下层元素”直接画成分支。它看起来像一棵倒挂的树：根在上，叶在下（课件 p4）。

1.2　树的定义：用“树”来定义“树”

定义：树（Tree）

树是 n $(n \geq 0)$ 个结点的有限集 T。

- 若 $n = 0$，称为**空树**；
- 若 $n > 0$, 则: (1) 有且仅有一个特定的称为**根**(root)的结点; (2) 其余结点可分为 m $(m \geq 0)$ 个**互不相交**的有限集 $T_1, T_2, \ldots, T_m$, 其中每一个集合本身又是一棵树, 称为根的**子树** (SubTree)。

这是一个**递归定义**：定义“树”的时候用到了“树”本身（子树也是树）。这并不循环，因为子树的结点数严格少于原树，层层缩小，最后一定落到“只有一个结点（$m = 0$）”或空树。后面所有的遍历、求高度、求结点数算法，都直接照着这个递归结构写——**这就是本章最重要的思想**。

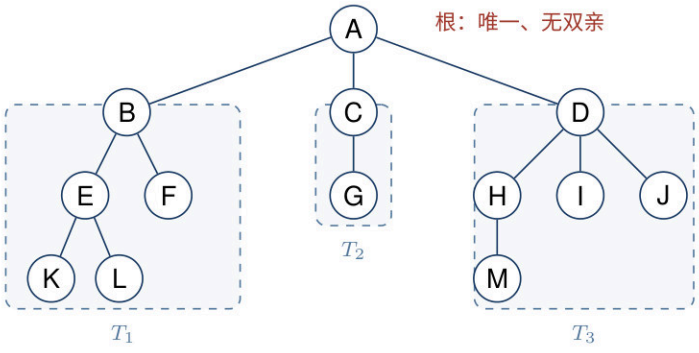
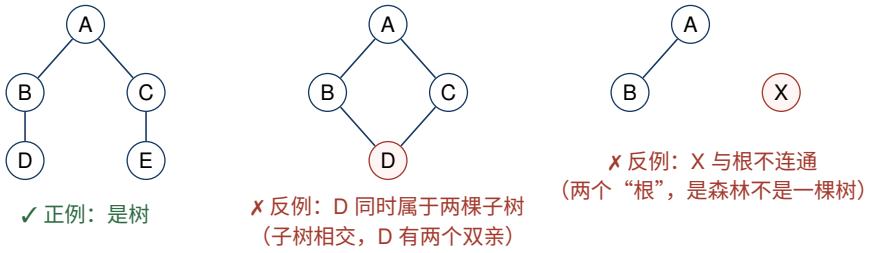


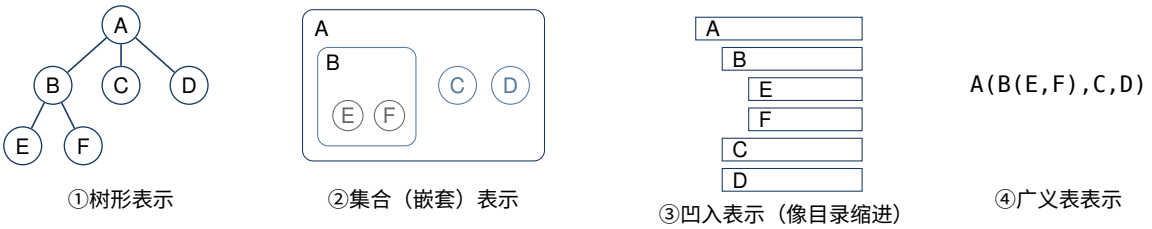
图 1: 笔记 p579 示意图：去掉根 A 后，剩下的结点恰好分成 3 棵互不相交的子树

定义中每个条件为什么不能少？　**“有且仅有一个根”**保证从上往下看有唯一起点；**“互不相交”**保证每个非根结点只有一个双亲——如果两棵子树共用一个结点，那个结点就有两个“上级”，结构变成“网”（图），不再是树。下面的反例说明这一点。



1.3　树的四种表示法

笔记 p579 列出：树形表示、集合（嵌套）表示、凹入表示、广义表表示。它们描述的是同一个逻辑结构，只是画法不同。以树 A(B(E,F), C, D) 为例：



凹入表示就是书的目录、文件管理器左栏的样子；广义表表示“括号里是孩子”，最方便在纸上快速写出一棵树——本讲义后面经常用它，例如 A(B(D(,G)),C(E,F)) 中“,G”表示 D 的左孩子为空、右孩子为 G。

1.4　基本术语：一张图看懂 14 个名词

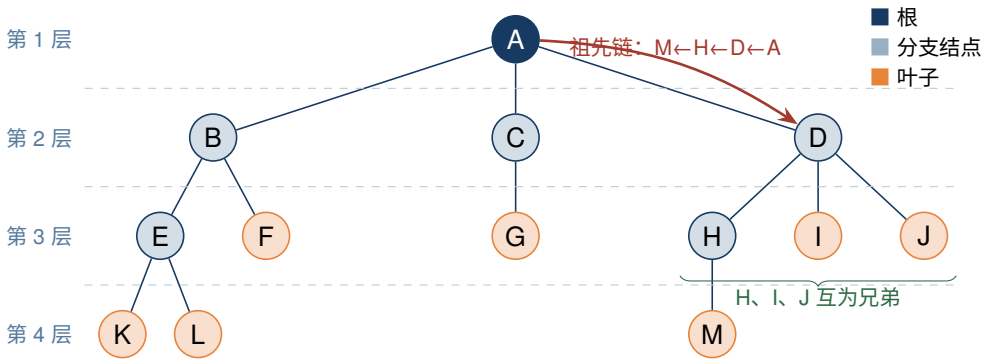


图 2: 笔记 p579 的术语示意树（本讲义称为“树 T_0”）

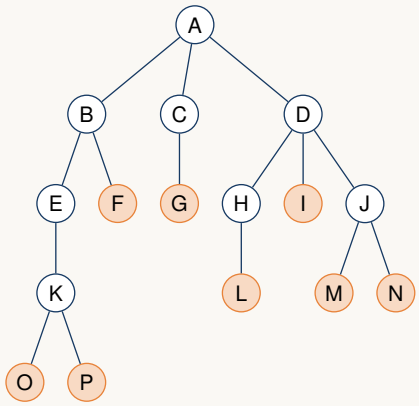
术语	定义（课件/笔记口径）	在树 T_0 中
结点	一个数据元素 + 若干指向子树的分支	共 13 个
结点的度	结点拥有的**子树个数**（= 孩子个数）	A:3, B:2, C:1, K:0
树的度	树内各结点的度的**最大值**	3
叶子（终端结点）	度为 0 的结点	K,L,F,G,M,I,J
分支（非终端）结点	度不为 0 的结点；除根以外的分支结点也叫**内部结点**	A,B,C,D,E,H
孩子 / 双亲	结点的子树的根称为它的孩子；它称为孩子的双亲	E 的双亲是 B
兄弟	同一双亲的孩子之间互称兄弟	H,I,J
堂兄弟	双亲在同一层的结点	G 与 H
祖先	从根到该结点所经分支上的所有结点	M 的祖先：H,D,A
子孙	以某结点为根的子树中的任一结点	D 的子孙：H,I,J,M
层次	根为第 1 层，其孩子为第 2 层，依此类推	K 在第 4 层
树的深度（高度）	树中结点的最大层次	4
有序树 / 无序树	各子树从左到右有次序（不能互换）/ 无次序	本课程默认有序
森林	m $(m \geq 0)$ 棵互不相交的树的集合	去掉 A 得 3 棵树

易错警示：三个高频错误

1. **“叶子”要列全。**笔记 p579 只举了 J、G、F 三个例子；该树叶子共 7 个（见上表）。课件 p7 同类例题也漏写了 I（见纠错汇总）。**方法**：逐个检查“有没有孩子”，而不是凭印象挑底层结点——叶子不一定在最底层（F、G、I、J 都在第 3 层）。
2. **“树的度”不是“结点数”，也不是“层数”。**它是所有结点度的最大值。
3. **层次从 1 开始数**（本课程口径）。很多英文教材让根的深度为 0，换算关系：中文深度 = 英文 height $+1$。考试按课本口径。

例题 1.1（课件 p7）

树 A(B(E(K(O,P)),F), C(G), D(H(L),I,J(M,N)))。求：A、B、C 的度；树的度；全部叶子；O 的层次；树的深度；K 的祖先。

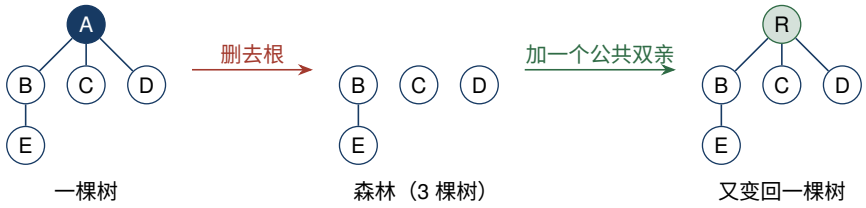


第一步数孩子：A 有 B,C,D → 度 3；B 有 E,F → 2；C 有 G → 1。
第二步树的度：最大的结点度是 3（A 和 D）。
第三步找叶子（逐个检查无孩子者，橙色）：O,P,F,G,L,I,M,N 共 8 个。
第四步层次：A1 → B2 → E3 → K4 → O5，故 O 在第 5 层，深度为 5。
第五步祖先：从根走到 K 经过 A,B,E。
检查：结点 A–P 共 16 个；叶子 8 个（O,P,F,G,L,I,M,N），分支结点 8 个（A,B,C,D,E,H,J,K），$8+8=16$ ✓。注意 K 有孩子 O、P，所以 K 是分支结点而不是叶子。

本题真正想说明什么：术语题要“按定义逐个检查”，并用“结点总数 = 叶子数 + 分支结点数”做最后核对；特别留意有孩子的中间层结点（如 K）不要误算成叶子。

1.5　有序树、森林，以及树和森林的关系

课件 p8 把一棵非空树看成一个二元组 Tree = (root, F)：root 是根，F 是它的子树构成的森林。笔记 p580 的红笔总结非常重要，后面 §6 的转换全部依赖它：



有序树中子树从左到右有次序，“最左边的是第一个孩子”（笔记 p580）。课件 p8 特别说明：后面实现的树的存储结构**都按有序树处理**。

1.6　与线性结构的比较

➢ 对照：线性表 ↔ 树（推广型，笔记 p580）

一句话定位：线性表是“每个结点至多一个孩子”的特殊的树——一根没有分叉的单支。

比较维度	线性结构	树结构
开头元素	第一个元素（无前驱）	根结点（无双亲）
结尾元素	最后一个元素（无后继）	叶子（无孩子），可以有多个
中间元素	一个前驱、一个后继	一个双亲、多个孩子（一对多）
访问顺序	天然唯一（从头到尾）	不唯一，必须约定“遍历次序”

迁移判断：✓“每个元素至多一个前驱/双亲”可直接迁移；✗“从头到尾只有一种访问顺序”不成立——这正是 §4 要专门研究遍历的原因；△“求长度”迁移为“求结点数”，但要用递归分别统计各子树。

➢ 对照：树的“度”↔ 图论中顶点的“度”（同名异义型，第 7 章）

树里“结点的度”只数**孩子**；图论里“顶点的度”数所有相连的边，**包括连向双亲的那条**。在树 T_0 中，B 的树度是 2，但把树看成无向图时 B 的图度是 3。**迁移警示**：学第 7 章时，“叶子的度为 0”要改成“叶子（非根）的图度为 1”。

✍ 生活中的它：电脑里的文件夹

生活场景：打开“此电脑 → D 盘 → 课程 → 数据结构 → 实验 1”。
对应关系：盘符 = 根；文件夹 = 分支结点；文件 = 叶子；“所在文件夹” = 双亲；路径 `D:\课程\数据结构` = 从根到结点的祖先链；文件夹层数 = 层次。
小判断：一个文件能不能同时直接放在两个文件夹里？不能（快捷方式是另一个文件）——这正对应定义里“子树互不相交”。
边界：文件夹树是多叉树，不是二叉树；系统中的“链接/快捷方式”会让真实结构比纯粹的树复杂。

学完这里，你应该能够回答

(1) 树的定义为什么是递归的？　(2)“互不相交”去掉会怎样？　(3) 叶子一定在最底层吗？　(4) 树的度与结点的度有何区别？　(5) 树和森林如何互相转化？

2　二叉树：定义、形态与性质 [课件 6.2.1–6.2.2 · 笔记 p580–585]

到这里，我们已经解决了什么？我们有了“树”这种表达层次的结构。**还剩什么问题？**一般树每个结点的孩子数可以是 0、1、5、100……，存储时每个结点该留几个指针？算法该怎样统一处理“若干棵子树”？这些都不方便。于是人们先研究一种**最规整**的树——二叉树。

2.1　为什么偏偏是“二”叉树

笔记 p581 给出两条理由：**①二叉树结构最简单、规律性最强**——每个结点固定两个“槽位”（左、右），因而能推出一系列精确的数量性质（§2.3）；**②所有树都能转化为唯一对应的二叉树**（§6 会给出转换方法），所以研究透二叉树，就等于掌握了一般树的处理方法。

定义：二叉树（Binary Tree）

二叉树是 n $(n \geq 0)$ 个结点的有限集，它或者是**空集**（$n = 0$），或者由**一个根结点**及**两棵互不相交的、分别称作这个根的左子树和右子树的二叉树**组成。

这同样是递归定义。与树的定义对照，关键差别只有一处：树的子树是“m 棵、无位置之分”，二叉树的子树是“恰好**左、右两个位置**，每个位置可以空”。由此得到三条特点（笔记 p581）：

1. 每个结点**最多有 2 个孩子**（度 ≤ 2）；
2. 子树**有左右之分，次序不能颠倒**；
3. 二叉树可以是空集；根可以有空的左子树或空的右子树。

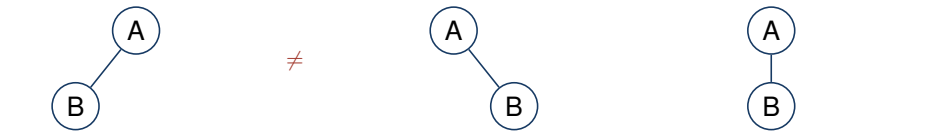


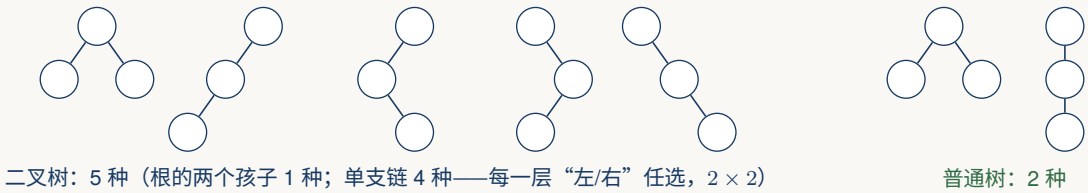
图 3: 笔记 p581 示意图：①②是两棵**不同**的二叉树，但作为树它们是同一棵

易错警示：二叉树不是“度为 2 的树”

笔记 p581：“二叉树不是树的特殊情况，它们是两个概念。”原因：树中结点只有一个孩子时**无须区分**左右；二叉树中即使只有一棵子树，也**必须说明**是左子树还是右子树。所以上图①②在二叉树意义下不同，在树意义下相同。

例题 2.1（课件 p12 单选题 • 笔记 p581 例）

具有 3 个结点的二叉树有几种不同形态？作为普通（有序）树又有几种？

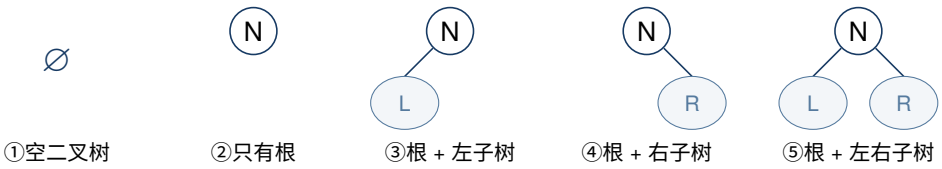


思路：先按“根有几个孩子”分类。根有 2 个孩子：1 种；根有 1 个孩子（左或右，2 种），这个孩子再有 1 个孩子（左或右，2 种），共 $2 \times 2 = 4$ 种。合计 $1 + 4 = \mathbf{5}$。普通树没有左右之分，单支链只剩 1 种，合计 **2**。

本题真正想说明什么：二叉树“左右有别”会让形态数大大增加。一般地，n 个结点的二叉树形态数是卡特兰数 b_n（§5.3），$b_3 = 5$。

二叉树的五种基本形态

由定义直接得到：任何一棵二叉树都是下面五种形态之一（笔记 p582、课件 p11）。后面写递归算法时，**“空树”就是递归出口**，另外四种只是左右子树空与不空的组合。



➢ 对照：二叉树 ↔ 度为 2 的有序树（同名近义型）

一句话定位：两者都“每个结点最多 2 个孩子”，但二叉树给孩子分配了**固定位置**。

维度	二叉树	度为 2 的有序树
能否为空	能	不能（树的度为 2 说明至少有一个结点度为 2）
只有一个孩子时	必须区分左/右	不区分，就是“第一个孩子”
3 个结点的形态数	5	1（只有“根带两个孩子”一种，单支链的度是 1）

迁移判断：✓递归定义思路可直接迁移；✗“形态数相同”不成立；△树的遍历方法（§6.4）迁移到二叉树时，要把“依次遍历各子树”具体化为“先左后右”。

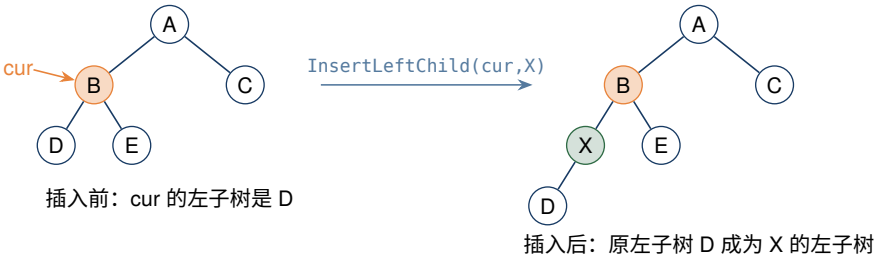
✍ 生活中的它：“二十问”猜谜与是非决策树

生活场景：朋友心里想一种动物，你只能问“是/否”问题：“会飞吗?”“是哺乳动物吗?”……
对应关系：每个问题 = 一个分支结点；“是”走左孩子、“否”走右孩子（左右不能对调，否则答案全错——这正是二叉树“左右有别”）；最终猜出的动物 = 叶子。
小估算：问 3 个问题最多能区分几种动物? 第 i 层至多 2^{i-1} 个结点，3 个问题之后到达第 4 层，至多 $2^3 = 8$ 种（§2.3 性质 1）。
边界：真实的机器学习决策树不一定是二叉的（有的算法允许多路分支），问题之间也不必等概率。

2.2　二叉树的抽象数据类型与基本操作 [课件 p13–18 • 笔记 p582–583]

ADT 由三部分组成：**数据对象** D（同类元素的集合）、**数据关系** R（若 $D = \varnothing$ 则 $R = \varnothing$；否则 $R = \{H\}$，H 满足：root 唯一；左右子树的结点集互不相交 $D_l \cap D_r = \varnothing$；左、右子树各自又是二叉树）和**基本操作** P。课件列出的 17 个操作在 C++ 类 `BinaryTree` 中一一对应，归类如下（cur 为指向某结点的指针）：

类别	操作（C++ 成员函数）	实现要点
查询	`GetRoot()`, `Empty()`, `GetElem(cur,e)`, `LeftChild(cur)`, `RightChild(cur)`, `Parent(cur)`	Parent 需从根递归查找（二叉链表没有双亲指针）
修改	`SetElem(cur,e)`, `InsertLeftChild(cur,e)`, `InsertRightChild(cur,e)`, `DeleteLeftChild(cur)`, `DeleteRightChild(cur)`	插入时 cur 原子树成为新结点的同侧子树
遍历	`PreOrder`, `InOrder`, `PostOrder`, `LevelOrder`（参数是函数指针 `visit`）	§4
统计	`NodeCount()`, `Height()`	递归，§5
构造/复制	`BinaryTree(e)`，复制构造函数，`operator=`	深复制，§5



插入前：cur 的左子树是 D　　插入后：原左子树 D 成为 X 的左子树

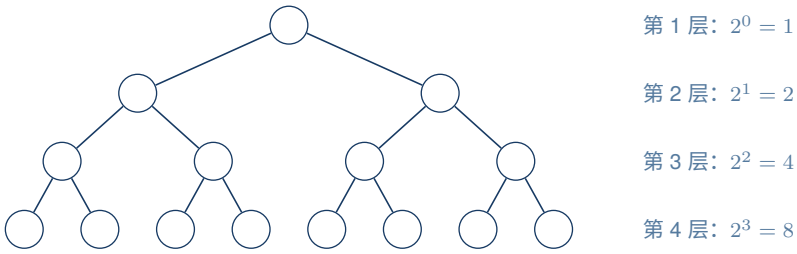
2.3　二叉树的五条性质 [课件 p19–24]

课件标注“证明部分自己看”，但证明恰恰能说明为什么，而且考试常考“直接用性质计算”。下面逐条给出完整证明。

性质 1：第 i 层至多 2^{i-1} 个结点

性质 1

在二叉树的第 i 层上**至多**有 2^{i-1} 个结点（$i \geq 1$）；至少有 1 个结点（若该层存在）。



证明：数学归纳法（课件 p19）

要证：对一切 $i \geq 1$，第 i 层结点数 $\leq 2^{i-1}$。
思路：下一层的结点都是上一层结点的孩子，而每个结点至多 2 个孩子，所以每下一层至多翻一倍。
归纳基础：$i = 1$ 时只有根，$1 = 2^0$，成立。
归纳假设：设第 $i-1$ 层至多有 2^{i-2} 个结点。
归纳步骤：第 i 层的每个结点都是第 $i-1$ 层某结点的孩子（二叉树定义），每个结点至多 2 个孩子（特点 1），故

$$\text{第} i \text{ 层结点数} \leq 2 \times 2^{i-2} = 2^{i-1}.$$

由归纳法，结论对一切 $i \geq 1$ 成立。■
条件为什么不能删：若允许结点有 3 个孩子，每层可翻 3 倍，结论变为 3^{i-1}——“2”直接来自“二”叉。

原图纠错：笔记 p583

原始写法：“在二叉树第 i 层至少有 2^{i-1} 个结点（至少有 1 个结点）”。**问题**：前半句应为**至多**。**为什么**：单支树每层只有 1 个结点，第 3 层只有 1 个，远少于 $2^2 = 4$，“至少 2^{i-1}”显然不对。**影响**：性质 2 正是把“至多”逐层相加得到的。

性质 2：深度为 k 的二叉树至多 2^k-1 个结点

性质 2

深度为 k 的二叉树**至多**有 2^k-1 个结点（$k\ge 1$），**至少**有 k 个结点（每层 1 个的单支树）。

证明

由性质 1，把每一层的上限相加，用等比数列求和公式 $S_n=\dfrac{a_1(1-q^n)}{1-q}$（$a_1=1,\ q=2,\ n=k$）：

$$\begin{aligned}\text{结点总数} &\le \sum_{i=1}^{k} 2^{i-1} = 1+2+4+\cdots+2^{k-1} && \text{（性质 1 逐层取上限）}\\ &= \frac{1\cdot(1-2^k)}{1-2} = 2^k-1. && \text{（等比求和）}\end{aligned}$$

下限：深度为 k 意味着第 $1\sim k$ 层每层至少 1 个结点，故至少 k 个。■

性质 3：$n_0=n_2+1$

性质 3（最常考）

对**任何一棵非空**二叉树，若叶子数为 n_0，度为 2 的结点数为 n_2，则 $\boxed{n_0=n_2+1}$。

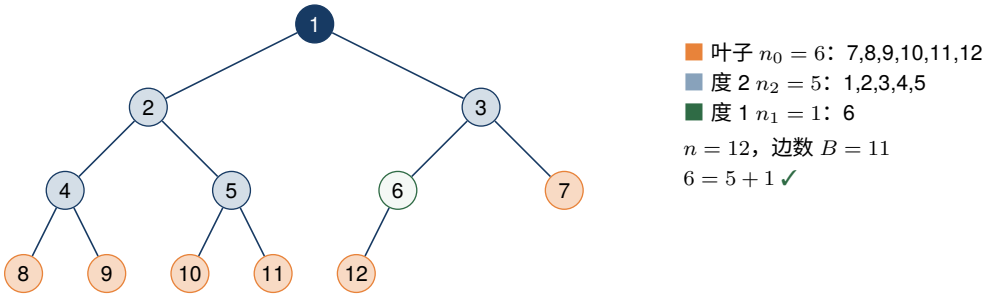


图 4: 笔记 p583 的证明用图：从两个方向数“边”

证明：从两个方向数边（课件 p21、笔记 p583）

要证：$n_0=n_2+1$。　**已知**：结点总数 n，度为 0、1、2 的结点数分别为 n_0,n_1,n_2，边（分支）总数为 B。

思路：同一个量（边数）用两种方式数，得到两个等式，联立消元。

$$\begin{aligned} n &= n_0+n_1+n_2 && \text{（每个结点的度只能是 0、1、2）} && (1)\\ B &= n-1 && \text{（从下往上看：除根外每个结点恰有一条边连向双亲）} && (2)\\ B &= n_1+2n_2 && \text{（从上往下看：度 1 结点发出 1 条边，度 2 结点发出 2 条）} && (3)\end{aligned}$$

由 (2)(3) 得 $n=n_1+2n_2+1$；代入 (1)：$n_0+n_1+n_2=n_1+2n_2+1$，消去 n_1 得 $n_0=n_2+1$。■

一句话直观：每“分叉”一次（多一个度 2 结点）就多出一片叶子。**条件**：“非空”不能删——空树 $n_0=n_2=0$，不满足。

✍ **生活中的它：单败淘汰赛**

生活场景：16 支队伍打单败淘汰赛，输一场就出局。

对应关系：每支队伍 = 叶子（$n_0=16$）；每场比赛 = 度为 2 的结点（两队相遇，胜者“晋级”为

上一层）；冠军赛 = 根。对阵表是一棵没有度 1 结点的二叉树。
小估算：一共要打几场？$n_2 = n_0 - 1 = 15$ 场。即使队数不是 2 的幂（例如 13 队，有队伍轮空），比赛数仍然是 $13 - 1 = 12$——每场淘汰恰好一队，最后只剩冠军。
边界：双败赛制、小组循环赛不满足“每场淘汰一队”，不能套用此结论。

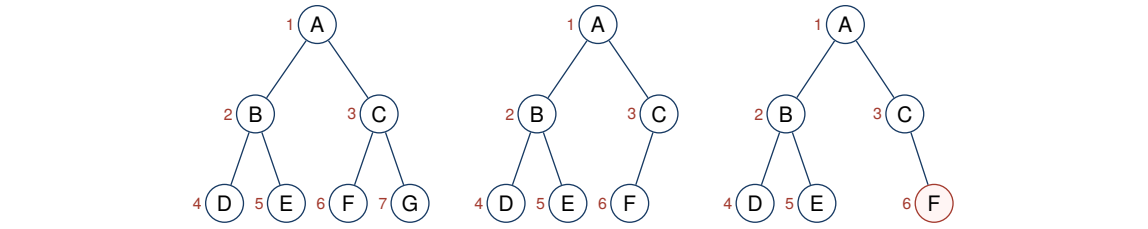
2.4　两种特殊形态：满二叉树与完全二叉树 [课件 p22 • 笔记 p584]

定义：满二叉树

深度为 k 且有 $2^k - 1$ 个结点的二叉树。特点：**每一层都满**，叶子全在最底层。约定对满二叉树**从根开始，自上而下、自左而右**连续编号 $1, 2, \ldots, 2^k - 1$。

定义：完全二叉树

深度为 k、有 n 个结点的二叉树，**当且仅当**其每一个结点都与深度为 k 的满二叉树中编号 $1 \sim n$ 的结点一一对应时，称为完全二叉树。



满二叉树（$k = 3$，$2^3 - 1 = 7$ 个结点）　完全二叉树 ✓　非完全二叉树 ✗（F 占了 7 号位，6 号位空缺）

第三棵树为什么不是完全二叉树？F 是 C 的右孩子，在满二叉树中对应编号 7，而 6 号位置是空的——编号 $1 \sim 6$ 与结点不能一一对应。笔记 p584 给出一个好用的判别口诀：

判别口诀　**“在满二叉树中，从最后一个结点开始，连续去掉任意个结点，得到的就是完全二叉树。”**（笔记 p584，“连续”二字被红框圈出）
等价说法：按层序编号时**中间不能有空位**。由此推出完全二叉树的特点：(1) 叶子只可能分布在层数最大的两层上；(2) 对任一结点，若其右子树的最大层次为 i，则其左子树的最大层次为 i 或 $i+1$；(3) 度为 1 的结点**至多 1 个**，且它只有左孩子。

教学补全：特点 (3) 的推导　按层序编号无空位，所以“只有一个孩子”的情况只可能出现在编号 n 的双亲 $\lfloor n/2 \rfloor$ 上，且仅当 n 为偶数（$n = 2\lfloor n/2 \rfloor$ 是左孩子编号）。于是：n 为偶数时 $n_1 = 1$，n 为奇数时 $n_1 = 0$。这一结论配合性质 3 可以秒算完全二叉树的叶子数（例题 2.2）。

课件 p22 还指出：**满二叉树一定是完全二叉树，反之不然**。

性质 4：完全二叉树的深度

性质 4

具有 n 个结点的完全二叉树的深度为 $\boxed{\lfloor \log_2 n \rfloor + 1}$（$\lfloor x \rfloor$ 表示不大于 x 的最大整数）。

证明：笔记 p585

设深度为 k。前 $k-1$ 层是满的（共 $2^{k-1} - 1$ 个结点），第 k 层至少 1 个、至多满层，由性质 2：

$$2^{k-1} - 1 < n \le 2^k - 1 \implies 2^{k-1} \le n < 2^k \implies k - 1 \le \log_2 n < k.$$

k 是整数且 $\log_2 n$ 落在 $[k-1, k)$ 内，所以 $k-1 = \lfloor \log_2 n \rfloor$，即 $k = \lfloor \log_2 n \rfloor + 1$。（第二步用了“整数不等式 $a-1 < n \le b-1 \iff a \le n < b$”。）■

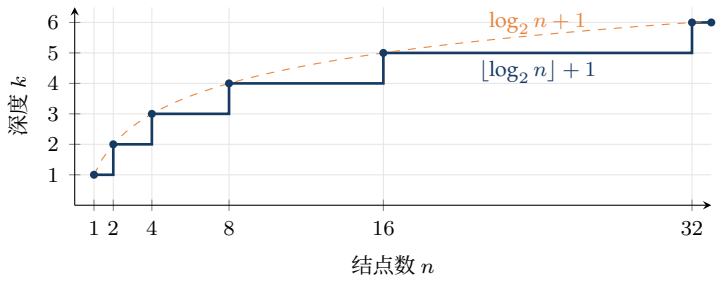
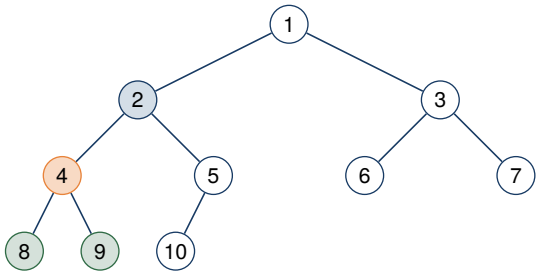


图 5: 深度随 n 呈“台阶”增长：n 每翻一倍，深度才加 1——这就是树结构能高效查找的根源

性质 5：完全二叉树的编号关系

性质 5

对含 n 个结点的完全二叉树按层序**从 1 开始**编号，则对任一编号为 i 的结点：(1) 若 $i = 1$，它是根，无双亲；若 $i > 1$，双亲编号为 $\lfloor i/2 \rfloor$；(2) 若 $2i > n$，它无左孩子（是叶子）；否则左孩子编号为 $2i$；(3) 若 $2i + 1 > n$，它无右孩子；否则右孩子编号为 $2i + 1$。



$i = 4$：双亲 $\lfloor 4/2 \rfloor = 2$
左孩子 $2 \cdot 4 = 8$，右孩子 9
$i = 5$：$2 \cdot 5 = 10 \le n$ 有左孩子；
$11 > n = 10$，无右孩子
$i = 6$：$12 > 10$，是叶子

教学补全：性质 5 为什么成立（直观证明） 第 j 层的第一个结点编号是 2^{j-1}。设 i 是第 j 层的第 t 个结点（$i = 2^{j-1} + t - 1$），它左边有 $t-1$ 个同层结点，每个都有 2 个孩子“排在它的孩子前面”；第 $j+1$ 层首编号为 2^j，所以 i 的左孩子编号为 $2^j + 2(t-1) = 2(2^{j-1} + t - 1) = 2i$，右孩子紧随其后为 $2i + 1$。反过来，孩子 $2i$ 与 $2i + 1$ 除以 2 向下取整都得 i，这就是双亲公式。

易错警示：从 0 编号时公式要改！

笔记 p585 的“一般二叉树的顺序存储”图把根编号为 **0**（a=0, b=1, c=2, ⋯）。此时：双亲 $\lfloor (i-1)/2 \rfloor$，左孩子 $2i + 1$，右孩子 $2i + 2$。**课程源程序 `SqBinaryTree` 从 1 开始**（`LeftChild` 返回 `2*cur`）。考试时先看清题目的编号起点，混用是最常见的丢分点。

例题 2.2（课件 p25 单选题）

一棵完全二叉树上有 1001 个结点，其中叶子结点的个数是（　）。A. 250　B. 500　C. 254　D. 501

第一步：$n = 1001$ 是奇数，由上面的补全结论，$n_1 = 0$。

第二步：$n = n_0 + n_1 + n_2 = n_0 + 0 + (n_0 - 1) = 2n_0 - 1$（用性质 3）。

第三步：$2n_0 - 1 = 1001 \Rightarrow n_0 = 501$，选 **D**。

检查：叶子编号应为 $\lfloor n/2 \rfloor + 1 \sim n$，即 $501 \sim 1001$，共 $1001 - 500 = 501$ 个 ✓。

本题真正想说明什么：完全二叉树 + 性质 3 + “n_1 只能是 0 或 1”三者联用。

➢ 对照：中文“满/完全二叉树” ↔ 英文 full / perfect / complete（同名异义型）

一句话定位：中文教材的“满二叉树”在 NIST 算法词典中叫 *perfect binary tree*；而英文 *full binary tree* 在 NIST 中指“每个结点有 0 个或 2 个孩子”——**与中文“满”不是一回事**。

中文（本课程）	NIST 用语	注意
满二叉树：每层都满	perfect binary tree	部分英文教材（如 CLRS）把它叫 complete
完全二叉树：编号无空位	complete binary tree	与 CLRS 的 complete 含义不同
（无专名）：每结点 0 或 2 个孩子	full / proper binary tree	哈夫曼树就是这种树（§7）
深度：根为第 1 层	depth/height：根为 0，按边数计	中文深度 = 英文 height +1

迁移判断：✗看英文资料时不能把 full 直接译成“满”；△英文性质“高度 h 的二叉树至多 $2^{h+1}-1$ 个结点”换成中文口径（$k=h+1$）就是性质 2；✓$n_0=n_2+1$ 与口径无关，直接成立。

✍ 生活中的它：家谱的“祖先编号法”

生活场景：族谱中常用的一种祖先编号（西方称 Ahnentafel）：自己记为 1，父亲 2、母亲 3，父亲的父亲 4、父亲的母亲 5、母亲的父亲 6……

对应关系：这正是把“祖先图”当作完全二叉树按层编号：编号 i 的人，其父亲是 $2i$、母亲是 $2i+1$，其子女（在这张图里）是 $\lfloor i/2 \rfloor$——与性质 5 完全一致。

小判断：编号 13 的人是谁？$13=2\times6+1$，是 6 号的母亲；6 号是 2×3，即 3 号（母亲）的父亲。所以 13 号是“外祖父的母亲”。

边界：真实家谱若有人信息缺失，就会出现“空位”，这棵树就不再是完全二叉树——正好对应下一节顺序存储中的“虚结点”。

学完这里，你应该能够回答

(1) 二叉树与度为 2 的树有哪三点不同？ (2) 写出性质 3 的两种数边方式。 (3) 为什么完全二叉树 $n_1\le1$？ (4) 从 0 编号时左孩子公式是什么？ (5) $n=100$ 的完全二叉树深度是多少？(7)

3　二叉树的存储结构 [课件 6.2.3 • 笔记 p585–587]

到这里，我们已经解决了什么？我们知道了二叉树长什么样、有哪些数量规律。**还剩什么问题？**计算机内存是一排连续编号的格子，没有“左下方”“右下方”。我们必须想办法把“谁是谁的左孩子、右孩子”这一关系也存下来。有两条路：**用位置（下标）隐含关系**——顺序存储；**用指针显式记录关系**——链式存储。

3.1　顺序存储：用下标表示父子关系

定义：顺序存储结构（笔记 p585）

按**满二叉树的结点层次编号**，依次把二叉树的数据元素存放到一组连续的存储单元中；不存在的结点（**虚结点**）位置留空并作标记。由性质 5，下标为 i 的结点：左孩子 $2i$，右孩子 $2i+1$，双亲 $\lfloor i/2 \rfloor$（下标从 1 开始，0 号单元不用）。

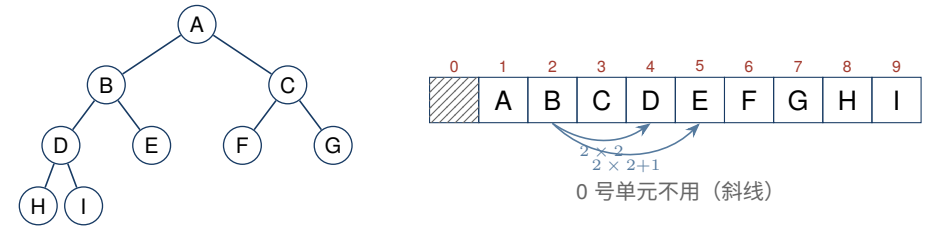
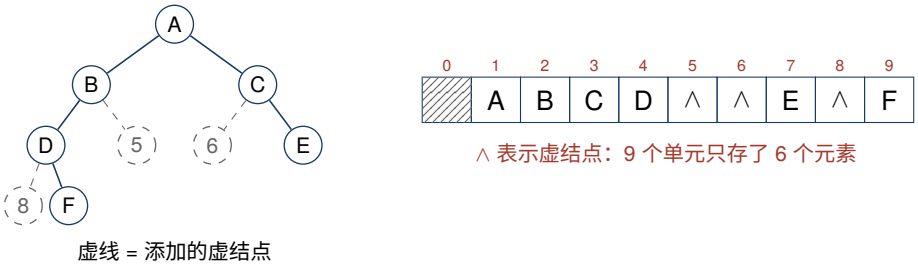


图 6: 课件 p26：完全二叉树的数组表示——B 在下标 2，它的孩子 D、E 在 4、5

对**一般二叉树**，先补上虚结点，把它“补成”完全二叉树，再按编号存放（课件 p27）：



易错警示：顺序存储的两个缺点（笔记 p586 · 课件 p28）

①最坏情况浪费惊人：深度为 k 且只有 k 个结点的单支树，需要长度为 2^k-1 的数组。例如右单支树 1→2→3（$k=3$）的数组为 `[1, ∧, 2, ∧, ∧, ∧, 3]`：7 个单元只用了 3 个（笔记 p586）。课件 p28 的 6 结点右单支树需要 $2^6-1=63$ 个单元。

②只适合满二叉树和完全二叉树。插入、删除结点还可能要求整体搬动子树的位置。

C++：顺序存储二叉树的核心（节选自课程源程序 SqBinaryTree）

```cpp
enum SqBinTreeNodeTagType { EMPTY_NODE, USED_NODE };  // 结点是否为空（虚）结点
template <class ElemType>
struct SqBinTreeNode {
    ElemType data;
    SqBinTreeNodeTagType tag;          // 标志：区分真结点与虚结点
};
// 以下为 SqBinaryTree<ElemType> 的成员（elems 数组，0 号单元不用，root = 1）
int LeftChild(const int cur) const  { return 2 * cur; }       // 性质 5
int RightChild(const int cur) const { return 2 * cur + 1; }
int Parent(const int cur) const     { return cur / 2; }      // 整数除法即向下取整
bool NodeEmpty(int cur) const {         // 越界或标为虚结点，都当作"空"
    return cur < 1 || cur > maxSize || elems[cur].tag == EMPTY_NODE;
}
```

代码解释：`tag` 是顺序存储特有的——因为数组里每个格子都“存在”，必须另用标志说明它是不是虚结点。`NodeEmpty` 把“下标越界”也当成空，这样递归遍历时 `PreOrderHelp(LeftChild(r))` 走到数组外会自然停止。课程源程序的测试树 2(3(,6),4(7,)) 中，7 存在下标 6，`Parent(6)`= 3 号的 4，运行结果“6 号节点是 7, 它的双亲是 4”与此一致。

➢ 对照：顺序存储二叉树 ↔ C++ std::priority_queue 的“堆”（思想重现型，后续“排序”一章）

一句话定位：二叉堆就是“始终保持完全二叉树形状”的顺序存储二叉树，因此永远没有虚结点、不浪费空间。

维度	本节顺序存储	二叉堆
形状	任意二叉树（补虚结点）	必为完全二叉树
下标起点	课程代码从 1 开始	教材常从 1，C++ 标准库 `std::push_heap` 等从 0
父子公式	$\lfloor i/2 \rfloor$, $2i$, $2i+1$	从 0 起：$\lfloor (i-1)/2 \rfloor$, $2i+1$, $2i+2$
额外约束	无	双亲 $\geq$（或 $\leq$）孩子

迁移判断：✓“完全二叉树用数组最省空间”直接迁移；△公式要按下标起点换算；✗普通二叉树不满足堆序，不能用堆的“上浮/下沉”操作。

3.2　链式存储：用指针显式记录关系

定义：二叉链表（课件 p40–42）

每个结点含三个域：`leftChild`（指向左孩子）、`data`（数据）、`rightChild`（指向右孩子）；孩子不存在时指针为空（图中记为 ∧）。整棵树只需一个指向根的指针 `root`。

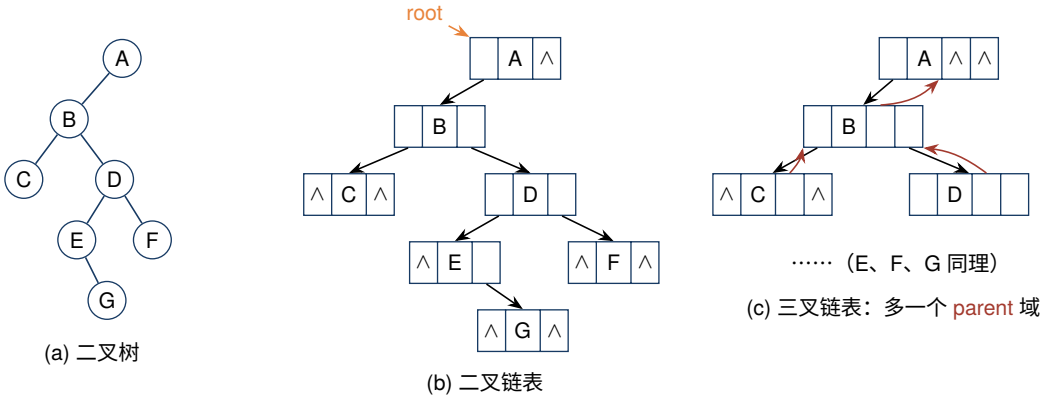


图 7: 课件 p41、笔记 p586–587 示例：同一棵树的三种画法

C++：二叉链表结点类模板（课程源程序）

```
template <class ElemType>
struct BinTreeNode {
    ElemType data;                          // 数据域
    BinTreeNode<ElemType> *leftChild;       // 左孩子指针
    BinTreeNode<ElemType> *rightChild;      // 右孩子指针
    BinTreeNode() { leftChild = rightChild = NULL; }          // 构造叶结点
    BinTreeNode(const ElemType &e, BinTreeNode<ElemType> *lChild = NULL,
                BinTreeNode<ElemType> *rChild = NULL)
    { data = e; leftChild = lChild; rightChild = rChild; }
};
```

二叉链表中的空指针个数

结论（笔记 p586）

在含 n 个结点的二叉链表中，有 $\boxed{n+1}$ 个空指针域。

证明

每个结点有 2 个指针域，共 $2n$ 个。从下往上看，除根以外每个结点恰好被它双亲的一个指针指向，所以**非空**指针恰好 $n-1$ 个（等于边数）。于是空指针个数 $= 2n-(n-1) = n+1$。■

验证：上图 7 个结点，数 ∧：A 右、C 左右、E 左、F 左右、G 左右，共 $1+2+1+2+2 = 8 = 7+1$ ✓。

拓展联想（当前不要求掌握）：这 $n+1$ 个“浪费”的空指针，可以改为指向遍历次序下的前驱/后继，得到“线索二叉树”（教材 6.4，本课程跳过）。

三叉链表解决什么问题？ 二叉链表只能“向下走”，求 `Parent(cur)` 必须从根开始递归搜索（课件 p51 的 `ParentHelp`，最坏要访问所有结点）。三叉链表多存一个 `parent` 指针，求双亲只需一步，代价是每个结点多一个指针、插入删除时多维护一个域。**以空间换时间**。

存储方式	空间	求孩子	求双亲 / 适用
顺序存储	完全二叉树最省；一般树可能极浪费	$O(1)$，算下标	$O(1)$；适合完全二叉树（堆）
二叉链表	$3n$ 个域，其中 $n+1$ 个空指针	$O(1)$，跟指针	需从根搜索；**最常用**（课件 p41）
三叉链表	$4n$ 个域	$O(1)$	$O(1)$；经常要向上找双亲时

➢ 对照：二叉链表 ↔ 单链表 / 双向链表（推广型，第 2 章）

一句话定位：单链表结点“一个指针指向唯一后继”，二叉链表结点“两个指针指向至多两个后继”；三叉链表加回指双亲的指针，正如双向链表加了 `prior`。

迁移判断：✓“用 `new` 申请结点、用 `delete` 释放”直接迁移；✓“空指针 `NULL` 表示没有后继”直接迁移；✗“从头指针出发一个循环就能访问全部结点”不成立——二叉链表有两条分支，必须用递归（或栈/队列）才能走遍，这引出下一节的遍历。

✍ 生活中的它：寻宝游戏的线索卡片

生活场景：班级寻宝游戏中，每张卡片写着一条线索，背面写“下一张卡片 A 藏在……”“下一张卡片 B 藏在……”，没有下一张就写“终点”。

对应关系：卡片 = 结点；线索内容 = `data`；背面的两个藏匿地点 = `leftChild`、`rightChild`；“终点” = 空指针；第一张卡片的位置 = `root`。卡片可以藏在校园任何地方（内存不连续），只要地点写对就能找到。

小判断：有 10 张卡片，背面一共有多少个“终点”？$n+1 = 11$ 个。

边界：这只是帮助理解“指针 = 地址”的类比；真实内存中的指针由操作系统和 `new` 分配。

学完这里，你应该能够回答

(1) 顺序存储为什么要设 `tag`？ (2) 深度为 5 的单支树顺序存储至少要多长的数组？(31) (3) 写出二叉链表结点的 C++ 定义。 (4) 证明 n 个结点有 $n+1$ 个空指针。

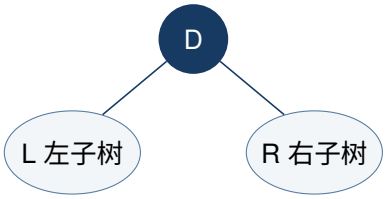
4　遍历二叉树 [课件 6.3.1–6.3.2 • 笔记 p587–593]

到这里，我们已经解决了什么？二叉树已经能存进内存了。**还剩什么问题？**线性表只要从头走到尾就能访问每个元素；二叉树的每个结点有两个“后继”，走了左边，右边怎么办？我们需要一套**规则**，保证每个结点都被访问、且只访问一次——这就是遍历。几乎所有二叉树算法（求结点数、求高度、复制、建树、求值）都是“遍历 + 把访问换成别的操作”。

4.1　遍历的定义与六种次序

定义：遍历（Traversal，笔记 p587 · 课件 p57）

顺着某一条搜索路径**巡访**二叉树中的结点，使得每个结点**均被访问一次，而且仅被访问一次**（又称“周游”）。目的：得到树中所有结点的一个**线性排列**。“访问”的含义可以很广，如输出结点信息、修改数据、计数等。



二叉树 = 根 D + 左子树 L + 右子树 R，三部分依次遍历即遍历整棵树。
三者排列共 $3! = 6$ 种：DLR、LDR、LRD、DRL、RDL、RLD。
规定**先左后右**，只剩 3 种（以“根”的位置命名）：

先（根）序 DLR　中（根）序 LDR　后（根）序 LRD

另有不按此递归模式的**层次遍历**：自上而下、自左而右。

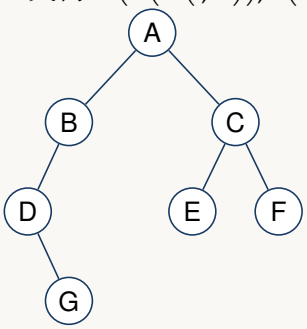
4.2　先序、中序、后序遍历

先序遍历 DLR	中序遍历 LDR	后序遍历 LRD
若二叉树为空，则空操作；否则	若二叉树为空，则空操作；否则	若二叉树为空，则空操作；否则
(1) 访问根结点；	(1) 中序遍历左子树；	(1) 后序遍历左子树；
(2) 先序遍历左子树；	(2) 访问根结点；	(2) 后序遍历右子树；
(3) 先序遍历右子树。	(3) 中序遍历右子树。	(3) 访问根结点。

三个定义都是**递归**的：“遍历左子树”本身又是一次完整的遍历。手算时最可靠的方法是**逐层展开**：先把整棵树写成“根 [左子树] [右子树]”的三段，再把每个方括号继续展开，直到只剩单个结点。

例题 4.1（教材图 6.9 · 课程源程序）

对二叉树 A(B(D(,G)),C(E,F)) 分别写出先序、中序、后序序列。



先序：A[B 的子树][C 的子树] → A[B[D 的子树]][CEF] → AB[DG]CEF → **A B D G C E F**

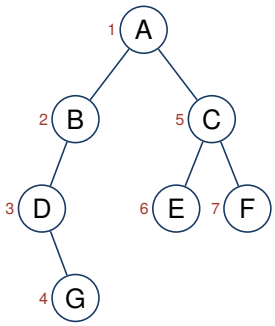
中序：[B 的子树]A[C 的子树] → [[D 的子树]B]A[ECF] → [DG]BAECF → **D G B A E C F**

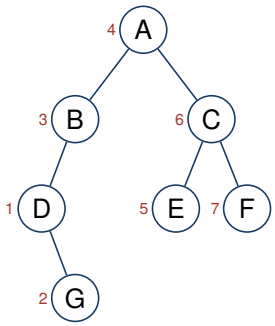
注意 D 的左子树为空，所以“中序遍历 D 子树”＝空，D，G。

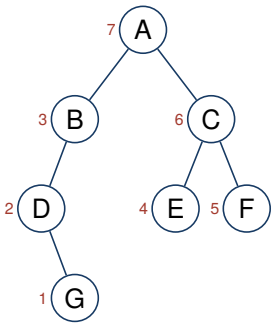
后序：[B 的子树][C 的子树]A → [[D 的子树]B][EFC]A → [GD]BEFCA → **G D B E F C A**

检查：三个序列都是 7 个字母且互不重复 ✓；先序第一个、后序最后一个都是根 A ✓；中序中 A 左边的 D G B 恰是左子树的结点 ✓。以上结果与课程源程序 `先序遍历.cpp`、`中序遍历.cpp`、`后序遍历.cpp` 的运行结果完全一致。

下面三幅图把每个结点被访问的**次序号**直接标在结点旁，可以直观看出三种次序的“性格”：先序是“见到就访问”，中序是“左边访问完才访问”，后序是“孩子全访问完才访问”。





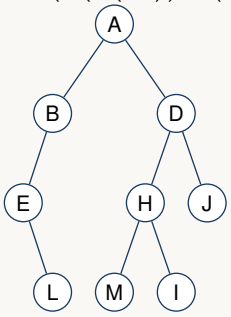


先序：ABDGCEF　　中序：DGBAECF　　后序：GDBEFCA

中序序列的一个好用的几何性质　把二叉树“压扁”到一条水平线上：只要画图时左子树整体在根的左边、右子树整体在右边，那么**从左到右读结点的横坐标顺序，就是中序序列**。上面中图中 D、G、B、A、E、C、F 从左到右排列正是如此。这条性质在“由序列还原二叉树”（§5.1）时非常有用。

例题 4.2（笔记 p588–589）

二叉树 A(B(E(,L)),D(H(M,I),J))：先序 **ABELDHMIJ**；中序 **ELBAMHIDJ**；后序 **LEBMIHJDA**。



易错核对：E 只有右孩子 L，所以中序是“E 再 L”（ELB…）而不是“L E”；后序是“L E”。B 只有左子树，中序里 B 在 E、L 之后、A 之前。用“压扁”性质验证中序：横坐标从左到右是 E L B A M H I D J ✓。

C++ 递归实现（课件 p62–67 · 课程源程序）

C++：先序 / 中序 / 后序：只有“访问根”这一行的位置不同

```
template <class ElemType>
void BinaryTree<ElemType>::PreOrderHelp(const BinTreeNode<ElemType> *r,
                                        void (*visit)(const ElemType &)) const {
    if (r != NULL) {                        // 递归出口：空树什么也不做
        (*visit)(r->data);                  // D 访问根       ← 先序：放在最前
        PreOrderHelp(r->leftChild, visit); // L 遍历左子树
        PreOrderHelp(r->rightChild, visit);// R 遍历右子树
    }
}
// 中序 InOrderHelp：   InOrderHelp(L); (*visit)(r->data); InOrderHelp(R);
// 后序 PostOrderHelp：PostOrderHelp(L); PostOrderHelp(R); (*visit)(r->data);
template <class ElemType>
void BinaryTree<ElemType>::PreOrder(void (*visit)(const ElemType &)) const {
    PreOrderHelp(root, visit);              // 公有接口：从根开始
}
```

代码解释：(1) `visit` 是**函数指针**，调用者传入 `Show<char>` 就是“输出”，传入别的函数就是别的操作——遍历框架与访问动作分离。(2) 为什么要有 `PreOrder` 和 `PreOrderHelp` 两层？递归需要“当前子树的根 `r`”作参数，而用户只知道整棵树；公有函数把 `root` 传给私有的 Help 函数，既隐藏了 `root`，又能递归。(3) `const BinTreeNode<ElemType> *r` 表示遍历不修改结点。

递归到底是怎么执行的？（笔记 p592–593）

以 A(B(,D),C) 的先序遍历为例。每次调用 `PreOrderHelp(r)` 都展开成“D、L、R”三步；遇到空指针 ∧ 立即返回。

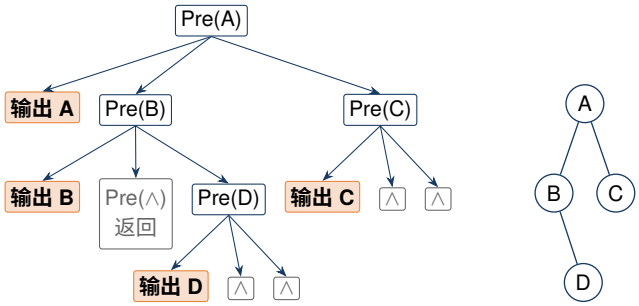
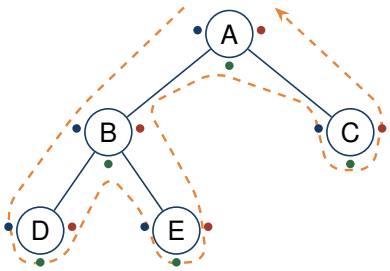


图 8: 递归调用树：从左到右读橙色框，得先序 A B D C（与笔记 p592 一致）

中序、后序的调用树形状**完全相同**，只是“输出”框分别排在中间或最后。笔记 p593 总结：“若去掉输出语句，从递归角度看，三种算法完全相同，即三种算法访问路径相同，只是访问结点的时机不同。”



沿虚线从根出发绕树一周，每个结点恰好**经过 3 次**：

- 第 1 次经过（左侧）时访问 → 先序 A B D E C
- 第 2 次经过（下方）时访问 → 中序 D B E A C
- 第 3 次经过（右侧）时访问 → 后序 D E B C A

复杂度（笔记 p593）：每个结点访问 3 次（常数），时间 $O(n)$；递归栈深度等于树高 h，空间 $O(h)$，最坏（单支树）为 $O(n)$。

图 9: 笔记 p593“从虚线的出发点到终点的路径上，每个结点经过 3 次”

➢ 对照：递归遍历 ↔ 数学归纳法（思想重现型）

一句话定位：写递归函数和做归纳证明是同一种思维——“假设子问题已经解决，只处理当前一层”。

数学归纳法	递归遍历	对应关系
归纳基础 $P(1)$ 成立	`if (r == NULL)` 空操作	递归出口
归纳假设：对更小的情形成立	相信 `PreOrderHelp(r->leftChild)` 能正确遍历左子树	子问题
归纳步骤：由假设推出 $P(k)$	访问根 + 两次递归调用	当前一层的工作

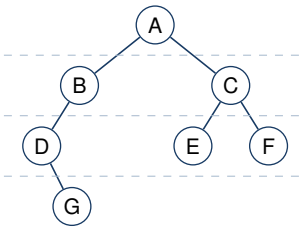
迁移判断：✓“不必追踪每一层递归、只要基础和步骤正确就整体正确”直接迁移；✗数学归纳法是证明，没有执行开销；递归是计算，每层调用占用栈空间，树太深可能栈溢出（附录 B 的非递归算法用显式栈解决）。

4.3　层次遍历：借助队列 [课件 p68–70 • 笔记 p593]

先、中、后序都是“一条路走到底再回头”（深度优先）。层次遍历则是**一层一层**地访问：从根开始，自上而下、自左而右。问题是：访问完 B 之后要访问 C，但 B 和 C 之间并没有指针相连——**必须有一个地方“记住”已经发现但还没访问的结点**，并且先发现的先访问。这正是**队列**（先进先出）。

层次遍历算法（笔记 p593）

(a) 将根结点入队；(b) 队列不空时循环：从队列中出队一个结点 p，访问它；若 p 有左孩子，左孩子入队；若 p 有右孩子，右孩子入队。



操作	队列（左为队头）	已输出
A 入队	A	
A 出队，B、C 入队	B C	A
B 出队，D 入队	C D	A B
C 出队，E、F 入队	D E F	A B C
D 出队，G 入队	E F G	A B C D
E 出队	F G	A B C D E
F 出队	G	A B C D E F
G 出队，队空，结束	（空）	A B C D E F G

图 10: 教材图 6.9 的层次遍历：队列变化过程（与课程源程序 **层次遍历.cpp** 的注释一致）

C++：层次遍历（课程源程序，链队列存放结点指针）

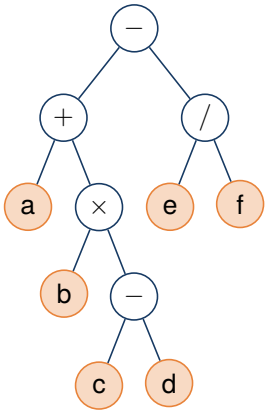
```cpp
template <class ElemType>
void BinaryTree<ElemType>::LevelOrder(void (*visit)(const ElemType &)) const {
    LinkQueue<const BinTreeNode<ElemType> *> q;   // 队列元素是"结点指针"
    const BinTreeNode<ElemType> *cur;
    if (root != NULL) q.InQueue(root);            // 根入队
    while (!q.Empty()) {                          // 还有已发现未访问的结点
        q.OutQueue(cur);                          // 队头出队
        (*visit)(cur->data);                      // 访问
        if (cur->leftChild != NULL)  q.InQueue(cur->leftChild);   // 先左
        if (cur->rightChild != NULL) q.InQueue(cur->rightChild);  // 后右
    }
}
```

代码解释：队列里放的是**指针**而不是数据，因为出队后还要通过它找孩子。“先左后右”入队保证了同一层自左而右。笔记 p593 例 a(b(c,d(e,)),f(g(,h),)) 的层次序列为 **abfcdgeh**；课件 p69 例 A(B(C,D),E(,G)) 为 **ABECDG**。

> **➢ 对照：层次遍历 ↔ 图的广度优先搜索 BFS（推广型，第 7 章）**
>
> **一句话定位**：层次遍历就是树上的 BFS；树上没有环，所以不必记录“是否访问过”。
> **迁移判断**：✓“用队列、先发现先访问”直接迁移；✗“每个结点只会被发现一次”在图中不成立——图有环、一个顶点可被多个邻居发现，必须加 `visited` 标记，否则死循环；△层次遍历的“层”在图中对应“到起点的最短边数”。

4.4　用二叉树表示表达式 [课件 p61–68 • 笔记 p590]

笔记 p582“案例引入 2：利用二叉树求解表达式的值”。把运算符放在分支结点、操作数放在叶子，每个运算符的左、右子树分别是它的左、右运算对象：



表达式 $a+b\times(c-d)-e/f$ 的表达式树（课件 p14、笔记 p590）：

遍历	序列	名称
先序 DLR	$-+a\times b-cd/ef$	前缀表达式（波兰式）
中序 LDR	$a+b\times c-d-e/f$	中缀表达式（**丢了括号**）
后序 LRD	$abcd-\times+ef/-$	后缀表达式（逆波兰式）
层次	$-+/a\times efb-cd$	（课件 p68）

易错警示：中序序列不等于原表达式

中序 $a+b\times c-d-e/f$ 若按通常优先级计算，会算成 $a+bc-d-e/f$，与原式 $a+b(c-d)-e/f$ 不同。**表达式树本身由结构表达了运算顺序，所以不需要括号；中序输出时若要还原原式，必须给每个子表达式补上括号**。前缀、后缀表达式则不需要括号也不会产生歧义——这正是计算机偏爱后缀式的原因。

为什么后序遍历 = 计算顺序？ 要算出根上的“−”，必须先算出它的左运算对象（左子树）和右运算对象（右子树）——“孩子先于双亲”，这正是后序。用后缀式求值：从左到右扫描，遇到操作数入栈，遇到运算符就弹出两个数运算再压回（第 3 章“栈”的经典应用）。附录 A 的 `Evaluate` 函数就是“后序遍历 + 访问根 = 做运算”。

➢ 对照：表达式树的后序遍历 ↔ 第 3 章“栈求后缀表达式的值”（思想重现型）

一句话定位：两者得到相同的计算顺序：递归后序遍历用的是系统调用栈，后缀式求值用的是显式操作数栈。

迁移判断：✓运算次序完全一致；△栈方法要求先把表达式转成后缀串，树方法要求先建好树；✗中缀串不能直接逐个字符入栈求值，需要运算符优先级处理（第 3 章的双栈法）。

✍ 生活中的它：逆波兰计算器

生活场景：一些科学计算器（如惠普 HP 系列）采用“逆波兰”输入：算 $(4+5)/(6+7)$ 时按 `4 ENTER 5 + 6 ENTER 7 + ÷`，全程不按括号。

对应关系：按键顺序 = 表达式树 $/\,(+(4,5),+(6,7))$ 的**后序遍历**；计算器内部的“栈”= 暂存中间结果。

小估算：按此方法依次得到 9、13、$9/13\approx 0.692$。

边界：大多数手机计算器用的是中缀输入，由软件在内部完成转换。

学完这里，你应该能够回答

(1) 三种递归遍历的代码差在哪一行？ (2) 层次遍历为什么用队列而不用栈？ (3) 中缀序列为什么会丢括号？ (4) 遍历的时间、空间复杂度？ (5)“三次经过”分别对应哪种遍历？

5　遍历的应用 [课件 6.3.3 • 笔记 p590–594]

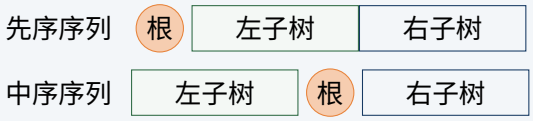
到这里，我们已经解决了什么？我们能把一棵树“读”成序列。**还剩什么问题？**反过来，给出序列能不能“写”回一棵树？以及：怎样把遍历框架用于计数、求高、复制等实际任务？本节回答这两类问题。

5.1　由遍历序列确定二叉树

结论（笔记 p590 • 课件 p82）

结点值互不相同时，由二叉树的**先序 + 中序**，或**后序 + 中序**，或**层次 + 中序**，都可以唯一确定一棵二叉树；但**先序 + 后序不可以**（笔记红字“先序 + 后序不可以!”）。

为什么必须有中序？ 先序（或后序）告诉我们**谁是根**：先序的第一个、后序的最后一个。但它们不告诉我们根之后哪些结点属于左子树、哪些属于右子树。**中序把根夹在中间**：根左边的全是左子树，右边的全是右子树——只有中序能“切开”左右。课件 p82 的示意：



两段“左子树”长度相同：
先在中序中找到根的位置，
数出左子树有几个结点，
再到先序中截取同样长度。

例题 5.1（笔记 p591 例 1）先序 + 中序

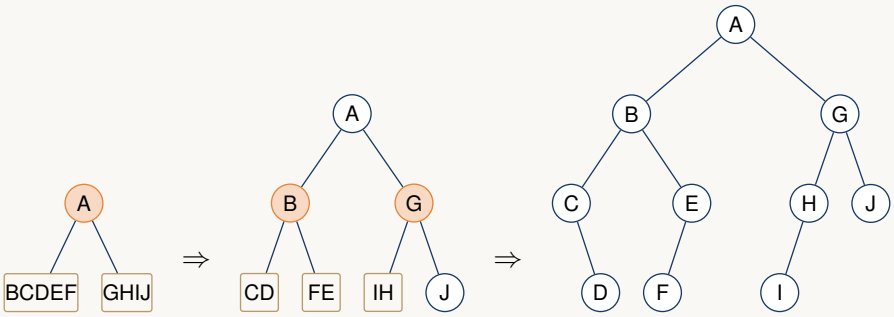
先序 ABCDEFGHIJ，中序 CDBFEAIHGJ，构造二叉树。

第一步：先序第一个 A 是根。在中序中找到 A：左边 **CDBFE**（5 个）是左子树，右边 **IHGJ**（4 个）是右子树。相应地先序 A 后面的前 5 个 **BCDEF** 是左子树先序，剩下 **GHIJ** 是右子树先序。

第二步（左子树，先序 BCDEF / 中序 CDBFE）：根 B；中序中 B 左边 CD 为左子树、右边 FE 为右子树。

第三步：CD（先序 CD / 中序 CD）根 C，D 在 C 右边 → D 是 C 的**右**孩子；EF（先序 EF / 中序 FE）根 E，F 在 E 左边 → F 是 E 的**左**孩子。

第四步（右子树，先序 GHIJ / 中序 IHGJ）：根 G，左子树 IH，右子树 J；IH（先序 HI / 中序 IH）根 H，I 是 H 的左孩子。

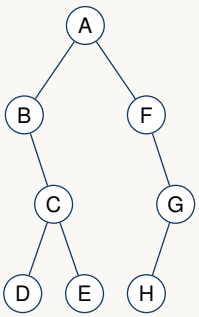


检查：对结果求先序得 ABCDEFGHIJ、中序得 CDBFEAIHGJ，与题设一致 ✓（已用程序验证）。

例题 5.2（笔记 p591 例 2）中序 + 后序

中序 BDCEAFHG，后序 DECBHGFA，求二叉树。

关键区别：**后序的最后一个**是根。A 为根；中序 BDCE | A | FHG。左子树后序取前 4 个 DECB，根为其最后一个 B；中序 B 左边为空、右边 DCE → B 只有右子树。DCE 的后序 DEC，根 C，D 左、E 右。右子树后序 HGF，根 F；中序 FHG 中 F 左边为空 → F 只有右子树 HG；HG 后序 HG，根 G，H 在 G 左边。



检查：先序 ABCDEFGH；中序 BDCEAFHG ✓；后序 DECBHGFA ✓。

易错：后序中“左子树后序”是从左边截取与中序左子树等长的一段，**右子树后序**是紧接着的一段（不含最后的根）。

证明：为什么先序 + 后序不能唯一确定（反例）

先序 AB、后序 BA。下面两棵二叉树都满足：



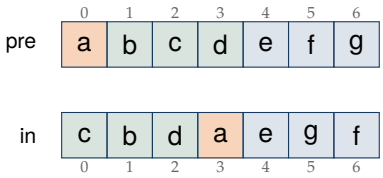
先序、后序都只告诉我们“A 是根，B 是它的孩子”，却无法分辨 B 在左还是在右；只有中序（BA 或 AB）能分辨。一般地，只要某结点只有一个孩子，就会出现这种歧义。

C++：由先序与中序构造二叉树（课件 p84）

C++：CreateBinaryTreeHelp：区间下标是最容易错的地方

```cpp
template <class ElemType>
void CreateBinaryTreeHelp(BinTreeNode<ElemType> *&r, ElemType pre[], ElemType in[],
                          int preLeft, int preRight, int inLeft, int inRight) {
    if (preLeft > preRight || inLeft > inRight) { r = NULL; return; } // 区间为空→空树
    r = new BinTreeNode<ElemType>(pre[preLeft]);   // 先序区间第一个元素是根
    int mid = inLeft;
    while (in[mid] != pre[preLeft]) mid++;         // 中序中根的位置
    int leftLen = mid - inLeft;                    // 左子树结点个数
    CreateBinaryTreeHelp(r->leftChild,  pre, in,   // 左子树：先序紧跟根的 leftLen 个
        preLeft + 1, preLeft + leftLen, inLeft, mid - 1);
    CreateBinaryTreeHelp(r->rightChild, pre, in,   // 右子树：先序剩余部分
        preLeft + leftLen + 1, preRight, mid + 1, inRight);
}
```

代码解释：课件原代码写的是 `preLeft + mid - inLeft`，就是这里的 `preLeft + leftLen`；引入 `leftLen` 只是让含义更清楚。参数 `r` 是**指针的引用** `*&r`，这样函数内 `r = new ...` 才能改到调用者的指针（同 `DestroyHelp`）。



课件 p83：pre = abcdefg，in = cbdaegf
根 = pre[0] = a，在 in 中 mid = 3，leftLen = 3
左子树：pre[1..3] = bcd，in[0..2] = cbd
右子树：pre[4..6] = efg，in[4..6] = egf

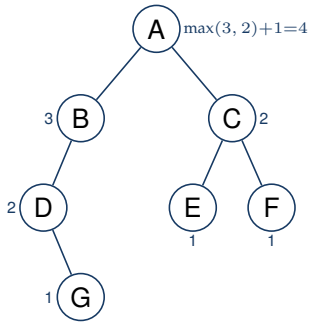
5.2　统计、求高、复制：把“访问”换成别的事

所有这些算法都套用同一个模板：**空树返回边界值；否则递归处理左右子树，再在根处合并结果**（本质是后序）。

问题	递归关系（r 为根，L, R 为左右子树）	空树时
结点数 `NodeCount`	$N(r) = N(L) + N(R) + 1$	0
叶子数 `LeafCount`	r 无孩子时为 1；否则 $\text{Leaf}(L) + \text{Leaf}(R)$	0
高度 `Height`	$H(r) = \max\{H(L), H(R)\} + 1$	0
结点值之和（作业 7）	$S(r) = S(L) + S(R) + r \to \text{data}$	0
复制 `CopyTree`	先复制左、右子树，再 new 根，挂上两棵副本	`NULL`
销毁 `Destroy`	先销毁左、右子树，再 delete 根（必须后序！）	什么也不做

C++：叶子数、高度、结点值之和（附录 A 中已编译运行）

```cpp
template <class ElemType>
int BinaryTree<ElemType>::LeafCountHelp(const BinTreeNode<ElemType> *r) const {
    if (r == NULL) return 0;                                    // 空树
    if (r->leftChild == NULL && r->rightChild == NULL) return 1;    // r 是叶子
    return LeafCountHelp(r->leftChild) + LeafCountHelp(r->rightChild);
}
template <class ElemType>
int BinaryTree<ElemType>::HeightHelp(const BinTreeNode<ElemType> *r) const {
    if (r == NULL) return 0;                    // 空二叉树高为 0
    int lHeight = HeightHelp(r->leftChild);     // 左子树的高
    int rHeight = HeightHelp(r->rightChild);    // 右子树的高
    return (lHeight > rHeight ? lHeight : rHeight) + 1;
}
template <class ElemType>                       // 课件作业 7：结点数值之和
ElemType BinaryTree<ElemType>::SumHelp(const BinTreeNode<ElemType> *r) const {
    if (r == NULL) return 0;
    return r->data + SumHelp(r->leftChild) + SumHelp(r->rightChild);
}
```



HeightHelp 在教材图 6.9 上的返回值（标在结点旁）：叶子 G、E、F 返回 1；D 返回 $\max(0,1)+1=2$；B 返回 $\max(2,0)+1=3$；C 返回 $\max(1,1)+1=2$；根 A 返回 $\max(3,2)+1=4$。

同一棵树：`NodeCount`$=7$，`LeafCount`$=3$（G、E、F），$n_2=2$（A、C），验证 $n_0=n_2+1$ ✓。

易错警示：两个写递归时的典型错误

①叶子判断漏写空树出口：若省略 `if (r == NULL) return 0;`，对只有一个孩子的结点递归到空孩子时会访问 `NULL->leftChild`，程序崩溃。

②销毁顺序写成先序：若先 `delete r` 再访问 `r->leftChild`，就是访问已释放的内存。销毁必须**后序**；复制则左右子树和根的先后均可，但课件写法是先复制子树再建根。

复制为什么要“深复制”？ 课件 p79–80：`BinaryTree(const BinaryTree &source)` 调用 `CopyTreeHelp(source.root)` 生成一套全新的结点。若只复制 `root` 指针（浅复制），两棵树共用同一批结点，一棵析构时 `DestroyHelp` 释放了结点，另一棵就成了“悬空指针”。这就是 C++ 中“有指针成员的类必须自定义复制构造函数和赋值运算符”的道理（课程代码的 `operator=` 先 `DestroyHelp(root)` 再复制，并判断 `&source != this` 防止自赋值）。

例题 5.3（课件作业 7）计算二叉树中所有结点的数值之和

二叉树 1(2(4,5),3(,6))。由 $S(r)=S(L)+S(R)+r$ → data：$S(4)=4$，$S(5)=5$，$S(2)=4+5+2=11$；$S(6)=6$，$S(3)=0+6+3=9$；$S(1)=11+9+1=$ **21**。附录 A 的程序输出

“作业 7 结点值之和 = 21”。**本题想说明**：任何“汇总型”问题都可以写成“左结果 ⊕ 右结果 ⊕ 根”的形式。

用带空标记的先序串建树（常见考法）

只给先序序列不能确定二叉树，但如果把**空子树也用'#' 写出来**，先序序列就能唯一确定树——因为每个'#' 精确地告诉我们“这里结束了”。例如教材图 6.9 写作 ABD#G###CE##F##。

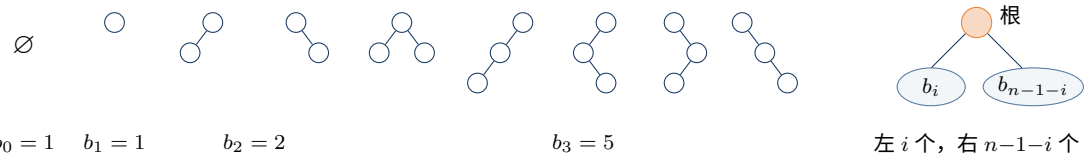
C++：按先序读入建树

```cpp
BinTreeNode<char> *CreateByPreString(const char *s, int &i) {
    char c = s[i++];                          // 读下一个字符
    if (c == '#' || c == '\0') return NULL;    // '#' 表示空子树
    BinTreeNode<char> *r = new BinTreeNode<char>(c);   // 先建根（先序）
    r->leftChild  = CreateByPreString(s, i);   // 再建左子树
    r->rightChild = CreateByPreString(s, i);   // 最后建右子树
    return r;
}
```

检查方法：串中'#' 的个数一定等于“字母个数 + 1”——这正是 n 个结点的二叉链表有 $n+1$ 个空指针（§3.2）。图 6.9 的串有 7 个字母、8 个'#' ✓。

5.3　二叉树的计数：卡特兰数 [课件 p74–75]

问题：n 个结点能构成多少种不同形态的二叉树？记为 b_n。



核心公式：递推式与卡特兰公式

根占 1 个结点，左子树取 i 个（$i = 0, 1, \ldots, n-1$），右子树取 $n-1-i$ 个，两边形态独立相乘，再对 i 求和（乘法原理 + 加法原理）：

$$b_0 = 1, \qquad b_n = \sum_{i=0}^{n-1} b_i \, b_{n-1-i} \quad (n \geq 1);$$

$$\boxed{b_n = \frac{1}{n+1}\binom{2n}{n} = \frac{1}{n+1} \cdot \frac{(2n)!}{n!\,n!}}$$

数列：$b_0, b_1, \cdots = 1, 1, 2, 5, 14, 42, 132, \ldots$（已用程序验证前 8 项）。例：$b_3 = b_0 b_2 + b_1 b_1 + b_2 b_0 = 2 + 1 + 2 = 5$；$b_4 = \frac{1}{5}\binom{8}{4} = \frac{70}{5} = 14$（课件 p75 画出了 4 个结点的 14 种二叉树）。

严格推导说明：由递推式到闭式需要“生成函数”工具，超出本课程要求；考试只需会用递推或公式计算前几项。

➢ 对照：二叉树计数 ↔ 第 3 章“出栈序列个数”（思想重现型）

一句话定位：$1, 2, \ldots, n$ 依次入栈、可随时出栈，可能得到的出栈序列数同样是卡特兰数 b_n；n 对括号的合法匹配方案数也是。

迁移判断：✓3 个元素出栈序列数 $= b_3 = 5$（123 进栈时，不可能出现 312）；△两者“都是 b_n”是因为它们都满足同一个递推式，而不是同一个问题——要在题目里认出这个递推结构；✗一般树（不分左右）的形态数不是 b_n（3 个结点的有序树只有 2 种）。

学完这里，你应该能够回答

(1) 为什么中序必不可少？　(2) 中序 + 后序时如何截取右子树后序？　(3) 写出 LeafCountHelp。(4) 销毁为什么必须后序？　(5) b_4 =？（14）

6　树和森林 [课件 6.5 · 笔记 p594–601]

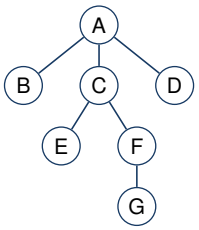
到这里，我们已经解决了什么？二叉树的存储、遍历和各种算法我们都会了。**还剩什么问题？**现实中的树大多是多叉的（文件夹、组织架构）。§2 开头说“所有树都能转化为唯一对应的二叉树”——本节就兑现这句话：先看一般树怎么存，再看怎样把树和森林变成二叉树，从而复用二叉树的全部算法。

6.1　树的三种存储结构

(1) 双亲表示法

定义：双亲表示法（笔记 p594 · 课件 p87）

用一个结构数组存放树的结点，每个结点含两个域：**数据域**（结点本身的信息）和**双亲域**（双亲结点在数组中的下标；根的双亲域为 −1）。



课件 p87 的树

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	2
5	F	2
6	G	5

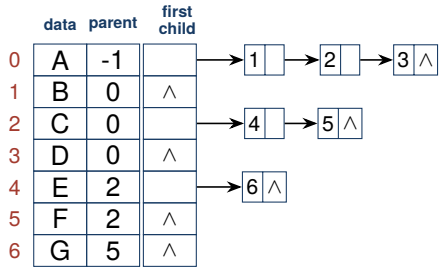
找双亲容易：E 的双亲 = `parent[4]` = 2，即 C，一步到位。
找祖先：沿 parent 一直走到 −1。G → 5(F) → 2(C) → 0(A)，所以 G 的祖先是 F、C、A。
找孩子难：要找 C（下标 2）的孩子，必须扫描整个数组，找出 parent 为 2 的结点（E、F）。
笔记 p594–595 的例子是 10 个结点的树 R(A(D,E),B,C(F(G,H,I)))，r=0，n=10，同样的读法。

C++：双亲表示法的结点（附录 A 中已运行“G 的祖先：F C A”）

```cpp
template <class ElemType>
struct ParentTreeNode {
    ElemType data;      // 数据域
    int parent;         // 双亲在数组中的下标，根为 -1
};
// 求下标为 i 的结点的所有祖先
for (int k = pt[i].parent; k != -1; k = pt[k].parent) cout << pt[k].data << " ";
```

(2) 孩子链表表示法（孩子双亲表示法）

把每个结点的孩子结点排列成一个**单链表**，n 个结点就有 n 个孩子链表；n 个头指针又组成一个线性表（顺序存放）。若在表头数组中再保留双亲域，就得到“孩子双亲表示法”（笔记 p595、课件 p88），此时**找孩子容易，找双亲也容易**。



A 的孩子链表：1(B) → 2(C) → 3(D)
C 的孩子链表：4(E) → 5(F)
F 的孩子链表：6(G)
链表结点存的是孩子在表头数组中的**下标**

图 11: 课件 p88 的孩子双亲表示（笔记 p595 结论：找孩子容易，找双亲也容易）

(3) 孩子兄弟表示法（二叉链表表示法）

定义：孩子兄弟表示法（笔记 p596 • 课件 p89）

用二叉链表作树的存储结构，链表中每个结点的两个指针域分别指向该结点的**第一个孩子**（`firstChild`）和**下一个兄弟**（`nextSibling`）。

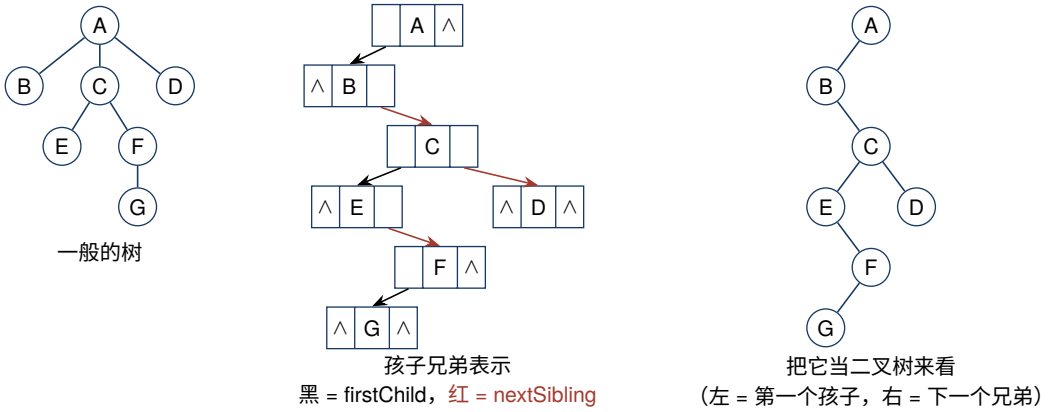


图 12: 课件 p90：同一个二叉链表，既是树的孩子兄弟表示，又是一棵二叉树

课件 p91 点明：“由此，树的各种操作均可对应二叉树的操作来完成。应当注意的是，和树对应的二叉树，其左、右孩子的概念已改变为：**左是树中第一个孩子，右是树中下一个兄弟**。”

例题 6.1（课件 p92 单选题）

利用二叉链表存储树，则根结点的右指针是（　）。A. 指向最左孩子　B. 指向最右孩子　C. 空　D. 非空

解：右指针指向“下一个兄弟”。根没有双亲，也就没有兄弟，所以右指针必为空，选 **C**。（若存的是森林，第一棵树根的右指针指向第二棵树的根，可以非空。）

C++：孩子兄弟表示的结点与树的先根、后根遍历

```cpp
template <class ElemType>
struct ChildSiblingNode {
    ElemType data;
    ChildSiblingNode<ElemType> *firstChild;    // 第一个孩子（相当于二叉树的左指针）
    ChildSiblingNode<ElemType> *nextSibling;   // 下一个兄弟（相当于二叉树的右指针）
};
template <class ElemType>
void TreePreRoot(const ChildSiblingNode<ElemType> *r) {   // 树的先根遍历
    if (r == NULL) return;
    cout << r->data << " ";                                // 先访问根
```

```
11      for (const ChildSiblingNode<ElemType> *c = r->firstChild; c != NULL; c = c->
        nextSibling)
12          TreePreRoot(c);                                    // 再依次先根遍历每棵子树
13  }
```

> ➢ **对照：孩子链表 ↔ 图的邻接表（推广型，第 7 章）**
>
> **一句话定位**：孩子链表就是“只记录指向孩子的边”的邻接表。**迁移判断**：✓“表头数组 + 每个顶点一条链”的结构直接迁移；✗树中每个结点只出现在一条孩子链里（只有一个双亲），图中一个顶点可出现在多条链里；△无向图的邻接表每条边存两次，树的孩子链表每条边只存一次。

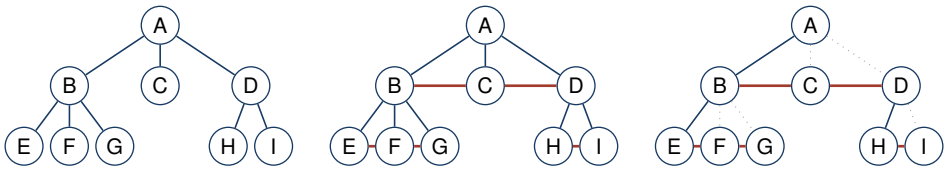
表示法	找双亲	找孩子	特点
双亲表示	容易（一步）	难（扫描全表）	结构最简单
孩子链表	难（不带 parent 时）	容易	带 parent 域后两者都容易
孩子兄弟	难	容易（先找第一个孩子，再沿兄弟链）	可直接套用二叉树算法

> ✍ **生活中的它：员工表里的“直属上级”一列**
>
> **生活场景**：公司人事表格每行一名员工，其中一列写“直属上级工号”，总经理那一格空着。
> **对应关系**：每一行 = 数组中的一个结点；“直属上级工号” = parent 域；总经理 = 根（parent 为 −1）。这就是双亲表示法。
> **小判断**：查“小王的上级的上级”只要看两次表格；查“张经理手下有谁”却得把整张表从头翻到尾——正对应“找双亲容易、找孩子难”。
> **边界**：组织架构是多叉树；真实公司可能存在“双线汇报”，此时就不再是严格的树。

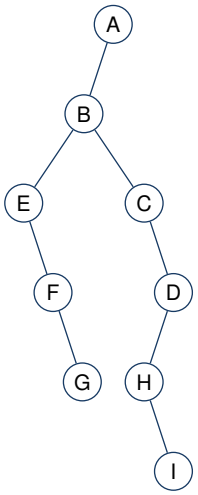
6.2　树与二叉树的转换 [笔记 p596–598 • 课件 p90–91]

树 → 二叉树：加线、抹线、旋转

①加线：在所有**兄弟**之间加一条连线；**②抹线**：对每个结点，除了与其**第一个（最左）孩子**的连线外，抹去它与其余孩子之间的连线；**③旋转**：以树根为轴心，将整棵树顺时针旋转约 45°，原来的“第一个孩子”成为左孩子，“兄弟”成为右孩子。



原树（笔记 p596）　①兄弟间加线　②只保留到第一个孩子的线



③旋转后得到二叉树（笔记 p597）。读法：
左分支 = 第一个孩子：A 的左孩子 B，B 的左孩子 E，D 的左孩子 H；
右分支 = 下一个兄弟：B→C→D 是兄弟链，E→F→G、H→I 也是。
检查：根 A 没有右孩子 ✓（树根无兄弟）；结点总数 9 个不变 ✓。

二叉树 → 树：加线、抹线、调整（笔记 p597–598）

①加线：若结点 p 是其双亲的左孩子，则把 p 的右孩子、右孩子的右孩子……（沿右分支找到的所有右孩子）都与 p 的双亲用线连起来；**②抹线**：抹掉原二叉树中所有双亲与右孩子的连线；**③调整**：把结点按层次排列，形成树结构。
前提：这棵二叉树的根**没有右孩子**（否则转换出来的是森林）。

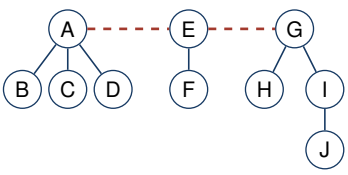
> **一句话记忆**　**“左孩子右兄弟”**。树 → 二叉树：长子当左孩子，弟弟当右孩子；二叉树 → 树：左孩子是长子，沿右链走到的都是长子的弟弟，它们共用长子的双亲。

6.3　森林与二叉树的转换 [笔记 p598–599 • 课件 p94–96]

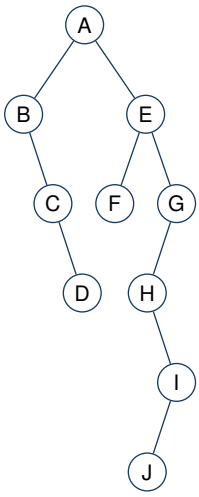
课件 p94：“森林由三部分构成：①森林中第一棵树的根结点；②森林中第一棵树的子树森林；③森林中其他树构成的森林。”这三部分恰好对应二叉树的**根、左子树、右子树**。

森林 → 二叉树（课件 p95 规则 1）

设森林 $F = \{T_1, T_2, \ldots, T_n\}$。①若 F 为空（$n = 0$），对应二叉树 B 为空；②否则 B 的根 = T_1 的根；B 的**左子树**由 T_1 的子树森林 $\{T_{11}, \ldots, T_{1m}\}$ 转化而来；B 的**右子树**由其余树 $\{T_2, \ldots, T_n\}$ 转化而来。
操作口诀（笔记 p598）：①将各棵树分别转换成二叉树；②将每棵树的根用线相连；③以第一棵树的根为二叉树的根，以根为轴心顺时针旋转。



森林（笔记 p598）：根之间加线（红）



结果（笔记 p599）：A 的左子树是 B→C→D（A 的孩子），右子树从 E 开始（第二、三棵树）。E 的右孩子 G 是第三棵树的根。

二叉树 → 森林（笔记 p599）

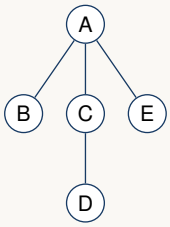
① **抹线**：把二叉树中根结点与其右孩子的连线，以及沿右分支搜索到的所有右孩子间的连线全部抹掉，得到若干棵孤立的二叉树；② **还原**：把每棵孤立的二叉树按“二叉树 → 树”的方法还原为树。

例如上面的二叉树：沿根的右链 A→E→G 抹线，得到三棵二叉树 A(B(,C(,D)))、E(F)、G(H(,I(J)))，分别还原为 A(B,C,D)、E(F)、G(H,I(J))（笔记 p599“还原”图）。

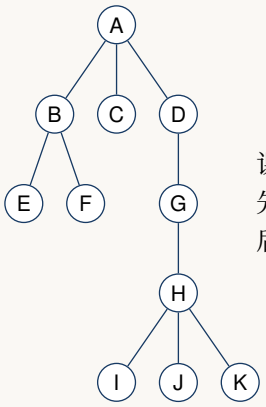
6.4　树和森林的遍历 [课件 6.5.3–6.5.4 • 笔记 p600–601]

遍历	规则
树的**先根**遍历	若树不空，先访问根结点，再依次先根遍历根的各棵子树
树的**后根**遍历	若树不空，先依次后根遍历根的各棵子树，然后访问根结点
树的**层次**遍历	若树不空，自上而下、自左而右访问树中每个结点
森林的**先序**遍历	访问第一棵树的根；先序遍历第一棵树的子树森林；先序遍历其余树组成的森林（= 依次对每棵树先根遍历）
森林的**中序**遍历	中序遍历第一棵树的子树森林；访问第一棵树的根；中序遍历其余树组成的森林（= 依次对每棵树后根遍历）

例题 6.2（笔记 p600 • 课件 p98）



笔记 p600 的树：
先根 **ABCDE**
后根 **BDCEA**
层次 **ABCED**



课件 p98 的树：
先根 **ABEFCDGHIJK**
后根 **EFBCIJKHGDA**

森林例（笔记 p600–601）：森林 A(B,C,D)、E(F)、G(H,I(J))：先序 **ABCDEFGHIJ**；中序（逐棵后根）**BCDA FE HJIG** = BCDAFEHJIG。

遍历的对应关系（课件 p100）

树	森林	对应的二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	**中序**遍历

证明：为什么“后根”对应二叉树的“中序”？（直观证明）

在对应的二叉树中，结点 x 的**左子树**是“x 的全部子孙”，**右子树**是“x 的弟弟们及其子孙”。树的后根遍历要求：先访问 x 的全部子孙，再访问 x，然后才轮到 x 的弟弟——这正是二叉树的“左子树、根、右子树”，即中序。同理，先根遍历“先 x、再子孙、再弟弟”= 二叉树的“根、左、右”= 先序。

验证：笔记 p596 的树 A(B(E,F,G),C,D(H,I))，后根 EFGBCHIDA；对应二叉树（§6.2）的中序也是 EFGBCHIDA ✓（已用程序验证）。

易错警示：森林的第二种遍历有两个名字

课件和笔记称为森林的“**中序**遍历”，有些教材（以及部分辅导书）称为“**后序**遍历”，指的是**同一种**遍历：逐棵树做后根遍历。考试按本课程叫法写“中序”，但看到“后序遍历森林”不要慌。另外，树**没有中序遍历**——一个结点可能有很多孩子，“根放在哪两个孩子中间”无法定义。

✍ **生活中的它：书的目录与文件夹大小**

生活场景：一本书的目录按“第 1 章、1.1、1.2、第 2 章……”排列；电脑统计一个文件夹的总大小。

对应关系：书的目录顺序 = 章节树的**先根遍历**（先写章名，再写它下面的各节）。统计文件夹大小 = **后根遍历**：必须先算出每个子文件夹的大小，才能加出父文件夹的大小。GNU 的 du 命令源码注释明确写着“一个目录只在其中所有条目都处理完之后才处理”。

小估算：文件夹课程 (数据结构 (实验 1: 2MB, 实验 2: 3MB), 讲义: 5MB)。后根顺序：实验 1、实验 2、数据结构 (=5MB)、讲义、课程 (=10MB)。

边界：目录和文件夹都是多叉树；真实文件系统还有链接、权限等复杂情况。

学完这里，你应该能够回答

(1) 三种存储法各自“找双亲/找孩子”哪个容易？ (2) 树转二叉树后根为什么没有右孩子？ (3) 森林的三部分对应二叉树的什么？ (4) 树的后根对应二叉树的什么遍历？ (5) 树为什么没有中序遍历？

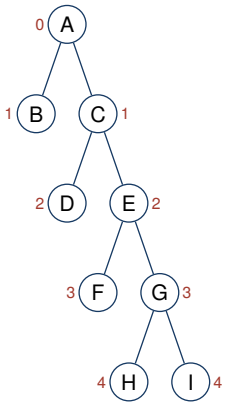
7　哈夫曼树与哈夫曼编码 [课件 6.6 • 笔记 p601–605]

到这里，我们已经解决了什么？树的结构、存储、遍历、转换都已掌握。**还剩什么问题？**笔记 p582 的“案例引入 1：数据压缩——将数据文件转换成由 0、1 组成的二进制串（编码）”。如果每个字符都用等长编码，出现频率高的字符和罕见字符占同样多的位，很浪费。能不能让**常用字符的编码短一些**？而且解码时不能产生歧义？答案是一棵特殊的二叉树——哈夫曼树（最优二叉树）。

7.1　路径长度与带权路径长度

定义：几个基本概念（笔记 p601 • 课件 p102–106）

- **路径**：从树中一个结点到另一个结点之间的分支构成这两个结点间的路径。
- **路径长度**：路径上的**分支数目**。
- **树的路径长度**：从树根到**每个结点**的路径长度之和（笔记口径）。
- **权**：赋予树中结点的一个有某种含义的数值（如字符出现的频率）。
- **结点的带权路径长度**：从根到该结点的路径长度与该结点的权的**乘积**。
- **树的带权路径长度**：树中所有**叶子结点**的带权路径长度之和，记作 $\mathrm{WPL}=\sum_{k=1}^{n} w_k\, l_k$，其中 w_k 是第 k 个叶子的权，l_k 是它到根的路径长度。

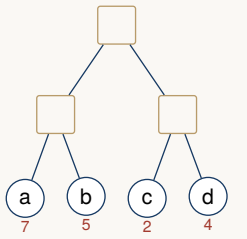


例（笔记 p601）：红字为各结点到根的路径长度。
A 到 B, C, D, E, F, G, H, I 的路径长度分别为 1, 1, 2, 2, 3, 3, 4, 4；
树的路径长度 TL $= 0+1+1+2+2+3+3+4+4 = 20$。
若 H 的权 $w = 2$，则 H 的带权路径长度为 $4 \times 2 = 8$。
笔记红字：结点数目相同的二叉树中，完全二叉树的路径长度最短（因为它把结点尽量排在离根近的层上）。

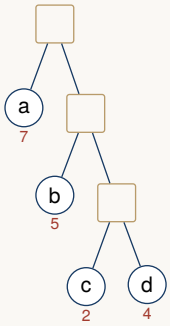
易错警示：WPL 只算叶子！

笔记 p601 把“叶子”二字用红框圈出：WPL 是**叶子**的带权路径长度之和，分支结点不计（分支结点在哈夫曼树里的“权”只是构造过程中的中间值，见课件 p107）。另外注意“树的路径长度”有两种口径：笔记计所有结点（例中 20），课件 p104 计所有叶子（$2+3+3+1 = 9$）。按题目给出的定义计算即可。

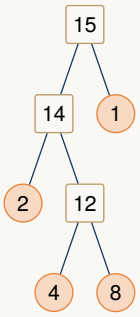
例题 7.1（笔记 p601–602 · 课件 p108–109）同样的叶子，不同的树，WPL 不同



WPL $= 2(7+5+2+4) = 36$

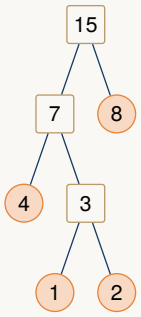


$7\times1+5\times2+2\times3+4\times3 = 35$



课件方法 1：

$2\cdot2+3\cdot4+3\cdot8+1\cdot1 = 41$



课件方法 2：

$2\cdot4+3\cdot1+3\cdot2+1\cdot8 = 25$

本题真正想说明什么：**权大的叶子离根近、权小的离根远**，WPL 就小。左边两棵树叶子相同：完全二叉树形态的 WPL 是 36，“不平衡”的反而是 35——WPL 最小的树不一定是完全二叉树。

7.2　哈夫曼树及其构造算法

定义：哈夫曼树（最优二叉树，课件 p108）

在所有含 n 个叶子结点、并带相同权值的二叉树中，必存在一棵其带权路径长度取最小值的树，称为“最优树”或“哈夫曼树”（Huffman Tree）。

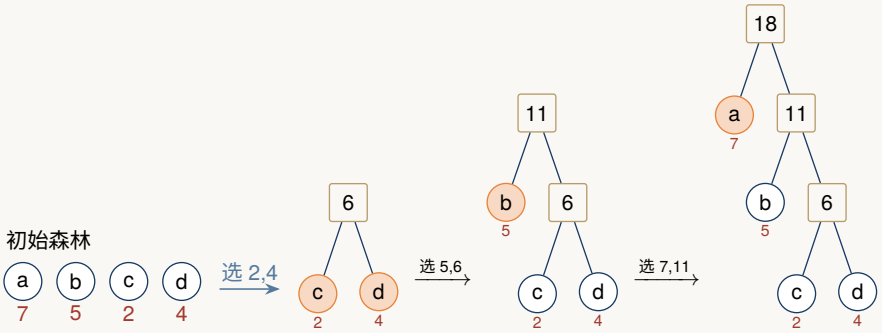
笔记 p602 用红笔写了四条注意：

1. WPL 要在**叶子个数相同、叶子权值相同**的二叉树之间比较（笔记原写“度相同”，不准确，见纠错汇总）；
2. **满二叉树不一定是哈夫曼树**（例 7.1 第一棵就不是）；
3. 哈夫曼树中**权越大的叶子离根越近**（更准确：不会比权小的叶子离根更远）；
4. 具有相同带权结点的哈夫曼树**不唯一**（但最小 WPL 唯一）。

哈夫曼算法（贪心：每次合并权值最小的两棵树）

(1) 根据给定的 n 个权值 $\{w_1,\dots,w_n\}$ 构造 n 棵二叉树的森林 $F=\{T_1,\dots,T_n\}$，其中 T_i 只有一个带权为 w_i 的根结点；
(2) 在 F 中选取两棵**根结点权值最小**的树作为左、右子树，构造一棵新的二叉树，新根的权值为其左、右子树根结点权值之和；
(3) 在 F 中删除这两棵树，同时将新得到的二叉树加入 F；
(4) 重复 (2)(3)，直到 F 中只剩一棵树为止。

例题 7.2（笔记 p602–603）权值 a:7, b:5, c:2, d:4



过程：森林 {7,5,2,4} → 选最小的 2、4 合并为 6，森林 {7,5,6} → 选 5、6 合并为 11，森林 {7,11} → 合并为 18，结束。
结果：$\mathrm{WPL}=7\times1+5\times2+2\times3+4\times3=7+10+6+12=\mathbf{35}$（笔记 p603）。
快速验算：WPL = 所有新建结点权值之和 $=6+11+18=35$ ✓（下面证明这条恒等式）。

哈夫曼树的性质（笔记 p603）

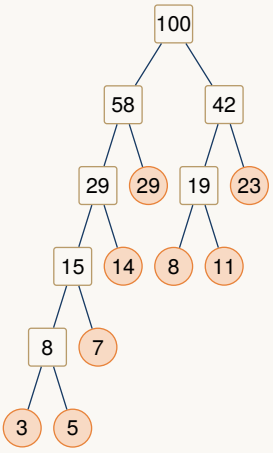
(1) 哈夫曼树中结点的度只有 2 和 0，**没有度为 1 的结点**；(2) 包含 n 个叶子结点的哈夫曼树中共有 $\boxed{2n-1}$ 个结点；(3) 度为 0 的结点 n 个，度为 2 的结点 $n-1$ 个。

证明

(1) 每个非叶结点都是“两棵树合并”时新建的，必有 2 个孩子。(3) 每次合并使森林中的树少一棵，从 n 棵到 1 棵共合并 $n-1$ 次，新建 $n-1$ 个度为 2 的结点（也可由 $n_0=n_2+1$ 直接得到）。
(2) 总数 $n+(n-1)=2n-1$。
WPL = 所有非叶结点权值之和：每个叶子 k 的权 w_k 被计入它的每一个祖先结点的权值里（祖先权 = 子树内叶子权之和），而叶子 k 恰好有 l_k 个祖先。于是 $\sum_{\text{非叶}v} w(v)=\sum_k w_k l_k=\mathrm{WPL}$。■

教学补全：为什么“每次合并最小的两个”就是最优？（思路） **关键事实 1**：最优树中，权最小的两个叶子可以放在最深层且互为兄弟——若有更大权的叶子比它们更深，交换位置只会让 WPL 变小或不变。**关键事实 2**：把这两个兄弟叶子合并成一个权为 w_1+w_2 的叶子后，新问题的最优 WPL 恰好比原问题少 w_1+w_2。于是“先合并最小两个，再递归求解规模为 $n-1$ 的问题”一定得到最优解。严格证明见《算法导论》贪心算法一章，本课程只要求理解思路。

例题 7.3（课件 p111–116）权值 5, 29, 7, 8, 14, 23, 3, 11



步	选出的两个最小权	新结点	合并后的森林（根权值）
1	3, 5	8	29, 7, 8, 14, 23, 11, **8**
2	7, 8	15	29, 14, 23, 11, 8, **15**
3	8, 11	19	29, 14, 23, 15, **19**
4	14, 15	29	29, 23, 19, **29**
5	19, 23	42	29, 29, **42**
6	29, 29	58	42, **58**
7	42, 58	100	**100**

WPL：叶子 3、5 深 5，7 深 4，14、8、11 深 3，29、23 深 2：

$$\mathrm{WPL} = 5{\cdot}3+5{\cdot}5+4{\cdot}7+3{\cdot}14+2{\cdot}29+3{\cdot}8+3{\cdot}11+2{\cdot}23 = 15+25+28+42+58+24+33+46 = \mathbf{271}.$$

验算：$8+15+19+29+42+58+100 = 271$ ✓。

原图纠错：课件 p116

原始写法：“节点 11 的带权路径长度……即 $3*11=15$”，“树的带权路径长度 $=15+25+28+42+58+24+15+46=253$”。**问题**：$3\times11=33$，不是 15；总和应为 **271**。**为什么这样改**：用两种方法（逐叶子相乘、非叶结点求和）都得 271。**影响**：若照抄 253，此类填空题会直接失分。

易错警示：构造时的三个常见错误

①只看原始权值：每次都要在当前森林的根中选最小——新合并出的结点也参与比较（例 7.3 第 3 步选的 8 就是第 1 步生成的）。
②把 WPL 算成所有结点：只乘叶子。
③权值相等时纠结选哪个：任选都对，树形可能不同，但 WPL 一定相同（这就是“哈夫曼树不唯一”）。笔记 p604 第二例 {7,5,5,2,4} 中有两个 5，结果 $\mathrm{WPL}=6+10+13+23=52$。

7.3　前缀编码与哈夫曼编码 [课件 6.6.3 • 笔记 p604–605]

定义：前缀编码

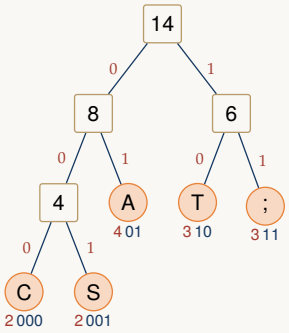
在一个编码方案中，如果**任一字符的编码都不是另一个字符编码的前缀**，称为前缀编码。

为什么需要“前缀”性质？ 设 A=0、B=01、C=1。收到 `01` 时，它是“B”还是“A C”？无法确定。原因是 A 的编码 0 是 B 的编码 01 的前缀。**前缀编码保证从左到右读位串时，一旦读到某个完整编码就可以立刻确定字符**，不必向后看，也不需要分隔符。
树上的解释：把编码画成二叉树（0 向左、1 向右），每个字符放在**叶子**上。从根走到某个叶子就结束，不可能“路过”另一个字符——所以叶子上的编码天然是前缀编码。

哈夫曼编码

以字符出现的频率为权构造哈夫曼树，约定**左分支标 0、右分支标 1**，从根到每个叶子路径上的 0/1 串就是该字符的编码。**性质**（笔记 p605）：①哈夫曼编码是前缀码；②哈夫曼编码是**最优前缀码**——在所有前缀码中它使电文总长最短（因为电文总长 $=\sum w_i l_i=$ WPL）。

例题 7.4（笔记 p604–605）字符集 {C, A, S, T, ;}，频率 {2, 4, 2, 3, 3}



构造：2+2=4（C、S）；3+3=6（T、;）；4+4=8（{C,S} 与 A）；8+6=14。
编码：T:10，;:11，A:01，S:001，C:000（笔记 p605）。
检查前缀性：没有一个码是另一个的开头 ✓。
电文总长：$2\cdot3+4\cdot2+2\cdot3+3\cdot2+3\cdot2=32$ 位 $=$ WPL $=4+6+8+14$ ✓。若用等长编码，5 个字符需 3 位，共 $14\times3=42$ 位。
译码：`0100110` → 01|001|10 → **A S T**。

例题 7.5（课件 p119）报文 CAST CAST SAT AT A TASA

字符集 {C, A, S, T}，出现次数 $W=\{2,7,4,5\}$。**等长编码**（每字符 2 位，A:00 T:10 C:01 S:11）：总长 $2\times(2+7+4+5)=36$。**哈夫曼编码**：2+4=6，5+6=11，7+11=18；得 A:0，T:10，C:110，S:111；总长 $7\cdot1+5\cdot2+2\cdot3+4\cdot3=$ **35**，恰好等于哈夫曼树的 WPL。**本题想说明**：频率最高的 A 只用 1 位，这正是“压缩”的来源。

C++：拓展：哈夫曼树的最简数组实现（课件“实现不讲”，此为理解用，附录 A 已运行）

```cpp
struct HuffNode { int weight, parent, left, right; };   // 下标表示指针，-1 表示无
int BuildHuffman(const int w[], int n, HuffNode ht[]) { // ht 长度 2n-1，返回 WPL
    for (int i = 0; i < 2 * n - 1; i++) {
        ht[i].weight = (i < n) ? w[i] : 0;
        ht[i].parent = ht[i].left = ht[i].right = -1;
    }
    int wpl = 0;
    for (int k = n; k < 2 * n - 1; k++) {               // n-1 次合并
        int s1 = -1, s2 = -1;                           // 找两个无双亲的最小权根
        for (int i = 0; i < k; i++) {
            if (ht[i].parent != -1) continue;
            if (s1 == -1 || ht[i].weight < ht[s1].weight) { s2 = s1; s1 = i; }
            else if (s2 == -1 || ht[i].weight < ht[s2].weight) s2 = i;
        }
        ht[k].weight = ht[s1].weight + ht[s2].weight;   // 新根 = 两者之和
        ht[k].left = s1; ht[k].right = s2;
        ht[s1].parent = ht[s2].parent = k;
        wpl += ht[k].weight;                            // 非叶结点权之和 = WPL
    }
    return wpl;
}
```

代码解释：数组前 n 个单元放叶子，后 $n-1$ 个放新建结点，正好 $2n-1$ 个（性质 2）。`parent == -1` 表示“还是森林中某棵树的根”。求编码时从叶子沿 `parent` 走到根，看自己是左孩子（记 0）还是右孩子（记 1），最后把串反转。对课件主观题 $W=\{5,6,2,9,7\}$，程序输出 WPL $=65$。

➢ 对照：哈夫曼编码 ↔ 摩尔斯电码（方法互补型）

一句话定位：两者都是“常用字符用短码”的变长编码，但摩尔斯电码**不是**前缀码。

维度	哈夫曼编码	摩尔斯电码
码长分配	由频率算出，可证最优	人工设计，常用字母短（E 为“·”）
前缀性	是前缀码	否：E=·是 I=··、S=···的前缀
如何断开	不需要分隔符	必须靠停顿：字母间停 3 个点长，单词间 7 个

迁移判断：✓“高频短码”思想相同；✗摩尔斯的“···”若没有停顿可以读成 S、EI、IE、EEE，无法唯一译码——这正说明了前缀性的价值。

➢ 对照：WPL ↔ 概率论中的数学期望（类比同构型）

一句话定位：若权是字符出现的**概率** p_i（$\sum p_i=1$），则 WPL $=\sum p_i l_i$ 就是“每个字符平均用多少位”，即码长的数学期望 $E[L]$。

小算例：概率 $\frac{1}{2},\frac{1}{4},\frac{1}{8},\frac{1}{8}$ 的哈夫曼码长为 1, 2, 3, 3，$E[L]=\frac{1}{2}+\frac{2}{4}+\frac{3}{8}+\frac{3}{8}=1.75$ 位；等长编码需 2 位。

迁移判断：✓期望的线性性质可用；△权是“次数”时 WPL 是总长，是“概率”时是平均长，两者差一个总次数因子。**拓展联想**：信息论中的“熵”给出了平均码长的理论下界，后续“信息论/通信原理”课程会学。

✍ 生活中的它：压缩包、PNG 图片和 JPEG 照片

生活场景：把作业文件夹打成 ZIP 压缩包，或保存一张 PNG 截图。

对应关系：ZIP/gzip/PNG 使用的 DEFLATE 格式（RFC 1951）先用别的方法找出重复片段，**最后一步用哈夫曼编码**把出现频繁的符号编成短码；基线 JPEG（ITU-T T.81）的熵编码也是哈夫曼编码。RFC 1951 原文就说明：前缀码“总能被无歧义地逐个符号解析”。

小判断：为什么压缩包里的文本文件通常比照片压缩得多？文本中字符频率很不均匀（字母 e、空格很常见），哈夫曼编码收益大；已经压缩过的照片频率接近均匀，收益很小。

边界：哈夫曼编码只是这些格式中的一个环节；而且这些格式只在文件里记录各符号的码长（“范式哈夫曼码”），不直接存整棵树。

学完这里，你应该能够回答

(1) WPL 算哪些结点？　(2) 写出哈夫曼算法四步。　(3) n 个叶子的哈夫曼树有几个结点？　(4) 为什么哈夫曼编码一定是前缀码？　(5) 哈夫曼树唯一吗？WPL 呢？

8　全章知识地图与总结

8.1　本章知识地图

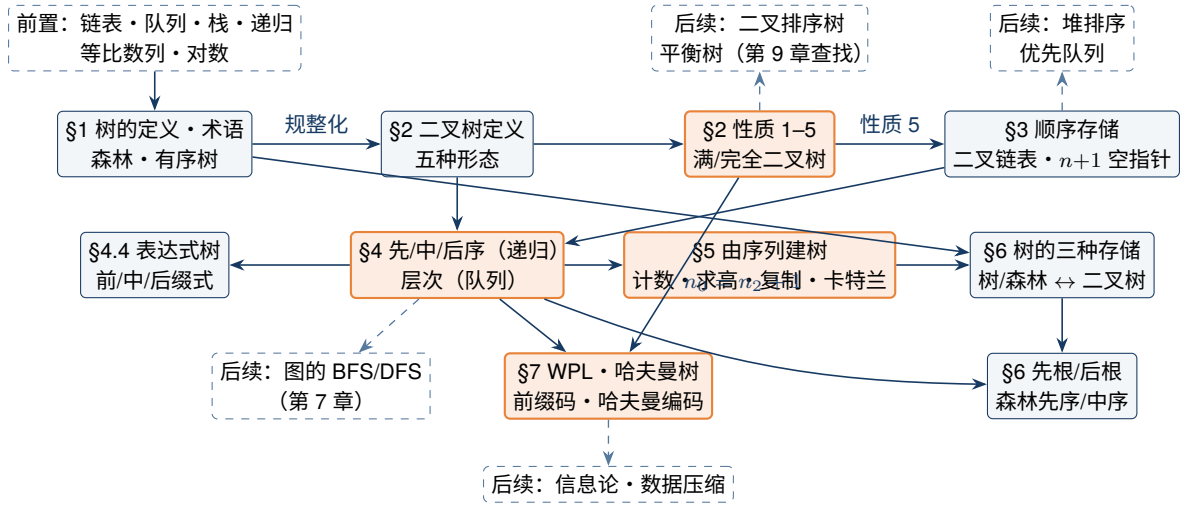


图 13: 实线：本章内部的依赖关系；虚线：与后续课程的联系。橙色框为考试重点。

8.2　高频易错点总表

易错点	错误想法	正确做法
叶子计数	只看最底层	逐个检查“有没有孩子”
性质 1	第 i 层“至少” 2^{i-1}	**至多** 2^{i-1}
二叉树 vs 树	二叉树是度为 2 的树	二叉树左右有别、可为空，是另一个概念
完全二叉树	每层都满	编号 $1\sim n$ 无空位；满 $\Rightarrow$ 完全，反之不然
编号起点	混用 $2i$ 与 $2i+1$	从 1 编号：$2i, 2i{+}1, \lfloor i/2\rfloor$；从 0：$2i{+}1, 2i{+}2, \lfloor (i{-}1)/2\rfloor$
只有一个孩子	中序时随手放左边	先看清是左孩子还是右孩子
序列建树	先序 + 后序也行	必须有中序
中缀序列	等于原表达式	丢失括号，需补括号
树的遍历	树也有中序	树只有先根、后根（及层次）
后根对应	后根 ↔ 二叉树后序	后根 ↔ 二叉树**中序**
WPL	所有结点都乘	只算叶子；验算：WPL = 非叶结点权之和
哈夫曼构造	只在原始权中挑	每次在**当前森林**的根中挑最小两个

8.3　基础解题步骤总结

六类必考题的“标准动作”

1. **写遍历序列**：把树写成“根 [左] [右]”，逐层展开；中序可用“压扁投影”验证。
2. **由序列建树**：先序首/后序尾定根 → 中序切左右 → 数左子树长度截取另一序列 → 递归；最后反求一遍序列验算。
3. **性质计算**：$n = n_0 + n_1 + n_2$ 与 $n_0 = n_2 + 1$ 联立；完全二叉树再加“n 奇则 $n_1 = 0$，n 偶则 $n_1 = 1$”。
4. **树/森林 ↔ 二叉树**：左孩子右兄弟；森林先把各树根连成兄弟。
5. **哈夫曼**：反复“取两最小、合并、放回”；WPL 两种算法互相验算；编码左 0 右 1。
6. **写递归函数**：空树出口 + 递归左右 + 在根处合并（后序思想）。

8.4　相近概念总对比

维度	先序 DLR	中序 LDR	后序 LRD
根的位置	序列第一个	左右子树序列之间	序列最后一个
递归中访问时机	第 1 次经过	第 2 次经过	第 3 次经过
表达式树	前缀式（波兰式）	中缀式（丢括号）	后缀式（逆波兰式）
典型用途	复制、带'#' 先序建树、目录列表	二叉排序树有序输出（后续）	求值、求高、销毁、统计文件夹大小
对应树/森林	树先根 / 森林先序	树后根 / 森林中序	——

8.5　跨学科 / 相似知识总对比表

对照	类型	最重要的迁移结论
线性表 ↔ 树	推广	✗访问次序不再唯一，需约定遍历
树的度 ↔ 图的度	同名异义	✗图的度要算上连双亲的边
二叉树 ↔ 度为 2 的树	同名近义	✗形态数不同（3 结点：5 种 vs 1 种）
满/完全 ↔ full/perfect/-complete	同名异义	✗full ≠ 满；中文深度 = 英文 height + 1
顺序存储 ↔ 二叉堆	思想重现	△下标起点不同公式要换算
二叉链表 ↔ 单/双向链表	推广	✗一个循环走不遍，需递归或栈/队列
递归遍历 ↔ 数学归纳法	思想重现	✓只需出口与一层步骤正确；✗递归有栈开销
层次遍历 ↔ 图的 BFS	推广	✗图需 visited 标记
表达式树后序 ↔ 栈求后缀式	思想重现	✓计算顺序相同
二叉树计数 ↔ 出栈序列数	思想重现	△同一递推式，均为卡特兰数
孩子链表 ↔ 邻接表	推广	✗图中顶点可出现在多条链
哈夫曼编码 ↔ 摩尔斯电码	方法互补	✗摩尔斯非前缀码，需停顿分隔
WPL ↔ 数学期望	类比同构	△权为概率时 WPL 为平均码长

9　正式练习（38 道）

知识点—题目对应表

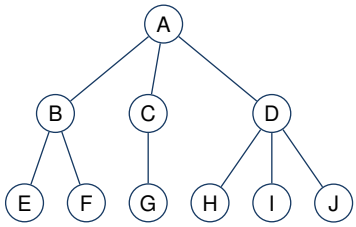
核心知识点	基础题（1–25）	提升题（26–35）	跨学科（36–38）
K1 树的定义与术语	1，11	—	—
K2 二叉树定义与形态	2，11	—	—
K3 性质 1–3	3，12，16，17	26，31，34	37
K4 满/完全二叉树、性质 4–5	4，5，12	26，29	—
K5 顺序存储	6	29	—
K6 链式存储、空指针	7	34	—
K7 先/中/后序遍历	8，13	28，34	—
K8 层次遍历（队列）	9	34	—
K9 表达式树	18	33	—
K10 由序列确定二叉树	15，21，22	28，34	—
K11 递归应用（计数/求高/求值）	25	33	38
K12 卡特兰计数	19	—	—
K13 树的存储表示	10	35	—
K14 树/森林 ↔ 二叉树	10，23	27，35	—
K15 树与森林的遍历	—	27，35	38
K16 WPL 与哈夫曼树	14，20，24	30，31，32	36
K17 前缀码与哈夫曼编码	14，24	30	36

说明：课件中的全部课堂单选题与主观题（p12、p25、p85 两题、p92、p93、p117、作业 6、作业 7）都已作为正文例题讲解或改编进下列练习。题号连续编排：1–25 基础，26–35 提升，36–38 跨学科。

9.1　A. 基础巩固训练（25 道）

基础单项选择题（10 道）

1. 基础　如下图所示的树，树的度和深度分别是（　）。

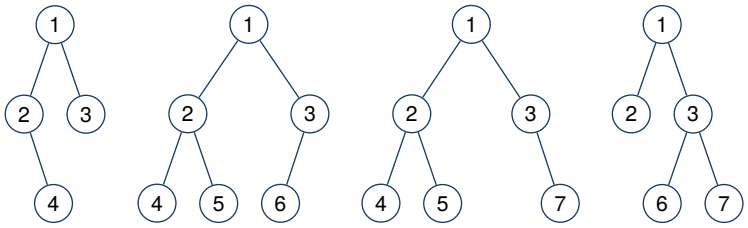


A. 3，3　　B. 3，4　　C. 10，3　　D. 2，3

2. 基础　下列关于二叉树的叙述中，正确的是（　）。
A. 二叉树就是度为 2 的有序树　　B. 二叉树中每个结点的度都是 2
C. 二叉树的左、右子树可以任意交换而不改变这棵树　　D. 二叉树可以是空树

3. 基础　一棵二叉树中有 10 个度为 2 的结点、5 个度为 1 的结点，则叶子结点个数为（　）。
A. 9　　B. 10　　C. 11　　D. 15

4. 基础　下列四棵二叉树中，**是**完全二叉树的是（　）。



A　　B　　C　　D

5. 基础 具有 100 个结点的完全二叉树的深度为（　）。

A. 6　　B. 7　　C. 8　　D. 50

6. 基础 完全二叉树按层序从 1 开始编号并顺序存储（如图），下标为 6 的结点的双亲下标和左孩子下标分别是（　）。

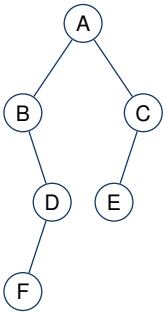


A. 3，12　　B. 2，12　　C. 3，13　　D. 3，11

7. 基础 含 20 个结点的二叉树采用二叉链表存储，空指针域的个数为（　）。

A. 19　　B. 20　　C. 21　　D. 40

8. 基础 如下图所示的二叉树，其**中序**遍历序列为（　）。



A. B F D A E C　　B. B D F A E C　　C. F D B E C A　　D. A B D F C E

9. 基础 二叉树的层次遍历算法中，用来暂存“已发现但尚未访问”的结点的数据结构通常是（　）。

A. 栈　　B. 队列　　C. 线性表的顺序存储　　D. 散列表

10. 基础 将森林 $F=\{T_1,T_2,\ldots,T_n\}$ 转换为二叉树 B 后，B 的根的**右子树**由（　）转换而来。

A. T_1 的子树森林　　B. $T_2,\ldots,T_n$ 构成的森林　　C. T_1 的根　　D. T_n

基础多项选择题（5 道）

11. 基础 下列说法中正确的有（　）。

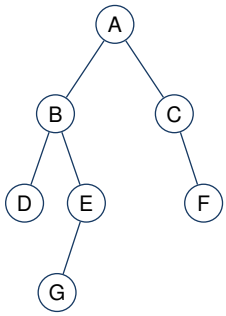
A. 树中的叶子结点不一定都在最底层　　B. 3 个结点的二叉树共有 5 种不同形态

C. 树的度等于树中结点数减 1　　D. 二叉树中每个结点的度不超过 2

12. 基础 一棵完全二叉树有 7 个结点，下列说法正确的有（　）。

A. 它一定是满二叉树　　B. 它的深度为 3　　C. 它有 4 个叶子　　D. 它有 1 个度为 1 的结点

13. 基础 如下图所示的二叉树，下列遍历序列正确的有（　）。



A. 先序 ABDEGCF　　B. 中序 DBEGACF　　C. 后序 DGEBFCA　　D. 层次 ABCDEGF

14. 基础 关于哈夫曼树和哈夫曼编码，下列说法正确的有（　）。
A. 哈夫曼树中没有度为 1 的结点　　B. 含 n 个叶子的哈夫曼树共有 $2n-1$ 个结点
C. 给定一组权值，哈夫曼树是唯一的　　D. 哈夫曼编码是前缀编码

15. 基础 设二叉树各结点的值互不相同，下列哪些遍历序列组合可以唯一确定这棵二叉树（　）。
A. 先序 + 中序　　B. 后序 + 中序　　C. 先序 + 后序　　D. 层次 + 中序

基础填空题（5 道）

16. 基础 一棵二叉树有 15 个叶子、8 个度为 1 的结点，则它共有_____ 个结点。

17. 基础 深度为 5 的二叉树最多有_____ 个结点。

18. 基础 表达式 $(a+b)\times c-d/e$ 对应的表达式树的后序遍历序列（后缀表达式）为_____。

19. 基础 由 4 个结点可以构成_____ 种不同形态的二叉树。

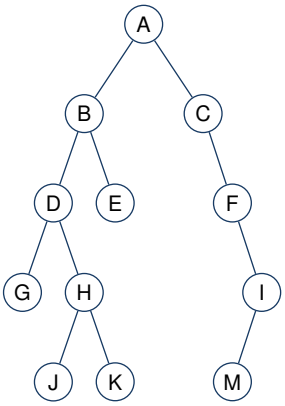
20. 基础 以 {2, 4, 8, 1} 为叶子权值构造哈夫曼树，其带权路径长度 WPL =_____。

基础解答题（5 道）

21. 基础 （课件主观题）设一棵二叉树的先序序列为 A B D F C E G H，中序序列为 B F D A G E H C。画出这棵二叉树，并写出它的后序序列。

22. 基础 （课件主观题）已知一棵二叉树的中序和后序序列如下，画出这棵二叉树，并写出它的先序序列。
中序：c, b, d, e, a, g, i, h, j, f　　后序：c, e, d, b, i, j, h, g, f, a

23. 基础 （课件作业 6）画出下图二叉树对应的森林。



24. 基础 （课件主观题）已知权值 $W=\{5,6,2,9,7\}$，试构造哈夫曼树，求其带权路径长度，并按“左 0 右 1”写出各权值对应的哈夫曼编码。

25. 基础 二叉树采用二叉链表存储（结点类型为课程的 `BinTreeNode<ElemType>`）。用 C++ 编写递归函数 `int DegreelCount(const BinTreeNode<ElemType> *r)`，求二叉树中**度为 1** 的结点个数；

并说明它对教材图 6.9 的二叉树 A(B(D(,G)),C(E,F)) 返回多少。

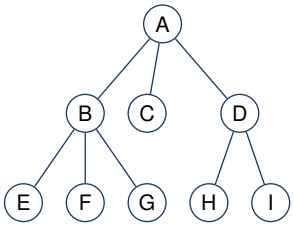
9.2　B. 中等偏下提升训练（10 道）

提升单项选择题（3 道）

26. 提升 一棵完全二叉树有 700 个结点，则其叶子结点个数为（　）。

A. 349　　B. 350　　C. 351　　D. 699

27. 提升 将下图的树转换为二叉树，该二叉树的**中序**遍历序列是（　）。



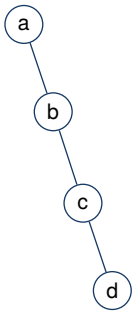
A. ABEFGCDHI　　B. EFGBCHIDA　　C. GFEIHDCBA　　D. ABCDEFGHI

28. 提升 已知二叉树的先序序列为 ABCDEFG，中序序列为 CBDAEGF，则其后序序列为（　）。

A. CDBGFEA　　B. CDBFGEA　　C. DCBGFEA　　D. CBDGFEA

提升多项选择题（2 道）

29. 提升 一棵深度为 4、只有 4 个结点的**右单支**二叉树（每个结点只有右孩子）采用顺序存储，下标从 1 开始。下列说法正确的有（　）。



A. 数组长度至少为 15　　B. 实际只用了 4 个单元

C. 结点 d 存放在下标 8　　D. 4 个结点的下标依次为 1, 3, 7, 15

30. 提升 某段电文中只含字符 a, b, c, d，出现次数分别为 4, 2, 1, 1。用哈夫曼编码（左 0 右 1），下列说法正确的有（　）。

A. a 的编码长度为 1　　B. 整段电文的编码总长为 14 位

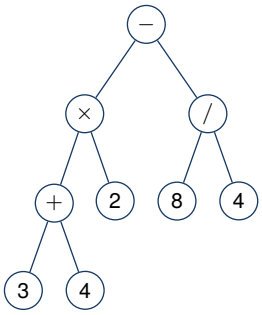
C. c 与 d 的编码长度相同　　D. 若改用等长编码，电文总长会更短

提升填空题（3 道）

31. 提升 一棵哈夫曼树共有 199 个结点，则它用于编码的字符（叶子）有_____ 个。

32. 提升 （课件例题）以 5, 29, 7, 8, 14, 23, 3, 11 为权值构造哈夫曼树，其 WPL =_____。

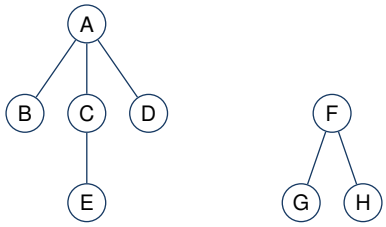
33. 提升 下图表达式树中叶子均为一位数字。用“后序遍历、访问根时做运算”的方法求值，结果为_____；该树的高度为_____。



提升综合大题（2 道）

34. 提升 已知二叉树的中序序列为 D B G E A C H F，后序序列为 D G E B H F C A。
(1) 画出这棵二叉树；(2) 写出它的先序序列和层次序列；(3) 求 n_0, n_1, n_2，并验证性质 3；(4) 若采用二叉链表存储，有多少个空指针域？

35. 提升 给定由两棵树组成的森林：$T_1 = \mathrm{A(B, C(E), D)}$，$T_2 = \mathrm{F(G, H)}$。
(1) 画出该森林的孩子兄弟表示（即对应的二叉树）；(2) 写出森林的先序序列和中序序列，并与二叉树的先序、中序序列比较；(3) 对应二叉树中根 A 的右指针为空吗？为什么？



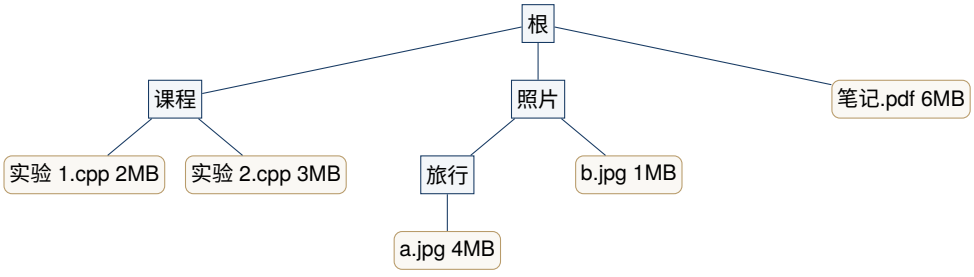
第 35 题的森林

9.3　C. 跨学科迁移训练（3 道）

36. 跨学科 （通信 × 概率）某传感器只输出 4 种状态 s_1, s_2, s_3, s_4，出现概率分别为 0.4, 0.3, 0.2, 0.1。为节省传输位数，对状态进行哈夫曼编码。**背景**：平均码长 $\bar{L} = \sum p_i l_i$（l_i 为 s_i 的码长），表示“每发送一个状态平均需要多少位”。则 $\bar{L} =$（　）。
A. 1.9 位　　B. 2 位　　C. 2.4 位　　D. 1.75 位

37. 跨学科 （体育赛事组织）13 支球队进行单败淘汰赛：每场两队比赛，负者淘汰、胜者晋级，人数为奇数的轮次有一队轮空直接晋级，直到决出冠军。把对阵表画成二叉树：球队为叶子，每场比赛为一个分支结点（两个孩子是该场比赛的双方来源），冠军赛为根。下列说法正确的有（　）。
A. 一共需要比赛 12 场　　B. 对阵树中没有度为 1 的结点
C. 若改为 16 支队，冠军恰好要赢 4 场　　D. 13 支队的对阵树一定是完全二叉树

38. 跨学科 （操作系统 × 文件管理）下图为一个文件夹结构（方框为文件夹，圆角框为文件及其大小）。系统统计文件夹大小时，必须先算出所有子项的大小，才能算出父文件夹的大小。按此规则，第一个被算出大小的**文件夹**是_____，根文件夹的总大小为_____ MB。



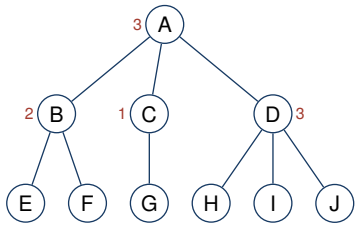
10　答案速查

基础单选	1. A　2. D　3. C　4. B　5. B　6. A　7. C　8. A　9. B　10. B
基础多选	11. ABD　12. ABC　13. AC　14. ABD　15. ABD
基础填空	16. 37　17. 31　18. $ab+c\times de/-$　19. 14　20. 25
基础解答	21. 后序 F D B G H E C A　22. 先序 a b c d e f g h i j　23. 森林 A(B(D(G),H(J,K)),E)、C、F、I(M) 24. WPL $=65$；6:00，7:01，2:100，5:101，9:11（树形可不唯一，WPL 唯一）　25. 见解析，返回 2
提升单选	26. B　27. B　28. A
提升多选	29. ABD　30. ABC
提升填空	31. 100　32. 271　33. 12；4
提升大题	34. 树 A(B(D,E(G,)),C(,F(H,))))；先序 ABDEGCFH，层次 ABCDEFGH；$n_0=3, n_1=3, n_2=2$；空指针 9 个　35. 先序 ABCEDFGH，中序 BECDAGHF；A 的右指针非空（指向 F）
跨学科	36. A　37. ABC　38. 课程；16

11　详细解析

11.1　基础单选题解析

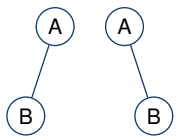
第 1 题　答案 A　基础 [K1 术语]



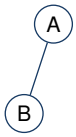
红字为结点的度

依据：树的度 = 各结点度的最大值；深度 = 最大层次。
判断：A 与 D 都有 3 个孩子，最大度为 3；根在第 1 层，B、C、D 第 2 层，E–J 第 3 层，深度 3。选 A。
B 错：把“层数”多数了一层（或按边数从 0 起算后又加了 1）。**C 错**：10 是结点数，不是度。**D 错**：误以为“度”一定是 2——那是二叉树的限制。
一句话：度看“最多几个孩子”，深度看“最多几层”。

第 2 题　答案 D　基础 [K2 二叉树定义]



C 的反例：交换后是不同的二叉树



A、B 的反例：二叉树但 A 的度为 1、B 的度为 0

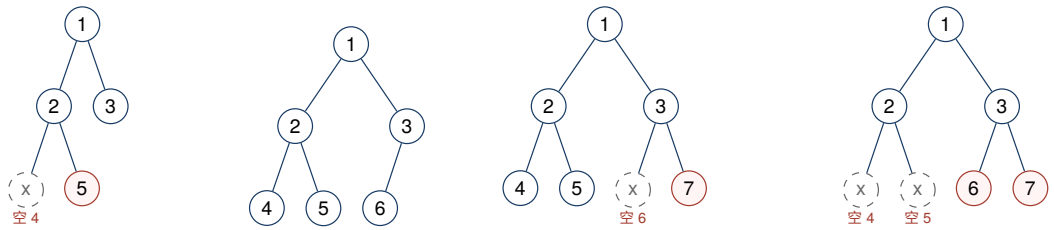
依据：二叉树定义允许 $n=0$（空集），D 正确。
A 错：二叉树和度为 2 的有序树是两个概念——二叉树可以为空，且只有一个孩子时也必须分左右（笔记 p581）。
B 错：度“不超过”2，可以是 0 或 1（见左下图）。
C 错：左右子树次序不能颠倒（左上两图是两棵不同的二叉树）。
常见错误：把“最多两个”读成“恰好两个”。

第 3 题　答案 C　基础 [K3 性质 3]

由性质 3，$n_0=n_2+1=10+1=11$，与 n_1 无关。选 C。**A 错**：n_2-1 符号记反；**B 错**：误以为 $n_0=n_2$；**D 错**：误把 n_1+n_2 当成叶子数。**检查**：$n=11+5+10=26$，边数 $n-1=25=n_1+2n_2=5+20$ ✓。

第 4 题　答案 B　基础 [K4 完全二叉树]

对每棵树按满二叉树编号，看编号是否连续（红色为“空位”）：



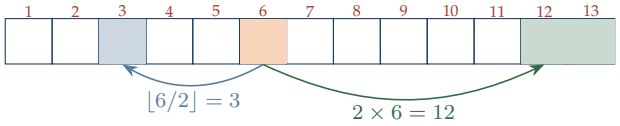
A ✗：结点占 5 号，4 号空　B ✓：1–6 连续　C ✗：结点占 7 号，6 号空　D ✗：6、7 有结点，4、5 空

注：题图 A 中结点“4”画在 2 的右孩子位置，按满二叉树编号它实际是 5 号。**图上证据**：只有 B 的结点编号从 1 到 6 无空位。**一句话**：完全二叉树 = 从满二叉树末尾**连续**去掉结点。

第 5 题　答案 B　基础 [K4 性质 4]

$2^6 = 64 \le 100 < 128 = 2^7$，$\lfloor \log_2 100 \rfloor = 6$，深度 $= 6 + 1 = 7$。**A 错**：忘了“+1”；**C 错**：误用 $\lceil \log_2 100 \rceil + 1$；**D 错**：把完全二叉树当成单支树。**验证**：深度 6 至多 63 个结点 < 100，深度 7 至多 127 个 ≥ 100 ✓。

第 6 题　答案 A　基础 [K5 顺序存储 + 性质 5]

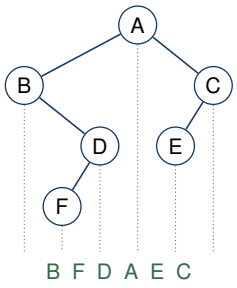


双亲 $\lfloor 6/2 \rfloor = 3$，左孩子 $2 \times 6 = 12 \le 13$（存在），选 A。**B 错**：双亲误算为 $6/3$；**C 错**：13 是右孩子；**D 错**：$2i - 1$ 是乱凑。**易错**：若从 0 编号，答案会变成双亲 2、左孩子 13。

第 7 题　答案 C　基础 [K6 空指针]

$2n$ 个指针域中，非空的恰为边数 $n - 1 = 19$，空的 $2 \times 20 - 19 = 21 = n + 1$。**A 错**：19 是非空指针数；**B 错**：误以为每个结点一个空指针；**D 错**：40 是指针域总数。

第 8 题　答案 A　基础 [K7 中序遍历]



“压扁投影”得中序

展开：[B 子树] A [C 子树] → [B [D 子树]] A [E C] → B F D A E C（D 的左孩子 F 先于 D）。选 A。

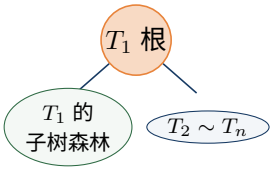
B 错：把 F 当成 D 的右孩子（D F）。**C 错**：这是后序 FDBECA。**D 错**：这是先序 ABDFCE。

一句话：选项 C、D 是同一棵树的另两种遍历，专门检验能否区分三种次序。

第 9 题　答案 B　基础 [K8 层次遍历]

层次遍历要求“先发现的先访问”，即先进先出，用队列（见 §4.3 队列变化图）。**A 错**：栈是后进先出，用于非递归的先/中/后序（附录 B）；**C、D 错**：顺序表、散列表都不能自然体现“先进先出”的访问次序。

第 10 题　答案 B　基础 [K14 森林 ↔ 二叉树]



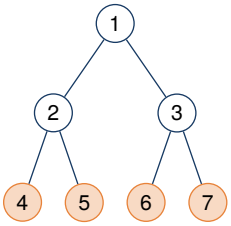
森林三部分 ↔ 二叉树的根、左子树、右子树（课件 p94–95）。右子树由其余树 $T_2, \ldots, T_n$ 转换而来，选 B。**A 错**：那是左子树；**C 错**：T_1 的根就是二叉树的根本身；**D 错**：T_n 只是右子树的一部分（沿右链最末端）。

11.2　基础多选题解析

第 11 题　答案 ABD　基础 [K1, K2]

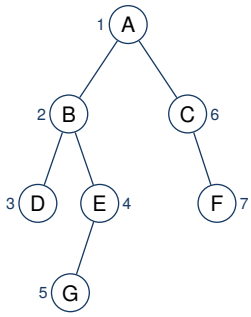
A 正确：例如笔记树 T_0 中 F、G 在第 3 层就是叶子，最底层是第 4 层。
B 正确：见例题 2.1 的五种形态图，$b_3 = 5$。
C 错误：结点数减 1 是边数（$B = n - 1$），不是树的度。反例：T_0 有 13 个结点，度为 3。
D 正确：二叉树特点 1。

第 12 题　答案 ABC　基础 [K3, K4]



编号 1–7 无空位，而深度为 3 的满二叉树恰好 $2^3 - 1 = 7$ 个结点，所以它就是满二叉树。
A 正确；**B 正确**：$\lfloor \log_2 7 \rfloor + 1 = 3$；**C 正确**：叶子 4、5、6、7；**D 错误**：$n = 7$ 为奇数，$n_1 = 0$。
验算：$n_0 = 4 = n_2 + 1 = 3 + 1$ ✓。

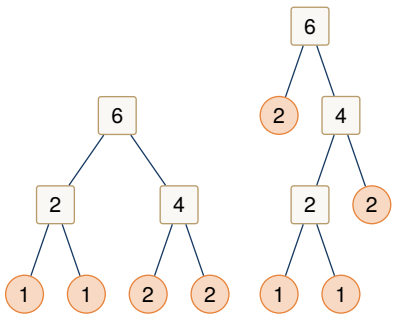
第 13 题　答案 AC　基础 [K7, K8]



蓝字：先序访问次序

A 正确：先序 A B D E G C F（见左图编号）。
B 错误：G 是 E 的**左**孩子，中序中 G 应在 E 之前，正确为 D B **G E** A C F。
C 正确：后序 D G E B F C A。
D 错误：第 3 层从左到右是 D、E、F，第 4 层是 G，层次序列应为 A B C D E F G；选项把 G 放到了 F 前面。
一句话：只有一个孩子时，先看清左右，再排中序。

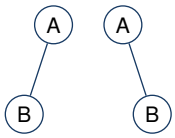
第 14 题　答案 ABD　基础 [K16, K17]



权 {1,1,2,2} 的两棵哈夫曼树，WPL 都是 12

A 正确：每个非叶结点都由两棵树合并而成。**B 正确**：n 个叶子，$n - 1$ 次合并。
C 错误：合并时 1+1=2 后，森林是 {2,2,2}，任选两个 2 合并都可以，得到左图两种不同树形，但 WPL 都等于 $2 + 4 + 6 = 12$。
D 正确：字符都在叶子上，从根到叶的路径不会经过别的叶子。

第 15 题　答案 ABD　基础 [K10]



C 的反例：先序均为 AB，后序均为 BA

A、B、D 正确：先序首（或后序尾、或层次首）给出根，中序把左右切开，递归即可唯一确定。
C 错误：左图两棵树先序、后序都相同，只有中序（BA 与 AB）能区分。笔记 p590 红字“先序 + 后序不可以!”。

11.3　基础填空题解析

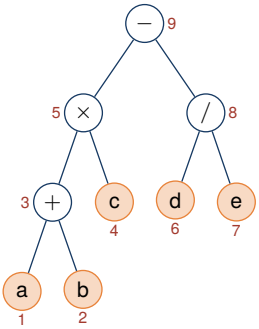
第 16 题　答案 37　基础 [K3]

$n_2 = n_0 - 1 = 14$，$n = n_0 + n_1 + n_2 = 15 + 8 + 14 = \mathbf{37}$。**易错**：直接写 $15 + 8 = 23$，漏了度为 2 的结点。

第 17 题　答案 31　基础 [K3 性质 2]

$2^5 - 1 = 31$（等比数列 $1 + 2 + 4 + 8 + 16$）。**易错**：写成 $2^5 = 32$ 或第 5 层的最大结点数 $2^4 = 16$。

第 18 题　答案 $a\,b + c \times d\,e / -$　基础 [K9 表达式树]

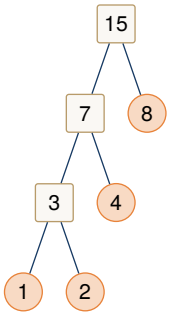


建树（课件 p72–73）：最后计算的运算符是“−”，它是根；左运算对象 $(a + b) \times c$，右运算对象 d/e，递归下去。
后序（红字为访问次序）：a b + c × d e / −。
检查：后缀式中运算符个数 4、操作数 5，运算符永远在它的两个运算对象之后 ✓。

第 19 题　答案 14　基础 [K12 卡特兰数]

$b_4 = b_0b_3 + b_1b_2 + b_2b_1 + b_3b_0 = 5 + 2 + 2 + 5 = 14$，或 $\frac{1}{5}\binom{8}{4} = \frac{70}{5} = 14$。四项分别对应“左子树 0、1、2、3 个结点”（课件 p75 的 (a)–(d) 四组）。

第 20 题　答案 25　基础 [K16]

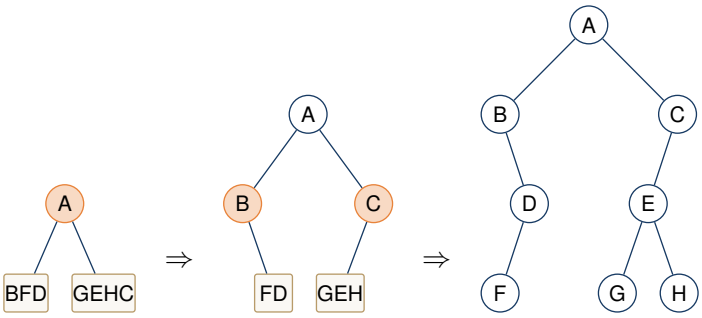


合并：$1+2=3, 3+4=7, 7+8=15$。$\mathrm{WPL} = 8\times1+4\times2+1\times3+2\times3 = 25$，验算 $3 + 7 + 15 = 25$ ✓。这正是课件 p109“构造方法 2”的树（方法 1 的 41 不是最优）。

11.4　基础解答题解析

第 21 题　后序 F D B G H E C A　基础 [K10]

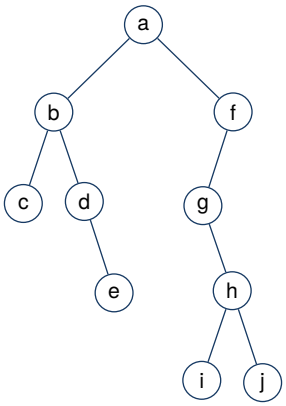
题意：先序 + 中序 → 建树 → 求后序。
第一步：根 A；中序 BFD | A | GEHC，左子树 3 个结点，先序 A 后取 3 个 BDF 为左子树先序，CEGH 为右子树先序。
第二步（左子树先序 BDF / 中序 BFD）：根 B，中序 B 左边为空 → 只有右子树 FD；FD 的先序 DF → 根 D，F 在 D 左边 → F 是 D 的左孩子。
第三步（右子树先序 CEGH / 中序 GEHC）：根 C，中序 C 右边为空 → 只有左子树 GEH；先序 EGH → 根 E，G 左、H 右。



结果：后序 **F D B G H E C A**。**检查**：对结果求先序 ABDFCEGH、中序 BFDAGEHC，均与题设一致（附录 A 程序输出相同）。
易错点：B 只有右子树、C 只有左子树，画图时方向别反。

第 22 题　先序 a b c d e f g h i j　基础 [K10]

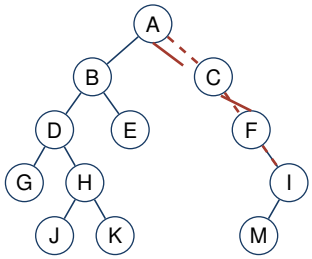
第一步：后序最后一个 a 为根；中序 cbde | a | gihjf。后序前 4 个 cedb 为左子树后序，ijhgf 为右子树后序。
第二步（左：中序 cbde / 后序 cedb）：根 b；c 为左孩子；右子树中序 de / 后序 ed → 根 d，e 是 d 的右孩子。
第三步（右：中序 gihjf / 后序 ijhgf）：根 f，中序 f 在末尾 → 只有左子树 gihj；后序 ijhg → 根 g，中序 g 在最前 → g 只有右子树 ihj；后序 ijh → 根 h，i 左、j 右。



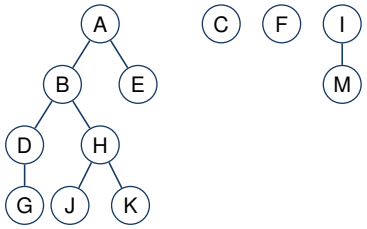
结果：先序 **a b c d e f g h i j**（恰好按字母顺序，可作为一个快速自检）。**检查**：中序 cbdeagihjf、后序 cedbijhgfa ✓。

第 23 题　森林：A(B(D(G),H(J,K)),E)，C，F，I(M)　基础 [K14]

方法（笔记 p599）：①沿根的右链抹线：A→C→F→I 依次断开，得到 4 棵二叉树；②每棵按“左孩子 = 第一个孩子，右孩子 = 下一个兄弟”还原。



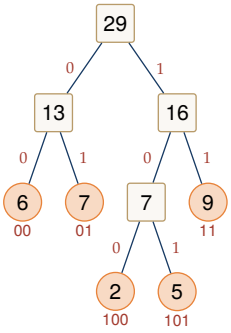
①抹去根右链 A–C–F–I 上的连线



②还原：4 棵树组成的森林

解释：B 的右孩子 E 是 B 的弟弟 → E 也是 A 的孩子；D 的右孩子 H 是 D 的弟弟 → H 也是 B 的孩子；J 的右孩子 K → K 也是 H 的孩子；I 的左孩子 M → M 是 I 的孩子。**检查**：结点总数 12 不变；森林先序 ABDGHJKE C F IM 与原二叉树先序 ABDGHJKECFIM 相同 ✓。

第 24 题　WPL = 65　基础 [K16, K17]

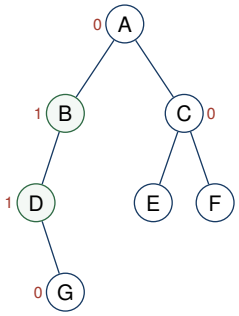


合并过程：{5,6,2,9,7} → 2+5=7 → {6,9,7,7} → 6+7=13 → {9,7,13} → 7+9=16 → {13,16} → 29。
WPL：$6\times2+7\times2+2\times3+5\times3+9\times2 = 12+14+6+15+18 = 65$；验算 $7+13+16+29 = 65$ ✓。
编码：6:00，7:01，2:100，5:101，9:11。
说明：第 2 步有两个权为 7 的结点（原始的 7 和新建的 7），选哪个与 6 合并都可以，树形和编码可能不同，但 WPL 都是 65（附录 A 程序输出一致）。

第 25 题　返回 2　基础 [K11 递归应用]

思路：套用“空树出口 + 递归左右 + 在根处合并”的模板，当前结点“恰有一个孩子”时计 1。

```
template <class ElemType>
int Degree1Count(const BinTreeNode<ElemType> *r) {
    if (r == NULL) return 0;                                  // 递归出口
    bool hasL = (r->leftChild != NULL), hasR = (r->rightChild != NULL);
    int self = (hasL != hasR) ? 1 : 0;                        // 恰有一个孩子
    return self + Degree1Count(r->leftChild) + Degree1Count(r->rightChild);
}
```



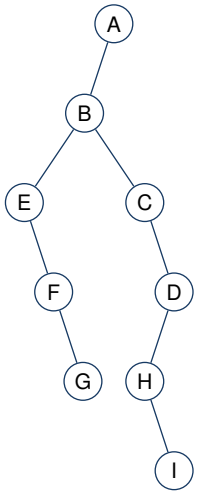
对图 6.9：B 只有左孩子 D、D 只有右孩子 G，返回 **2**（附录 A 实际运行输出“度为 1 的结点数 =2”）。
验算：$n = 7$，$n_0 = 3$（G、E、F），$n_2 = 2$（A、C），$n_1 = 7-3-2 = 2$ ✓。
同类题：求叶子数改判断为“两孩子都空”；求度 2 结点改为“两孩子都非空”。

11.5　提升单选题解析

第 26 题　答案 B　提升 [K3, K4]

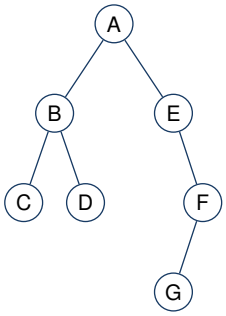
知识链：完全二叉树 $n_1 \in \{0,1\}$ → n 为偶数时 $n_1 = 1$ → 性质 3。
$n = 700$ 为偶数，$n_1 = 1$；$700 = n_0 + 1 + (n_0 - 1) = 2n_0$，$n_0 = \mathbf{350}$。**另一种方法**：叶子编号为 $\lfloor 700/2 \rfloor + 1 = 351$ 到 700，共 350 个。
A 错：误取 $n_1 = 0$ 后再减 1；**C 错**：套用奇数情形 $(n+1)/2$ 并取整；**D 错**：$n - 1$ 是边数。**本题卡点**：忘了“n 为偶数时恰有一个度为 1 的结点”。

第 27 题　答案 B　提升 [K14, K15]



第一步（转换）：左孩子右兄弟，得左图（即 §6.2 的例子）。
第二步（捷径）：树的后根遍历 = 对应二叉树的中序遍历。后根：E F G B C H I D A。选 **B**。
A 错：ABEFGCDHI 是树的先根 = 二叉树的先序。**C 错**：GFEIHDCBA 是二叉树的后序（与树的后根不同）。**D 错**：按字母排序，不是任何遍历。
若条件变化：问二叉树的先序，则答 A。

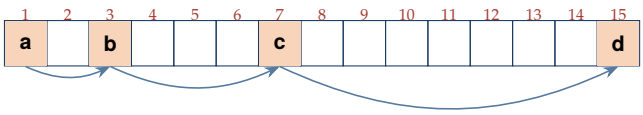
第 28 题　答案 A　提升 [K10, K7]



这是课件 p83 的例题。根 A；中序 CBD | A | EGF；左子树先序 BCD → B(C,D)；右子树先序 EFG、中序 EGF → E 无左子树，右子树先序 FG、中序 GF → F 的左孩子 G。后序：C D B G F E A，选 A。
B 错：把 G 当成 F 的右孩子或颠倒了 F、G 的次序；**C 错**：C、D 颠倒；**D 错**：把中序的左段 CBD 原样抄进后序。

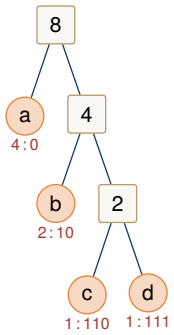
11.6　提升多选题解析

第 29 题　答案 ABD　提升 [K5, K4]



下标依次 $1 \to 2 \cdot 1 + 1 = 3 \to 7 \to 15$。**A 正确**：最大下标 15，即 $2^4 - 1$；**B 正确**：只有 4 个单元有数据，浪费 11 个；**C 错误**：8 是 2×4，但 d 是 c（下标 7）的右孩子，下标 15；**D 正确**。**本题想说明**：顺序存储对单支树极不划算（笔记 p586）。

第 30 题　答案 ABC　提升 [K16, K17]



合并 1+1=2，2+2=4，4+4=8。
A 正确：a 码“0”，长 1。**B 正确**：总长 $4\cdot1+2\cdot2+1\cdot3+1\cdot3=14$ = WPL $=2+4+8$。**C 正确**：c、d 都是 3 位（兄弟叶子同深度）。**D 错误**：4 个字符等长编码要 2 位，总长 $8\times2=16>14$。
若条件变化：四个字符频率相同 (2,2,2,2)，哈夫曼树就是满二叉树，与等长编码一样长。

11.7　提升填空题解析

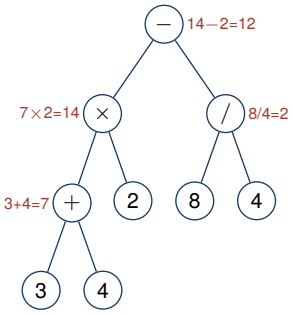
第 31 题　答案 100　提升 [K16, K3]

哈夫曼树结点数 $2n-1=199$，故 $n=100$。也可以由“无度 1 结点”与 $n_0=n_2+1$：$n_0+n_2=2n_0-1=199$。**易错**：写成 $199/2\approx99$。

第 32 题　答案 271　提升 [K16]

即例题 7.3（课件 p111–116）。合并得新结点 8, 15, 19, 29, 42, 58, 100，WPL $=8+15+19+29+42+58+100=$ **271**。**特别提醒**：课件 p116 把 3×11 算成 15，总和写成 253，是错误的（见纠错框）。

第 33 题　答案 12；4　提升 [K9, K11]



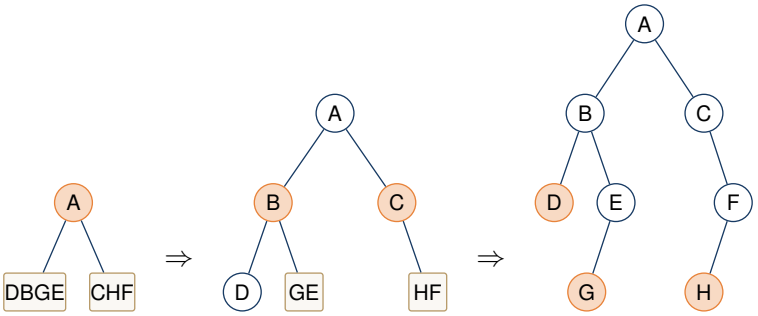
后序求值：后序 3 4 + 2 × 8 4 / −。左子树：$3+4=7$，$7\times2=14$；右子树：$8/4=2$；根：$14-2=$ **12**（附录 A 的 `Evaluate` 输出 12）。
高度：− → × → + → 3 共 4 层，HeightHelp 返回 **4**。
易错：按中序 $3+4\times2-8/4$ 直接计算得 9——中序丢失了括号。

11.8　提升综合大题解析

第 34 题　提升 [K10, K7, K8, K3, K6]

题目在问什么：中序 + 后序 → 建树；再用这棵树做遍历、性质验证、空指针计数。**知识链**：序列建树 → 遍历 → 性质 3 → 二叉链表。

(1) 建树。后序末尾 A 为根；中序 DBGE | A | CHF。左子树后序取前 4 个 DGEB，根 B；中序 D | B | GE → D 为左孩子；GE 的后序 GE → 根 E，G 在 E 左边，是 E 的左孩子。右子树后序 HFC，根 C；中序 C 在最前 → C 无左子树，右子树中序 HF / 后序 HF → 根 F，H 是 F 的左孩子。



(2) 先序 **A B D E G C F H**；层次（第 1 层 A，第 2 层 B C，第 3 层 D E F，第 4 层 G H）**A B C D E F G H**。

(3) 叶子 D、G、H：$n_0 = 3$；只有一个孩子的 E、C、F：$n_1 = 3$；两个孩子的 A、B：$n_2 = 2$。$n_0 = 3 = n_2 + 1$ ✓；$n = 3 + 3 + 2 = 8$ ✓。

(4) 空指针 $n + 1 = 9$；逐个数：D 两个、G 两个、H 两个、E 右、C 左、F 右，共 9 ✓。

结果检查：对所建树求中序 DBGEACHF、后序 DGEBHFCA，与题设一致（已用程序验证）。

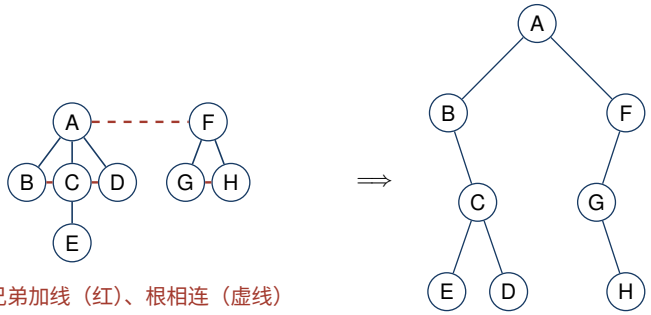
最容易卡在哪里：右子树后序要取“紧接左子树之后、去掉最后根”的一段（HFC 中的 HF），不能从中序直接抄。

变式思考：若只给出先序 ABDEGCFH 和后序 DGEBHFCA，能否确定这棵树？（不能：C、E、F 都只有一个孩子，左右无法判断。）

第 35 题　提升　[K13, K14, K15]

题目在问什么：森林 → 二叉树，并验证“森林先序 = 二叉树先序，森林中序 = 二叉树中序”。

(1) 转换：每棵树内“左孩子右兄弟”；两棵树的根 A、F 视为兄弟，F 成为 A 的右孩子。



(2) 森林先序：依次先根遍历各树 → A B C E D | F G H = **ABCEDFGH**；二叉树先序也是 ABCEDFGH ✓。

森林中序：依次后根遍历各树 → B E C D A | G H F = **BECDAGHF**；二叉树中序也是 BECDAGHF ✓（已用程序验证）。

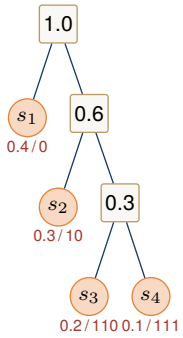
(3) **不为空**，A 的右指针指向 F。因为这是森林：A 有“兄弟”——第二棵树的根 F。只有单独一棵树转换时根的右指针才一定为空（对比第 6.1 例题）。

最容易卡在哪里：C 的孩子 E 应挂在 C 的左边，C 的弟弟 D 挂在 C 的右边。

变式思考：若再加第三棵树 T_3 = K，K 会出现在哪里？（F 的右孩子。）

11.9　跨学科迁移题解析

第 36 题　答案 A　跨学科 [K16, K17]



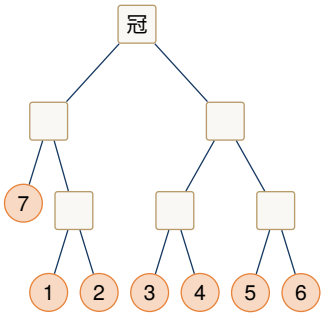
当前专题知识：哈夫曼树构造与 WPL。**借用的外部情境**：通信中的平均码长（概率论中的期望）。**为什么不算超纲**：题干已给出 $\bar{L}=\sum p_i l_i$，它就是以概率为权的 WPL。

解：$0.1+0.2=0.3$，$0.3+0.3=0.6$，$0.4+0.6=1.0$。码长 1, 2, 3, 3；$\bar{L}=0.4+0.6+0.6+0.3=$ **1.9** 位。**验算**：非叶结点权之和 $0.3+0.6+1.0=1.9$ ✓。

B 错：等长编码的码长；**C 错**：码长 1,2,3,3 与概率配反（大概率配长码）；**D 错**：这是概率 $\frac{1}{2},\frac{1}{4},\frac{1}{8},\frac{1}{8}$ 时的结果（§7 对照框），与本题数据不符。

迁移点：课本里“权”可以是次数，也可以是概率；后者得到的 WPL 就是平均码长。

第 37 题　答案 ABC　跨学科 [K3]



7 队、轮空队放在最左：比赛 $6=7-1$ 场，但编号 8、9 空缺，不是完全二叉树

当前专题知识：性质 3 $n_0=n_2+1$、满二叉树深度。**外部情境**：赛事组织；题干已说明“球队为叶子、比赛为分支结点”。

A 正确：每场比赛恰有两个来源，是度为 2 的结点；$n_2=n_0-1=12$。（直观：每场淘汰一队，13 队要淘汰 12 队。）

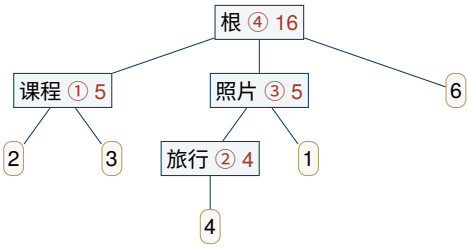
B 正确：轮空的队直接晋级，不产生新结点，所以不存在“只有一个孩子”的比赛。

C 正确：16 队时对阵树是 16 个叶子的满二叉树，深度 5，冠军从叶子到根经过 4 条边，即赢 4 场。

D 错误：完全二叉树要求按层编号无空位，而对阵树的形状取决于轮空安排。左图以 7 队为例：把轮空的 7 号放在最左边，它在第 3 层（编号 4）就是叶子，第 4 层编号 8、9 空缺而 10–15 有结点，所以不是完全二叉树。

迁移点：“每次分叉多一片叶子”在赛程中变成“每场比赛淘汰一队”。

第 38 题　答案课程；16　跨学科 [K15, K11]



红色：文件夹被算出的次序与大小

当前专题知识：树的后根遍历（先依次处理各子树，再访问根）。**外部情境**：操作系统统计目录大小；题干已给出“先子项、后父文件夹”的规则，这就是后根遍历的定义，不需要任何操作系统知识。

解：后根顺序访问文件夹：课程（$2+3=5$）→ 旅行（4）→ 照片（$4+1=5$）→ 根（$5+5+6=16$）。第一个算出的是**课程**，根总大小 **16 MB**。

易错：误按先根顺序认为“根”最先——根恰恰是最后一个才能算出的。

迁移点：二叉树中的“求高度、求结点数”都是同样的“孩子先、双亲后”。

12　后续课程连接

本章知识	后续在哪里再出现	课程/章节
层次遍历（队列）	图的广度优先搜索、最短路径	第 7 章图
先序遍历（递归/栈）	图的深度优先搜索	第 7 章图
中序遍历	二叉排序树中序有序、平衡二叉树	查找
完全二叉树顺序存储	堆、堆排序、优先队列 `std::priority_queue`	排序
$n+1$ 个空指针	线索二叉树（教材 6.4）	本章拓展
表达式树	语法分析树、抽象语法树	编译原理
树/森林、目录树	文件系统目录结构	操作系统
哈夫曼编码	信源编码、熵与平均码长	信息论、通信原理
卡特兰数	组合计数、生成函数	组合数学

附录 A　考试代码合集（C++，已编译运行）

以下代码与课件、课程源程序写法一致（类模板 + `Help` 递归辅助函数 + 链队列），把本讲义用到的全部算法整合为一个可直接编译的文件 **二叉树 _ 考试代码合集.cpp**（随讲义一起提供）。编译命令：`g++ -std=c++11 二叉树 _ 考试代码合集.cpp -o tree`。已在 g++ 下编译通过，并用 valgrind 检查无内存泄漏。

```cpp
/*
 * 《数据结构与算法》第6章 树与二叉树 —— 考试代码合集（C++）
 * 与课件、教材源程序的写法保持一致：类模板 + Help 递归辅助函数 + LinkQueue。
 * 编译：g++ -std=c++11 二叉树_考试代码合集.cpp -o tree
 * 本文件中的每一段代码都在讲义中被引用，并已实际编译运行。
 */
#include <iostream>
#include <cstring>
using namespace std;

// ==================== 链队列（第3章，层次遍历要用） ====================
template <class ElemType>
struct Node {
    ElemType data;
    Node<ElemType> *next;
    Node() { next = NULL; }
    Node(ElemType e, Node<ElemType> *link = NULL) { data = e; next = link; }
};

template <class ElemType>
class LinkQueue {
protected:
    Node<ElemType> *front, *rear;          // front 指向头结点
public:
    LinkQueue() { rear = front = new Node<ElemType>; }
    virtual ~LinkQueue() { Clear(); delete front; }
    bool Empty() const { return rear == front; }
    void Clear() { ElemType tmp; while (!Empty()) OutQueue(tmp); }
    bool OutQueue(ElemType &e) {
        if (Empty()) return false;
        Node<ElemType> *p = front->next;    // 队头元素
        e = p->data;
        front->next = p->next;
        if (rear == p) rear = front;        // 出队前只有一个元素
        delete p;
        return true;
    }
    bool InQueue(const ElemType &e) {
        Node<ElemType> *p = new Node<ElemType>(e);
```

```
        rear->next = p; rear = p;
        return true;
    }
};

// ==================== 二叉链表结点 ====================
template <class ElemType>
struct BinTreeNode {
    ElemType data;                          // 数据域
    BinTreeNode<ElemType> *leftChild;       // 左孩子指针
    BinTreeNode<ElemType> *rightChild;      // 右孩子指针
    BinTreeNode() { leftChild = rightChild = NULL; }
    BinTreeNode(const ElemType &e, BinTreeNode<ElemType> *l = NULL,
                BinTreeNode<ElemType> *r = NULL) { data = e; leftChild = l; rightChild = r; }
};

// ==================== 二叉链表二叉树类（课件 6.2.3） ====================
template <class ElemType>
class BinaryTree {
protected:
    BinTreeNode<ElemType> *root;
    // ---- 递归辅助函数：都以"某棵子树的根 r"为参数 ----
    void PreOrderHelp(const BinTreeNode<ElemType> *r, void (*visit)(const ElemType &)) const {
        if (r != NULL) {
            (*visit)(r->data);                      // D：访问根
            PreOrderHelp(r->leftChild, visit);      // L：先序遍历左子树
            PreOrderHelp(r->rightChild, visit);     // R：先序遍历右子树
        }
    }
    void InOrderHelp(const BinTreeNode<ElemType> *r, void (*visit)(const ElemType &)) const {
        if (r != NULL) {
            InOrderHelp(r->leftChild, visit);       // L
            (*visit)(r->data);                      // D
            InOrderHelp(r->rightChild, visit);      // R
        }
    }
    void PostOrderHelp(const BinTreeNode<ElemType> *r, void (*visit)(const ElemType &)) const {
        if (r != NULL) {
            PostOrderHelp(r->leftChild, visit);     // L
            PostOrderHelp(r->rightChild, visit);    // R
            (*visit)(r->data);                      // D
        }
    }
    int HeightHelp(const BinTreeNode<ElemType> *r) const {
        if (r == NULL) return 0;                    // 空树高为 0
        int lh = HeightHelp(r->leftChild);
        int rh = HeightHelp(r->rightChild);
        return (lh > rh ? lh : rh) + 1;             // 较高子树 + 1
    }
    int NodeCountHelp(const BinTreeNode<ElemType> *r) const {
        if (r == NULL) return 0;
        return NodeCountHelp(r->leftChild) + NodeCountHelp(r->rightChild) + 1;
    }
    int LeafCountHelp(const BinTreeNode<ElemType> *r) const {   // 课堂笔记：求叶子数
        if (r == NULL) return 0;                                 // 空树：0 个叶子
        if (r->leftChild == NULL && r->rightChild == NULL) return 1; // r 本身是叶子
        return LeafCountHelp(r->leftChild) + LeafCountHelp(r->rightChild);
    }
    ElemType SumHelp(const BinTreeNode<ElemType> *r) const {     // 课件作业7：结点值之和
        if (r == NULL) return 0;
        return r->data + SumHelp(r->leftChild) + SumHelp(r->rightChild);
    }
    BinTreeNode<ElemType> *CopyTreeHelp(const BinTreeNode<ElemType> *r) {
        if (r == NULL) return NULL;
```

```
104         BinTreeNode<ElemType> *l = CopyTreeHelp(r->leftChild);   // 复制左子树
105         BinTreeNode<ElemType> *rr = CopyTreeHelp(r->rightChild); // 复制右子树
106         return new BinTreeNode<ElemType>(r->data, l, rr);        // 复制根
107     }
108     void DestroyHelp(BinTreeNode<ElemType> *&r) {                // 后序销毁
109         if (r != NULL) {
110             DestroyHelp(r->leftChild);
111             DestroyHelp(r->rightChild);
112             delete r;
113             r = NULL;
114         }
115     }
116     const BinTreeNode<ElemType> *ParentHelp(const BinTreeNode<ElemType> *r,
117                                             const BinTreeNode<ElemType> *cur) const {
118         if (r == NULL) return NULL;
119         if (r->leftChild == cur || r->rightChild == cur) return r;
120         const BinTreeNode<ElemType> *p = ParentHelp(r->leftChild, cur);
121         if (p != NULL) return p;
122         return ParentHelp(r->rightChild, cur);
123     }
124 public:
125     BinaryTree() { root = NULL; }
126     BinaryTree(const ElemType &e) { root = new BinTreeNode<ElemType>(e); }
127     BinaryTree(BinTreeNode<ElemType> *r) { root = r; }
128     BinaryTree(const BinaryTree<ElemType> &src) { root = CopyTreeHelp(src.root); }
129     BinaryTree<ElemType> &operator=(const BinaryTree<ElemType> &src) {
130         if (&src != this) { DestroyHelp(root); root = CopyTreeHelp(src.root); }
131         return *this;
132     }
133     virtual ~BinaryTree() { DestroyHelp(root); }
134     BinTreeNode<ElemType> *GetRoot() const { return root; }
135     bool Empty() const { return root == NULL; }
136     void PreOrder(void (*visit)(const ElemType &)) const { PreOrderHelp(root, visit); }
137     void InOrder(void (*visit)(const ElemType &)) const { InOrderHelp(root, visit); }
138     void PostOrder(void (*visit)(const ElemType &)) const { PostOrderHelp(root, visit); }
139     void LevelOrder(void (*visit)(const ElemType &)) const {
140         LinkQueue<const BinTreeNode<ElemType> *> q;
141         const BinTreeNode<ElemType> *cur;
142         if (root != NULL) q.InQueue(root);          // 根入队
143         while (!q.Empty()) {
144             q.OutQueue(cur);                        // 出队一个
145             (*visit)(cur->data);                    // 访问它
146             if (cur->leftChild != NULL) q.InQueue(cur->leftChild);   // 左孩子入队
147             if (cur->rightChild != NULL) q.InQueue(cur->rightChild); // 右孩子入队
148         }
149     }
150     int Height() const { return HeightHelp(root); }
151     int NodeCount() const { return NodeCountHelp(root); }
152     int LeafCount() const { return LeafCountHelp(root); }
153     ElemType Sum() const { return SumHelp(root); }
154     const BinTreeNode<ElemType> *Parent(const BinTreeNode<ElemType> *cur) const { return ParentHelp(root, cur); }
155     void InsertLeftChild(BinTreeNode<ElemType> *cur, const ElemType &e) {
156         if (cur == NULL) return;
157         BinTreeNode<ElemType> *child = new BinTreeNode<ElemType>(e);
158         child->leftChild = cur->leftChild;          // 原左子树成为 e 的左子树
159         cur->leftChild = child;
160     }
161     void InsertRightChild(BinTreeNode<ElemType> *cur, const ElemType &e) {
162         if (cur == NULL) return;
163         BinTreeNode<ElemType> *child = new BinTreeNode<ElemType>(e);
164         child->rightChild = cur->rightChild;        // 原右子树成为 e 的右子树
165         cur->rightChild = child;
166     }
167     void DeleteLeftChild(BinTreeNode<ElemType> *cur) { if (cur != NULL) DestroyHelp(cur->leftChild); }
```

```
    void DeleteRightChild(BinTreeNode<ElemType> *cur) { if (cur != NULL) DestroyHelp(cur->rightChild); }
};

// ==================== 练习第25题：度为1的结点个数 ====================
template <class ElemType>
int Degree1Count(const BinTreeNode<ElemType> *r) {
    if (r == NULL) return 0;                                    // 空树：0 个
    bool hasL = (r->leftChild != NULL), hasR = (r->rightChild != NULL);
    int self = (hasL != hasR) ? 1 : 0;                          // 恰有一个孩子时计 1
    return self + Degree1Count(r->leftChild) + Degree1Count(r->rightChild);
}

// ==================== 由先序 + 中序构造（课件 6.3.3-6） ====================
template <class ElemType>
void CreateBinaryTreeHelp(BinTreeNode<ElemType> *&r, ElemType pre[], ElemType in[],
                          int preLeft, int preRight, int inLeft, int inRight) {
    if (preLeft > preRight || inLeft > inRight) { r = NULL; return; }   // 空子树
    r = new BinTreeNode<ElemType>(pre[preLeft]);     // 先序第一个 = 根
    int mid = inLeft;
    while (in[mid] != pre[preLeft]) mid++;           // 在中序里找根
    int leftLen = mid - inLeft;                      // 左子树结点个数
    CreateBinaryTreeHelp(r->leftChild, pre, in, preLeft + 1, preLeft + leftLen, inLeft, mid - 1);
    CreateBinaryTreeHelp(r->rightChild, pre, in, preLeft + leftLen + 1, preRight, mid + 1, inRight);
}
template <class ElemType>
BinTreeNode<ElemType> *CreateBinaryTree(ElemType pre[], ElemType in[], int n) {
    BinTreeNode<ElemType> *r;
    CreateBinaryTreeHelp<ElemType>(r, pre, in, 0, n - 1, 0, n - 1);
    return r;                    // 返回根指针，交给 BinaryTree(r) 管理，避免内存泄漏
}

// ==================== 由中序 + 后序构造（笔记例题，课件主观题4） ====================
template <class ElemType>
void CreateByInPostHelp(BinTreeNode<ElemType> *&r, ElemType in[], ElemType post[],
                        int inLeft, int inRight, int postLeft, int postRight) {
    if (inLeft > inRight) { r = NULL; return; }
    r = new BinTreeNode<ElemType>(post[postRight]);  // 后序最后一个 = 根
    int mid = inLeft;
    while (in[mid] != post[postRight]) mid++;
    int leftLen = mid - inLeft;
    CreateByInPostHelp(r->leftChild, in, post, inLeft, mid - 1, postLeft, postLeft + leftLen - 1);
    CreateByInPostHelp(r->rightChild, in, post, mid + 1, inRight, postLeft + leftLen, postRight - 1);
}

// ==================== 由带空标记 '#' 的先序串建树（常见考题） ====================
BinTreeNode<char> *CreateByPreString(const char *s, int &i) {
    char c = s[i++];
    if (c == '#' || c == '\0') return NULL;          // '#' 表示空子树
    BinTreeNode<char> *r = new BinTreeNode<char>(c);
    r->leftChild = CreateByPreString(s, i);
    r->rightChild = CreateByPreString(s, i);
    return r;
}

// ==================== 表达式树求值（后序思想） ====================
double Evaluate(const BinTreeNode<char> *r) {
    if (r->leftChild == NULL && r->rightChild == NULL)
        return r->data - '0';                        // 叶子是一位数字操作数
    double a = Evaluate(r->leftChild);               // 先算左子树
    double b = Evaluate(r->rightChild);              // 再算右子树
    switch (r->data) {                               // 最后"访问根" = 做运算
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
```

```
        default:  return a / b;
    }
}

// ==================== 树的双亲表示法（课件 6.5.1） ====================
template <class ElemType>
struct ParentTreeNode {
    ElemType data;       // 数据域
    int parent;          // 双亲在数组中的下标，根为 -1
};

// ==================== 孩子兄弟表示法（= 二叉链表表示树） ====================
template <class ElemType>
struct ChildSiblingNode {
    ElemType data;
    ChildSiblingNode<ElemType> *firstChild;   // 第一个孩子（相当于二叉树的左指针）
    ChildSiblingNode<ElemType> *nextSibling;  // 下一个兄弟（相当于二叉树的右指针）
    ChildSiblingNode(ElemType e) { data = e; firstChild = nextSibling = NULL; }
};
template <class ElemType>
void DestroyCS(ChildSiblingNode<ElemType> *r) {             // 后根次序释放
    if (r == NULL) return;
    DestroyCS(r->firstChild); DestroyCS(r->nextSibling);
    delete r;
}
template <class ElemType>
void TreePreRoot(const ChildSiblingNode<ElemType> *r) {     // 树的先根遍历
    if (r == NULL) return;
    cout << r->data << " ";                                  // 先访问根
    for (const ChildSiblingNode<ElemType> *c = r->firstChild; c != NULL; c = c->nextSibling)
        TreePreRoot(c);                                      // 再依次先根遍历各子树
}
template <class ElemType>
void TreePostRoot(const ChildSiblingNode<ElemType> *r) {    // 树的后根遍历
    if (r == NULL) return;
    for (const ChildSiblingNode<ElemType> *c = r->firstChild; c != NULL; c = c->nextSibling)
        TreePostRoot(c);                                     // 先依次后根遍历各子树
    cout << r->data << " ";                                  // 最后访问根
}

// ==================== 哈夫曼树（拓展：课件"实现不讲"，此处给最简数组版） ====================
struct HuffNode { int weight, parent, left, right; };
// w[0..n-1] 为叶子权值；结果存入 ht[0..2n-2]，返回 WPL
int BuildHuffman(const int w[], int n, HuffNode ht[]) {
    for (int i = 0; i < 2 * n - 1; i++) {
        ht[i].weight = (i < n) ? w[i] : 0;
        ht[i].parent = ht[i].left = ht[i].right = -1;
    }
    int wpl = 0;
    for (int k = n; k < 2 * n - 1; k++) {            // 共做 n-1 次合并
        int s1 = -1, s2 = -1;                        // 选两个"无双亲且权最小"的根
        for (int i = 0; i < k; i++) {
            if (ht[i].parent != -1) continue;
            if (s1 == -1 || ht[i].weight < ht[s1].weight) { s2 = s1; s1 = i; }
            else if (s2 == -1 || ht[i].weight < ht[s2].weight) s2 = i;
        }
        ht[k].weight = ht[s1].weight + ht[s2].weight;  // 新根权值 = 两者之和
        ht[k].left = s1; ht[k].right = s2;
        ht[s1].parent = ht[s2].parent = k;
        wpl += ht[k].weight;                         // WPL = 所有新建结点权值之和
    }
    return wpl;
}
// 从叶子 i 沿 parent 走到根，得到其哈夫曼编码（左 0 右 1）
```

```
296 void HuffCode(const HuffNode ht[], int i, char code[]) {
297     char tmp[64]; int len = 0;
298     for (int c = i, p = ht[i].parent; p != -1; c = p, p = ht[p].parent)
299         tmp[len++] = (ht[p].left == c) ? '0' : '1';  // 自下而上记录
300     for (int k = 0; k < len; k++) code[k] = tmp[len - 1 - k];  // 反转
301     code[len] = '\0';
302 }
303
304 // ==================== 测试 ====================
305 void ShowChar(const char &e) { cout << e << " "; }
306 void ShowInt(const int &e) { cout << e << " "; }
307
308 int main() {
309     // 1. 教材图6.9：A(B(D(∧,G),∧), C(E,F))
310     int i = 0;
311     BinaryTree<char> bt(CreateByPreString("ABD#G###CE##F##", i));
312     cout << "先序: "; bt.PreOrder(ShowChar); cout << endl;
313     cout << "中序: "; bt.InOrder(ShowChar); cout << endl;
314     cout << "后序: "; bt.PostOrder(ShowChar); cout << endl;
315     cout << "层次: "; bt.LevelOrder(ShowChar); cout << endl;
316     cout << "结点数=" << bt.NodeCount() << " 叶子数=" << bt.LeafCount()
317          << " 高=" << bt.Height() << " 度为1的结点数=" << Degree1Count(bt.GetRoot()) << endl;
318
319     // 2. 课件主观题3：先序 ABDFCEGH + 中序 BFDAGEHC
320     char pre[] = "ABDFCEGH", in1[] = "BFDAGEHC";
321     BinaryTree<char> t3(CreateBinaryTree<char>(pre, in1, 8));
322     cout << "主观题3 后序: "; t3.PostOrder(ShowChar); cout << endl;
323
324     // 3. 课件主观题4：中序 cbdeagihjf + 后序 cedbijhgfa
325     char in2[] = "cbdeagihjf", post2[] = "cedbijhgfa";
326     BinTreeNode<char> *r4;
327     CreateByInPostHelp<char>(r4, in2, post2, 0, 9, 0, 9);
328     BinaryTree<char> t4(r4);
329     cout << "主观题4 先序: "; t4.PreOrder(ShowChar); cout << endl;
330
331     // 4. 作业7：整数二叉树结点值之和   1(2(4,5),3(,6))
332     BinaryTree<int> it(1);
333     BinTreeNode<int> *p = it.GetRoot();
334     it.InsertLeftChild(p, 2); it.InsertRightChild(p, 3);
335     it.InsertLeftChild(p->leftChild, 4); it.InsertRightChild(p->leftChild, 5);
336     it.InsertRightChild(p->rightChild, 6);
337     cout << "作业7 结点值之和 = " << it.Sum() << endl;
338
339     // 5. 表达式树 (3+4)*2-8/4 ：先序串 -*+34##2##/8##4## 的写法见讲义
340     i = 0;
341     BinTreeNode<char> *e = CreateByPreString("-*+3##4##2##/8##4##", i);
342     BinaryTree<char> et(e);
343     cout << "后缀式: "; et.PostOrder(ShowChar); cout << " 值 = " << Evaluate(e) << endl;
344
345     // 6. 树 A(B(E,F,G),C,D(H,I)) 用孩子兄弟表示，先根/后根遍历
346     ChildSiblingNode<char> *A = new ChildSiblingNode<char>('A'), *Bn = new ChildSiblingNode<char>('B'),
347         *C = new ChildSiblingNode<char>('C'), *D = new ChildSiblingNode<char>('D'),
348         *E = new ChildSiblingNode<char>('E'), *F = new ChildSiblingNode<char>('F'),
349         *G = new ChildSiblingNode<char>('G'), *Hn = new ChildSiblingNode<char>('H'),
350         *I = new ChildSiblingNode<char>('I');
351     A->firstChild = Bn; Bn->nextSibling = C; C->nextSibling = D;
352     Bn->firstChild = E; E->nextSibling = F; F->nextSibling = G;
353     D->firstChild = Hn; Hn->nextSibling = I;
354     cout << "树先根: "; TreePreRoot(A); cout << endl;
355     cout << "树后根: "; TreePostRoot(A); cout << endl;
356     DestroyCS(A);
357
358     // 7. 哈夫曼：课件主观题 W={5,6,2,9,7}
359     int w[] = {5, 6, 2, 9, 7};
```

```
    HuffNode ht[9];
    cout << "哈夫曼 WPL = " << BuildHuffman(w, 5, ht) << endl;
    char code[64];
    for (int k = 0; k < 5; k++) { HuffCode(ht, k, code); cout << w[k] << ":" << code << " "; }
    cout << endl;

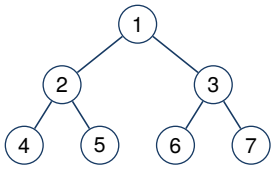
    // 8. 双亲表示法（课件 6.5.1）：A(B,C(E,F(G)),D)
    ParentTreeNode<char> pt[7] = {{'A',-1},{'B',0},{'C',0},{'D',0},{'E',2},{'F',2},{'G',5}};
    cout << "G 的祖先: ";
    for (int k = pt[6].parent; k != -1; k = pt[k].parent) cout << pt[k].data << " ";
    cout << endl;
    return 0;
}
```

C++：运行结果（实际输出）

```
先序: A B D G C E F
中序: D G B A E C F
后序: G D B E F C A
层次: A B C D E F G
结点数=7 叶子数=3 高=4 度为1的结点数=2
主观题3 后序: F D B G H E C A
主观题4 先序: a b c d e f g h i j
作业7 结点值之和 = 21
后缀式: 3 4 + 2 * 8 4 / -  值 = 12
树先根: A B E F G C D H I
树后根: E F G B C H I D A
哈夫曼 WPL = 65
5:101 6:00 2:100 9:11 7:01
G 的祖先: F C A
```

附录 B　拓展：非递归中序遍历（课件附录，标注“不讲”）

递归的本质是系统帮我们维护了一个“调用栈”。如果自己用一个栈保存“还没访问的祖先”，就能写出非递归遍历。课件 p125–150 的动画演示了对树 1(2(4,5),3(6,7)) 的过程：沿左链一路入栈 1、2、4 → 弹出 4 访问 → 弹出 2 访问，转向右子树 5 入栈 → ……最终中序输出 **4,2,5,1,6,3,7**，先序输出 **1,2,4,5,3,6,7**。



栈底 → 顶	输出
[1,2,4]	
[1,2]	4
[1]	4,2
[1,5]	4,2
[1]	4,2,5
[]	4,2,5,1
[3,6]	4,2,5,1
[3]	4,2,5,1,6
[]	4,2,5,1,6,3
[7]	…
[]	4,2,5,1,6,3,7

C++：非递归中序遍历（据课件 p138–140 整理，使用 std::stack 以便独立编译）

```
#include <stack>
template <class ElemType>
void NonRecurInOrder(const BinTreeNode<ElemType> *root, void (*visit)(const ElemType &))
    {
    std::stack<const BinTreeNode<ElemType> *> s;
    const BinTreeNode<ElemType> *cur = root;
```

```
    while (cur != NULL || !s.empty()) {
        while (cur != NULL) { s.push(cur); cur = cur->leftChild; }  // 一路向左入栈
        cur = s.top(); s.pop();                                      // 最左结点出栈
        (*visit)(cur->data);                                         // 访问
        cur = cur->rightChild;                                       // 转向右子树
    }
}
```

课件版本用 `GoFarLeft` 函数封装“一路向左入栈”，逻辑相同。这段代码已单独编译测试，对图 6.9 输出 D G B A E C F。

附录 C　参考资料

原始教学依据

- 某高校　《数据结构与算法》课件《06 第 6 章：树与二叉树》（150 页）；课堂笔记（2026 年 10 月，28 页）；课程源程序：二叉链表/顺序存储二叉树、先序/中序/后序/层次遍历（共 6 份）。

核心定义与公式核验

- NIST Dictionary of Algorithms and Data Structures：full binary tree、perfect binary tree、complete binary tree 词条。https://xlinux.nist.gov/dads/
- T. H. Cormen 等，*Introduction to Algorithms*（CLRS），附录 B.5 树、第 16.3 节哈夫曼编码。
- D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, 40(9): 1098–1101, 1952. DOI: 10.1109/JRPROC.1952.273898.

工程应用核验

- P. Deutsch, RFC 1951 *DEFLATE Compressed Data Format Specification* v1.3, 1996. https://www.rfc-editor.org/rfc/rfc1951
- ITU-T T.81 (ISO/IEC 10918-1) JPEG 标准，§4.6 基线顺序编码使用哈夫曼编码。
- WHATWG DOM Standard, §1.1 Trees: “tree order is preorder, depth-first traversal”. https://dom.spec.whatwg.org/
- GNU coreutils `du` 源码注释（目录在其全部条目处理完后才汇总）。
- cppreference：`std::priority_queue`、`std::push_heap`。https://en.cppreference.com/
- ITU-R M.1677-1 国际摩尔斯电码（字母与单词间隔的规定）。

学科史资料

- A. Cayley, “On the theory of the analytical forms called trees,” *Philosophical Magazine*, 13: 172–176, 1857.
- I. Pak, “History of Catalan numbers,” arXiv:1408.5711（含欧拉 1751 年信件、卡特兰 1838 年论文及命名史）。
- A. W. Burks, D. W. Warren, J. B. Wright, “An analysis of a logical machine using parenthesis-free notation,” *MTAC* 8(46): 53–57, 1954.
- J. W. J. Williams, “Algorithm 232: Heapsort,” *Communications of the ACM*, 7(6): 347–348, 1964.
- D. E. Knuth, *The Art of Computer Programming*, Vol. 1, 1968.